

Contents

核心速查版	4
核心目录	4
OPS-00 第 -1 卷：统一接口与标准容器	4
0. 考场使用顺序	5
1. 全局 C++17 骨架	5
2. 函数命名规范	6
3. Graph 协议	6
4. Array 协议	8
5. Query 协议	9
6. Compressor 协议	9
7. State 协议	10
8. 模块契约卡	11
9. 最小自检	11
OPS-01 第 0 卷：考场作战流程	12
1. 考场总原则	12
2. 3 小时时间策略	12
3. 32 次提交策略	13
4. 先交部分再升级	13
5. 提交前检查清单	14
6. 完全不会时兜底策略	14
7. 卡住时的 5 分钟复位	15
ROUTE-00 第 0A 卷：题面路由表	15
1. 3 分钟路由动作	16
2. 题面信号路由表	16
3. 数据范围预算表	17
4. 操作类型路由表	18
5. 路由冲突时怎么判	18
CPP-001 C++17 主骨架与输入输出	18
CPP-10：输入输出与格式化	21
考场目标	22
1. 先选 IO 套路	22
1A. 分隔方式总表	22
空格和换行等价的例子	23
行边界有意义的例子	23
2. cin/cout 常用片段	23
快速 cin/cout 骨架	23
多组数据	23
EOF 读到输入末尾就停止	24
小数保留位数	25
右对齐、左对齐、补零	25
3. scanf/printf 常用片段	25
基础骨架	25
常用格式符	25
小数、宽度、补零	26
4. 自写快读快写	26
5. 字符、整行与换行处理	27
读一个非空白字符	27
读下一个原始字符	28
读整行	28
6. 复杂格式处理	28
标点固定：用 char 接住	28
一行里有不定个整数	28
简单逗号分隔	29
key=value 或 key: value	29
读带空格的名字和值	29
网格读入	29
7. 提交前格式检查	30

CPP-013 STL 容器成员函数速查	30
顺序容器简表	31
通用成员函数速查	31
vector/deque/list/array/string 常用模板	31
stack/queue/priority_queue	32
priority_queue 存 pair 与自定义排序	33
set/multiset	33
map/multimap	34
unordered_map/unordered_set	34
pair/tuple	35
迭代器遍历与 erase 安全写法	35
常用完整模板	35
BRUTE-01: 复杂度与数据范围速查	37
BRUTE-07: 记忆化搜索总论	39
DP-00: DP 总流程	42
考场总流程	44
五问检查卡	44
建模 8 问: 从题面到代码前必须过一遍	44
初始化常用表	45
初始化与答案位置总表	45
转移顺序心法: pull、push 与依赖图	45
复杂度预算卡	45
最小表推骨架	46
暴力/部分分替代	46
DP 调试口令	46
DP-01: DP 路由表	47
一眼路由表	48
背包细分表	48
不是 DP 的常见误判	48
DP-02: 状态句式库	49
句式 1: 线性序列	50
句式 2: 选/不选	50
句式 3: 背包	50
句式 4: 两个序列	50
句式 5: 网格	51
句式 6: 区间	51
句式 7: 树形	51
句式 8: DAG	51
句式 9: 状压集合	51
句式 10: 数位	51
状态完整性检查	51
DP-03: DFS -> 记忆化 -> 表推升级图	52
升级图	53
什么时候可以直接加 memo	53
通用暴力 DFS 版本	53
通用记忆化版本	53
通用表推版本	54
从 DFS 到表推的对照表	54
map<tuple> 救场版本	54
什么时候不改表推	54
DP-03B: 状态增维/升维路由	55
核心原则	56
一眼增维表	56
维度 1: last	57
维度 2: mask	57
维度 3: cnt/rest	58
维度 4: tight	58

维度 5: leading	59
维度 6: color	59
维度 7: parity/mod	59
维度 8: parent/prev	60
维度 9: phase	60
map<tuple> 完整状态救场	61
状态完整性检查卡	61
DP-21: P1874 快速求和建模例题	62
题意压缩	63
第一阶段: 先写暴力, 找决策点	63
第二阶段: 找重叠子问题	63
第三阶段: 定义状态并验可行性	63
第四阶段: 推导转移	64
第五阶段: 细节与陷阱	64
DP-22: 编辑距离建模例题	67
题意压缩	68
第一阶段: 从暴力尝试开始, 找决策点	68
第二阶段: 找到无后效性	68
第三阶段: 定义状态并验可行性	68
第四阶段: 倒推最后一步	69
情况一: 最后字符相等	69
情况二: 最后字符不相等	69
第五阶段: 初始化	69
从 P1874 到编辑距离: DP 五步法	70
DP-25: 两道例题的暴力 DFS 到记忆化搜索	72
1. 考场口令	73
2. P1874 快速求和: 记忆化 DFS 完整代码	73
3. 编辑距离: 记忆化 DFS 完整代码	74
4. 什么时候记忆化能直接满分	75
5. 什么时候只能拿部分分	76
6. 数组 memo 与 map memo 的选择	76
7. 考场策略总结	76
DP-26: 发现有后效性怎么办: 升维吸收关键历史	77
1. 核心原则	78
2. 例题一: 网格最大收益, 不能连续向下走两步	78
3. 例题二: 爬楼梯, 不能连续跳同样步数	80
4. 例题三: 恰好选 K 个数, 且不能选相邻	81
5. 例题四: 网格最大收益, 最多一次收益翻倍	82
6. 例题五: 股票买卖, 冷冻期一天	83
7. 升维清单	84
8. 从 DFS 角度理解升维	84
DS-00 数据结构路由与接口总表	85
DS-06: 双指针与滑动窗口	87
一眼路由	89
模板 1: 最长连续子数组, 和不超过 S	89
模板 2: 最短连续子数组, 和至少 S	89
模板 3: 排序数组两数之和是否存在	90
模板 4: 有序数组原地去重	90
模板 5: 最长无重复字符子串	90
模板 6: 三数之和为 0, 去重计数/列举	91
GRAPH-00 标准 Graph 与建图场景	92
图类型与模块路由协议	94
GRAPH-03 Dijkstra、路径恢复、多源最短路	95
MATH-01 gcd / lcm	97
SIM-01 模拟、字符串扫描与高精度	100

STR-01 string 常用操作	104
STR-05 Manacher 回文算法	107
TRAIN-00 第 7 卷: 通用 WA / RE / TLE 自检	112
1. 90 秒错误定位	112
2. 通用 WA 检查	113
3. 通用 RE 检查	113
4. 通用 TLE 检查	113
5. 提交前 7 问	113
6. 失败后的回退路线	114

核心速查版

这是考场第一页到前几十页的快速导航包：只放最高频、最能救分、最容易拼接的模块。完整资料见 `openbook_full.md/pdf` 和 `openbook_printable_full.md/pdf`。

核心目录

- OPS-00-unified-protocols.md: OPS-00 第 -1 卷: 统一接口与标准容器
- OPS-01-exam-operations.md: OPS-01 第 0 卷: 考场作战流程
- ROUTE-00-routing-tables.md: ROUTE-00 第 0A 卷: 题面路由表
- CPP-001-main-io.md: CPP-001 C++17 主骨架与输入输出
- CPP-10-io-formatting.md: CPP-10: 输入输出与格式化
- CPP-013-stl-containers-reference.md: CPP-013 STL 容器成员函数速查
- BRUTE-01-complexity-cheatsheet.md: BRUTE-01: 复杂度与数据范围速查
- BRUTE-07-memoized-search-overview.md: BRUTE-07: 记忆化搜索总论
- DP-00-total-flow.md: DP-00: DP 总流程
- DP-01-routing-table.md: DP-01: DP 路由表
- DP-02-state-sentence-library.md: DP-02: 状态句式库
- DP-03-dfs-memo-table-upgrade.md: DP-03: DFS -> 记忆化 -> 表推升级图
- DP-03B-state-dimension-router.md: DP-03B: 状态增维/升维路由
- DP-21-p1874-modeling-example.md: DP-21: P1874 快速求和建模例题
- DP-22-edit-distance-modeling-example.md: DP-22: 编辑距离建模例题
- DP-25-dfs-memo-case-strategy.md: DP-25: 两道例题的暴力 DFS 到记忆化搜索
- DP-26-aftereffect-state-augmentation.md: DP-26: 发现有效性怎么办: 升维吸收关键历史
- DS-00-data-structure-routing.md: DS-00 数据结构路由与接口总表
- DS-06-two-pointers-sliding-window.md: DS-06: 双指针与滑动窗口
- GRAPH-00-standard-graph.md: GRAPH-00 标准 Graph 与建图场景
- GRAPH-03-dijkstra-path-multisource.md: GRAPH-03 Dijkstra、路径恢复、多源最短路
- MATH-01-gcd-lcm.md: MATH-01 gcd / lcm
- SIM-01-high-precision.md: SIM-01 模拟、字符串扫描与高精度
- STR-01-basic-operations.md: STR-01 string 常用操作
- STR-05-manacher.md: STR-05 Manacher 回文算法
- TRAIN-00-debug-checklist.md: TRAIN-00 第 7 卷: 通用 WA / RE / TLE 自检

OPS-00 第 -1 卷: 统一接口与标准容器

模块编号: OPS-00

模块名称: 第 -1 卷 统一接口与标准容器

标签: 前置卷、接口协议、标准容器、C++17、考场装配

一句话用途: 把题面输入整理成统一形状, 让后面的图论、数据结构、DP、暴力模块可以直接拼上去。

题面触发词: 任意题目开写前都先看本卷。

什么时候用: 每道题开始前, 用本卷确定索引、类型、容器、函数名和模块调用外壳。

不要什么时候用: 不要把本卷当算法教材; 它只规定“零件接口”, 不解释算法原理。

复杂度: 本卷本身没有算法复杂度; 复杂度由下游模块决定。

数据范围参考: 所有规模都适用, 尤其适合需要快速拼接多个模块的题。

依赖的标准容器：无。

输入如何整理：先读题面原始输入，再转为 1-index 全局数组、Graph、Query、State、Compressor。

接口：init/build/add/add_edge/setv/range_add/query/prefix/at/solve_xxx。

输出能力：提供统一外壳和下游模块调用约定。

下游可接：PrefixSum、树状数组、SegmentTree、DSU、BFS、Dijkstra、Topo、DP、DFS memo。

可拼接模块：所有模块。

模板代码：见本文件各协议代码块。

调用示例：见各协议的“考场抄法”。

常见坑：混用 0-index/1-index、区间半开半闭、图边重复、把 int 当距离、函数直接输出导致无法升级。

暴力/部分替代：统一保留 solve_bruteforce()，先拿小数据分，再替换 solve_optimized()。

升级方向：把暴力外壳升级为 memo、DP、图论或数据结构正解，但输入整理层不改。

最小测试样例：每个模块接入后都测单点、边界、空结果、重复元素、极值。

0. 考场使用顺序

1. 读题，圈出 n/m/q、是否有修改、是否是图/树/区间/状态。
2. 按本卷确定标准容器：Graph / Array / Query / State。
3. 把输入只整理一次，不在算法里边读边算。
4. 先写 solve_bruteforce() 或最简单合法版本。
5. 再把核心函数替换成正解模块。
6. 提交前只检查接口契约：索引、区间、类型、方向、初始化。

1. 全局 C++17 骨架

每份代码开头统一抄这一段。不要加考试环境不一定支持的优化开关。

如果现场 GNU g++ 支持 bits/stdc++.h，直接用短版。若不支持，把第一行替换成 CPP-001 的“标准头文件安全包”，不要现场逐个猜缺哪个头。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 1000000000;
const ll LINF = 4'000'000'000'000'000'000LL;
const ll MOD = 1000000007LL; // 题目给别的模数就改这里

void solve() {
    // write here
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
    return 0;
}
```

硬规则：

项目	统一约定
C++ 标准	C++17
禁止	#pragma GCC optimize、freopen、文件 IO、#define int long long
点编号	默认 1..n

项目	统一约定
数组/DP 表	默认 1-index; 上限明确时优先全局静态数组 <code>a[MAXN] / dp[MAXN][MAXM]</code>
区间	默认闭区间 <code>[1, r]</code>
字符串	默认 C++ 自然 0-index, 进入字符串模块时单独提醒
网格	默认 <code>1..n, 1..m</code> , 转图时用 <code>id(i, j)</code>
权值/距离/答案	优先 <code>long long</code>
数量/下标	一般 <code>int</code>

数组选择口令:

题目给明确上限: 优先全局静态数组, 编号 `1..n`, 防御性多开 `5~10`。

输入规模不固定或需要可变长度: 再用 `vector`, 并主动开 `n+1` 保持 1-index。

字符串、`queue`、`deque`、`priority_queue`、`set/map`、`unordered_map` 这类容器行为: 直接用 STL。

集合状压的 `mask` 数值必须从 `0` 枚举, 但业务对象编号仍写 `1..k`, 取位统一写 `i-1`。

2. 函数命名规范

统一动词让模块能替换。

函数名	用途	例子
<code>init(...)</code>	清空并重新初始化	<code>fw.init(n)</code> 、 <code>G.init(n)</code>
<code>build(...)</code>	从已有数组/图建立结构	<code>ps.build(a)</code> 、 <code>st.build(a)</code>
<code>add(pos, val)</code>	单点增加	树状数组单点加
<code>add_undirected/add_directed</code>	图加边	<code>G.add_undirected(u, v, w)</code>
<code>setv(pos, value)</code>	单点赋值	线段树点赋值, 避免撞 <code>std::set</code>
<code>range_add(l, r, val)</code>	闭区间加	Lazy Segment Tree
<code>query(l, r)</code>	闭区间查询	和/最值/gcd
<code>prefix(pos)</code>	查询 <code>[1, pos]</code>	前缀和、树状数组
<code>at(pos)</code>	单点查询	差分树状数组
<code>solve_bruteforce()</code>	小数据精确版	部分分
<code>solve_memo()</code>	记忆化版	从 DFS 升级
<code>solve_optimized()</code>	正解版	DP/图论/数据结构

接口原则:

算法函数尽量返回结果, 不直接 `cout`。

`solve()` 负责读入、选择版本、输出。

标准外壳:

```
void read_input() {
    // 只读入并整理成标准容器
}

ll solve_bruteforce() {
    // 小数据精确版
    return 0;
}

ll solve_optimized() {
    // 正解版
    return 0;
}

void solve() {
    read_input();
    cout << solve_optimized() << "\n";
}
```

3. Graph 协议

目标: 一份建图同时服务 BFS、DFS、Dijkstra、Topo、Kruskal、Floyd、Bellman-Ford、LCA。

```

struct AdjEdge {
    int to;
    ll w;
    int edge_index; // 内部边下标, 只给模板内部用
    bool directed;
    int input_id;    // 对外边号, 1-index
};

struct FullEdge {
    int from, to;
    ll w;
    bool directed;
    int input_id;    // 对外边号, 1-index
};

struct Graph {
    int n;
    vector<vector<AdjEdge>> g;
    vector<FullEdge> edges;

    Graph(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        edges.clear();
    }

    void add_edge_raw(int u, int v, ll w, bool directed) {
        int edge_index = (int)edges.size(); // 内部 0-based, 不输出
        int input_id = edge_index + 1;      // 题面边号 1-based
        edges.push_back({u, v, w, directed, input_id});
        g[u].push_back({v, w, edge_index, directed, input_id});
        if (!directed) g[v].push_back({u, w, edge_index, directed, input_id});
    }

    void add_undirected(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, false);
    }

    void add_directed(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, true);
    }
};

```

考场抄法:

```

int n, m;
cin >> n >> m;
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 无向图
}

```

无权图:

```

int u, v;
cin >> u >> v;
G.add_undirected(u, v); // 无向无权, 推荐写法
G.add_directed(u, v);   // 有向无权, 推荐写法

```

Graph 的双视图:

视图	用途	遍历方式
G.g[u]	BFS、DFS、Dijkstra、Topo、SCC、LCA	for (auto e : G.g[u])
G.edges	Kruskal、Floyd 初始化、Bellman-Ford、全边排序	for (auto e : G.edges)

Graph 接口坑位:

1. Kruskal 只遍历 G.edges, 不遍历 G.g, 否则无向边会重复。
2. Bellman-Ford 遇到无向边要松弛两个方向。
3. Topo 只适用于 `G.add\_directed(u, v)` 建出的有向边。
4. LCA/树 DP 要用无向边, 并从根 DFS。
5. 最大流不要用 Graph, 另用 FlowGraph。
6. 考场只写 add_undirected/add_directed, 不直接调用 add_edge_raw。
7. 点编号默认 1-index; 对外边号用 input_id, 也是 1-index。edge_index 是内部跳父边用的下标, 不输出。

4. Array 协议

标准读入:

```
int n;
cin >> n;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

区间约定:

所有区间都是闭区间 $[1, r]$ 。

空区间 $1 > r$ 时, 查询类函数返回 0 或对应单位元。

静态前缀和协议:

```
struct PrefixSum {
    int n;
    vector<ll> pre;

    void build(const vector<ll> &a) {
        n = (int)a.size() - 1;
        pre.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
    }

    ll prefix(int pos) const {
        if (pos <= 0) return 0;
        if (pos > n) pos = n;
        return pre[pos];
    }

    ll query(int l, int r) const {
        if (l > r) return 0;
        return prefix(r) - prefix(l - 1);
    }
};
```

动态区间和协议:

```
struct BIT {
    int n;
    vector<ll> bit;

    void init(int n_) {
        n = n_;
        bit.assign(n + 1, 0);
    }

    void add(int pos, ll val) {
```



```

    if (pos <= 0 || pos > n) return;
    for (; pos <= n; pos += pos & -pos) bit[pos] += val;
}

void build(const vector<ll> &a) {
    init((int)a.size() - 1);
    for (int i = 1; i <= n; i++) add(i, a[i]);
}

ll prefix(int pos) const {
    pos = min(pos, n);
    ll res = 0;
    for (; pos > 0; pos -= pos & -pos) res += bit[pos];
    return res;
}

ll query(int l, int r) const {
    if (l > r) return 0;
    return prefix(r) - prefix(l - 1);
}

ll at(int pos) const {
    return query(pos, pos);
}
};

```

线段树统一接口先记名字，代码放数据结构卷：

```

build(a);
add(pos, delta);
setv(pos, value);
range_add(l, r, delta);
query(l, r);

```

5. Query 协议

多次询问统一先存成 Query，不要把各种题写成各自的临时变量。

```

struct Query {
    int l = 0, r = 0;
    int id = 0;
    int op = 0;    // 题目有操作类型时使用
    int pos = 0;   // 单点位置
    ll x = 0;      // 修改值、阈值、权值
};

```

常见读法：

```

int q;
cin >> q;
vector<Query> qs(q + 1);
for (int i = 1; i <= q; i++) {
    cin >> qs[i].op >> qs[i].l >> qs[i].r;
    qs[i].id = i;
}

```

离线查询约定：

id 保留原顺序。

排序只排序 Query 数组，不改答案数组。

答案统一放 ans[id]。

6. Compressor 协议

值域很大但实际出现的数不多时，先压缩再接 树状数组/Segment Tree。

```

struct Compressor {
    vector<ll> xs;

    void build(vector<ll> v) {
        xs = v;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
    }

    int id(ll x) const {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int lower_id(ll x) const {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int upper_id(ll x) const {
        return upper_bound(xs.begin(), xs.end(), x) - xs.begin();
    }

    ll val(int pos) const {
        return xs[pos - 1];
    }

    int size() const {
        return (int)xs.size();
    }
};

```

接法:

```

vector<ll> all;
// 把所有 a[i]、查询里的 x、区间端点等需要比较的值放进 all
Compressor cp;
cp.build(all);

```

```

BIT fw;
fw.init(cp.size());
fw.add(cp.id(x), 1);

```

压缩坑位:

1. 压缩后编号仍然从 1 开始。
2. x 一定出现过, 用 id(x)。
3. 查询原坐标范围 [L, R], 用 lower_id(L), upper_id(R)。
4. lower_id(L) > upper_id(R) 表示范围内没有点。

7. State 协议

状态统一拆成三类:

State = {进度, 资源, 记忆}

类别	常用变量名	例子
进度	i, pos, u, step	处理到第几个物品、当前点
资源	cap, k, rem, cost	剩余容量、次数、预算
记忆	mask, last, color, used	已访问集合、上一个点、颜色

纸质默认稳妥写法:

```

map<tuple<int, int, int>, ll> memo;

ll dfs(int i, int cap, int mask) {
    if (i == n + 1) return 0; // base case 按题意替换

```

```

    auto key = make_tuple(i, cap, mask);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 枚举选择
    return memo[key] = ans;
}

```

边界清晰时换成数组：

```

const int MAXN = 200000 + 5;
const int MAXW = 5000 + 5;
static ll dp[MAXN][MAXW];

for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= W; j++) dp[i][j] = -LINF;
}

```

状态存储选择：

情况	容器	考场优先级
范围小且明确	vector / 多维 vector	最高
有集合访问	vector + mask	高
维度复杂但状态少	map<tuple<...>, ll>	最稳
需要速度且能编码	unordered_map<ll, ll>	升级

8. 模块契约卡

每个算法模块接入前，先在纸上补全这张卡。

模块名：

输入依赖：

输出能力：

下游可接：

不能处理：

初始化顺序：

查询/更新接口：

复杂度：

最小测试：

例：Dijkstra

输入依赖：Graph，非负边权，起点 s

输出能力：dist[i]

下游可接：路径恢复、最短路 DAG、计数 DP

不能处理：负权边

初始化顺序：建图 -> dist=LINF -> pq

查询/更新接口：vector<ll> dijkstra(const Graph& G, int s)

复杂度：O((n+m)logn)

最小测试：3 点链、不可达点、重边、起点到自己

9. 最小自检

每次拼完模块，提交前用 60 秒过这张表。

检查项	问自己
索引	输入是 0-index 还是 1-index? 我有没有统一?
区间	所有 [1, r] 都是闭区间吗?
类型	距离、答案、乘法有没有用 ll?
初始化	多组数据时数组、图、memo 有没有清空?
图方向	directed 写对了吗?
不可达	LINF 会不会被拿去加法溢出?
排序	离线排序后答案有没有按 id 放回?
输出	每个查询都有输出吗? 换行够吗?

OPS-01 第 0 卷：考场作战流程

模块编号: OPS-01

模块名称: 第 0 卷 考场作战流程

标签: 考试节奏、3 小时、32 次提交、提交检查、兜底输出

一句话用途: 在 3 小时机考里保持节奏, 先拿确定分, 再升级。

题面触发词: 正式考试开始后全程使用。

什么时候用: 开场扫题、每题开写、每次提交前、卡住时。

不要什么时候用: 不要把时间花在重读流程本身; 流程是为了减少犹豫。

复杂度: 时间管理复杂度为 0(果断)。

数据范围参考: 所有题。

依赖的标准容器: OPS-00、ROUTE-00、ROUTE-01。

输入如何整理: 每题先按 OPS-00 标准容器整理输入。

接口: 读题 -> 路由 -> 部分分 -> 升级 -> 自检 -> 提交。

输出能力: 3 小时时间策略、32 次提交策略、提交前检查清单、完全不会时兜底策略。

下游可接: 所有卷。

可拼接模块: 所有模块。

模板代码: 无。

调用示例: 开场按时间表执行。

常见坑: 死磕一题、第一版就追正解、提交次数被样例调试耗尽、不会时连合法输出都不写。

暴力/部分分替代: 先写小数据精确解, 再看是否升级。

升级方向: 按分值密度升级, 而不是按个人执念升级。

最小测试样例: 样例、手造边界、随机小数据对拍可选。

1. 考场总原则

目标不是写出最漂亮题解。
目标是在限制时间和提交次数内, 交出最多确定分。

五条铁律:

- 1. 每题先有合法输出, 再追求高分。
- 2. 输入整理层只写一次, 后面升级不要重写读入。
- 3. 第一版优先暴力/部分分, 能过样例再升级。
- 4. 每次提交只验证一个明确变化。
- 5. 最后 5 分钟不写新算法, 只做低风险修补和兜底。

2. 3 小时时间策略

按 180 分钟计算。若题数不同, 比例不变。

时间段	动作	产出
0-10 分钟	全部扫题, 圈 n/m/q、题型、明显部分分	给每题标 A/B/C: 会、可部分、暂放
10-35 分钟	做最容易题的第一版	至少 1 题通过样例并提交
35-70 分钟	给每题补 “能写出的最低合法版本”	每题至少有读入和兜底/暴力思路
70-115 分钟	攻击中档升级: 数据结构、BFS/Dijkstra、背包、DP	提升已提交题分数
115-145 分钟	处理难题部分分: 小数据 DFS、memo、特殊性质	不空题
145-165 分钟	回头修 WA/TLE 最明显的题	只改能解释清楚的 bug
165-175 分钟	全题提交前检查, 补缺输出	防 RE/PE/漏交

时间段	动作	产出
175-180 分钟	冻结新功能，只做最终确认	所有题保留最稳版本

题目优先级：

会写正解且代码短 > 能拿稳定部分分 > 正解长但容易错 > 完全没路由

单题止损线：

20 分钟没有跑通样例：降级写部分分。

35 分钟没有任何提交：写合法兜底并换题。

同一题连续 3 次 WA 且原因不明：停止提交，回到本地手造样例。

3. 32 次提交策略

每题最多 32 次提交时，不要把提交当编译按钮。提交前本地至少过样例和 2 个手造点。

提交编号	用途	允许动作
1	最小可运行版本	暴力/兜底/样例通过后交
2-4	修读入、输出格式、明显边界	每次只改一个问题
5-8	部分版本稳定	小数据 DFS、前缀和、BFS 等
9-16	正解升级	树状数组、Dijkstra、DP、线段树等
17-22	针对 WA/TLE 修复	必须写明猜测原因再改
23-26	替代路线	例如 Dijkstra 改 Floyd 小图、递推改 memo
27-30	保守修补	越界、初始化、11、方向、取模
31-32	最终保险	只提交最稳版本，不做大改

每次提交前在草稿上写一句：

这次提交只验证：_____

禁止提交：

1. 刚大改完但没跑样例。
2. 一次改了三个以上位置且不知道哪个影响结果。
3. 只是“感觉可能好了”。
4. 最后 5 分钟切换全新算法。

4. 先交部分分再升级

标准升级链：

合法输出

- > 小数据暴力
- > 记忆化/剪枝
- > 标准容器 + 主模块
- > 特判/边界修补

常见升级表：

第一版	升级版	保留不变
每次循环求区间和	PrefixSum/树状数组	read_input()、查询循环
DFS 判可达	BFS 最短路	Graph
Bellman-Ford 小数据	Dijkstra	Graph
双重循环逆序对	Compressor + 树状数组	Array
全排列访问点	BitmaskDP	距离矩阵
纯 DFS 选物品	背包 DP	物品数组
DFS DAG 路径	Topo + DP	Graph directed
每问爬树路径	LCA	Graph undirected、depth

代码结构：

```

void solve() {
    read_input();

    // 需要时保留部分分开关, 最终可手动切到 optimized.
    if (false) {
        cout << solve_bruteforce() << "\n";
    } else {
        cout << solve_optimized() << "\n";
    }
}

```

5. 提交前检查清单

60 秒快检:

类别	检查项
编译	C++17; 无 freopen; 无 #pragma; 无 #define int long long
输入	多组数据是否处理; 读入数量是否和题面一致; 字符串是否含空格
索引	图/数组是否 1-index; 字符串是否 0-index; n+1/n+2 是否够
区间	[1,r] 闭区间; r+1 是否越界保护; l>r 怎么处理
类型	距离/答案/乘法/计数是否 ll; INF 是否够大
初始化	dist/memo/vis/dp/ans 是否清空; 多测是否重新 init
图	有向/无向是否写对; 权值是否读入; 负边是否误用 Dijkstra
DP	初值是 0、INF 还是 -INF; 循环顺序是否正确
取模	加减乘是否取模; 负数是否 (x%MOD+MOD)%MOD
输出	每个询问都有输出; 大小写; 空格/换行; 无调试打印

手造最小点:

n=1
 q=1
 l=r
 全 0 / 全相等 / 全负数
 图不连通
 起点等于终点
 容量 W=0
 没有可行解

6. 完全不会时兜底策略

完全不会也要做三件事: 读完输入、输出合法形状、避免 RE/PE。

兜底步骤:

1. 读题面输出格式, 判断输出几行、每行几个数。
2. 完整读入所有输入, 不要提前 return 导致交互/多测错位。
3. 若有 q 个询问, 循环输出 q 行。
4. 优先输出题目允许的“不存在”标记, 如 -1、NO、0。
5. 如果问最小值且允许不可达, 常见输出 -1。
6. 如果问方案数, 输出 0。
7. 如果问 YES/NO, 输出 NO。
8. 如果问构造且允许无解, 输出 NO 或 -1。
9. 如果必须输出一个排列, 输出 1..n。
10. 如果必须输出一个数组, 输出全 0 或原数组, 按题面限制选择。

常见输出类型兜底表:

输出要求	兜底输出	注意
单个答案数	0	若题面规定无解为 -1, 优先 -1
每个查询一个数	每行 0	行数必须等于查询数
YES/NO	NO	大小写按题面
是否存在并构造	NO	不要乱输出构造
最短路不可达	-1	题面若要求 INF 或别的标记按题面

输出要求	兜底输出	注意
方案数取模	0	合法且稳定
排列	1 2 ... n	仅在任意排列格式合法时
路径	-1 或 0	按题面“不存在路径”的格式

兜底不是放弃：

先交兜底防空题。

然后回到 ROUTE-00，找最像的信号。

能写暴力就替换兜底。

能写模块就替换暴力。

7. 卡住时的 5 分钟复位

1. 停止改代码。
2. 写下当前模块输入、输出、不能处理什么。
3. 用 ROUTE-00 重新看：数据范围、操作类型、图边权。
4. 做一个 $n=1$ 或 3 点样例手算。
5. 决定：修一个 bug、降级部分分、还是换题。

判断是否该换题：

读入都没写完：再给 5 分钟。

样例不过且原因不明：换题。

正解差一点但部分分已交：换题。

还有更短的题没看：换题。

ROUTE-00 第 0A 卷：题面路由表

模块编号：ROUTE-00

模块名称：第 0A 卷 题面路由表

标签：路由、关键词、数据范围、操作类型、模块选择

一句话用途：把题面信号、数据范围和操作类型快速映射到可抄模块。

题面触发词：区间、修改、查询、最短、连通、树、DAG、容量、方案数、字符串、坐标很大。

什么时候用：读完题后 3 分钟内，用本表决定第一版代码写什么。

不要什么时候用：不要在还没看数据范围时直接套高级模板。

复杂度：本表只做选择；复杂度见数据范围预算表。

数据范围参考：所有题目。

依赖的标准容器：Graph、Array、Query、State、Compressor。

输入如何整理：先圈 $n/m/q$ /值域/是否修改/是否有边权/是否有环，再查表。

接口：路由到 build/add/range_add/query/dijkstra/topo/dp/dfs 等模块。

输出能力：给出优先模型、主模块、辅助模块和不要使用的场景。

下游可接：所有算法卷。

可拼接模块：见各表。

模板代码：无。

调用示例：按“信号 -> 模型 -> 容器 -> 模块 -> 验错”的顺序执行。

常见坑：只看关键词不看修改；只看 n 不看 q ；图有负边还用 Dijkstra；树题没确认是否真是树。

暴力/部分分替代：查不到就先按数据范围写暴力或 memo。

升级方向：从预算表升级到 $O(n \log n)$ 、 $O(n)$ 或图/DP 专门模块。

最小测试样例：每条路由至少测边界、小数据、极端值。

1. 3 分钟路由动作

第 1 分钟: 圈数据范围 n, m, q , 值域, 写下目标复杂度。

第 2 分钟: 圈题面动作: 查询、修改、路径、连通、选择、计数、匹配。

第 3 分钟: 查本表, 决定标准容器和第一版模块。

优先级:

先看数据范围能不能承受。

再看有没有修改。

再看图边权和方向。

最后看目标: 最小/最大/方案数/可行性/构造。

2. 题面信号路由表

题面信号	优先模型	标准容器	主模块	辅助模块	不要用在
数组不变, 多次问 $[1, r]$ 和	静态区间和	<code>vector<ll> a</code>	PrefixSum	Query	有修改
数组不变, 多次问 $\min/\max/\gcd$	静态 RMQ	<code>vector<ll> a</code>	SparseTable	Log 表	有修改、区间和
单点修改, 区间和	动态前缀	Array	树状数组	SegmentTree	区间修改
单点修改, 区间最值	动态区间最值	Array	SegmentTree	Compressor	只问区间和时优先 树状数组
区间加, 单点查	差分	Array	Difference / 差分 树状数组	Query	还要区间和
区间加, 区间和	动态区间结构	Array	LazySegTree / 双 树状数组	Compressor	只做最后输出
值域很大, 但只比较大小/排名	离散化	Compressor	树状数组 / SegmentTree	Sort	需要真实连续距离
求逆序对、前面有多少数大/小	动态统计	Compressor	树状数组	Sort	n 很小可暴力
滑动窗口最大/最小	单调队列	Array	MonotonicQueue	Deque	窗口长度不固定且有复杂删除
最近更大/更小、贡献边界	单调栈	Array	MonotonicStack	前后边界数组	需要任意区间动态查询
连通性、合并集合	并查集	边表	DSU	Kruskal	需要删除边
最小生成树、修路最小费用	MST	<code>Graph.edges</code>	Kruskal	DSU	有向图最小树形图
无权最短路、最少步数	BFS	Graph / Grid	BFS	queue	有权边
边权非负最短路	单源最短路	Graph	Dijkstra	priority_queue	有负边
有负边最短路	松弛最短路	<code>Graph.edges</code>	Bellman-Ford / SPFA	负环判断	n, m 很大且无负边
小图任意两点最短路	全源最短路	<code>Graph.edges</code> + 矩阵	Floyd	路径恢复	$n > 500$ 通常危险
DAG、依赖、先后顺序	拓扑	Graph directed	Topo	DP	有环
DAG 上最长/最短/方案数	DAG DP	Graph directed	Topo + DP	入度	有环未处理
树上祖先、两点路径	树上查询	Graph undirected	LCA	DFS depth/dist	不是树或森林
树上选点、父子依赖	树形 DP	Graph undirected	DFS DP	State	一般图有环
树上选 K 个、子树合并容量	树上背包	Graph tree + State	DP-14 / DP-19	Knapsack	一般图有环
每个点作为根/中心、贡献重算	换根 DP	Graph tree	TREE-02	TreeDP	非树
选/不选、容量限制	背包	State	0/1 Knapsack	滚动数组	物品可无限选
可以无限选某物品	完全背包	State	Complete Knapsack	循环顺序	每物只能选一次
每组最多选一个	分组背包	State	Group Knapsack	分组循环	物品无组限制
两个字符串/序列匹配	双序列 DP	string / Array	LCS / EditDistance	DP 表	只需子串可考虑哈希/KMP

题面信号	优先模型	标准容器	主模块	辅助模块	不要用在
LIS/LCS 变体、LCIS、公共且上升	序列 DP 变体	Array / string	DP-23	树状数组/SegmentTree	连续子串要换模型
训练集、测试集、标签、模型、SPJ 得分	AI 包装题	样本表 / 特征矩阵	AI-00 / AI-10	AI-02..15	不要上第三方库或随机大模型
SVM、DNN、反向传播、自动求导	AI 公式模拟	计算图 / 矩阵向量	AI-11..15	SIM-03	数据大时先 baseline
JSON/CSV/INI、表达式、脚本规则	解析模拟	Token / AST	SIM-03/04/05	string / map	不要临场乱写半解析
日期、时区、经过天数、历法	日期模拟	Date / day number	SIM-06	数学取模	夏令时规则不明时按题面
区间合并、区间删除、括号匹配	区间 DP	Array	IntervalDP	PrefixSum	n 很大
n <= 20 且访问集合	状压	State mask	BitmaskDP / DFS	Floyd/Dijkstra	n > 22 基本爆
1..N、数位限制、上界很大	数位 DP	digits + state	DP-17	DFS memo	小范围普通枚举
dp[i]=min/max dp[j]+cost 且查询范围	DP 优化	State + query	DP-18	树状数组/SegmentTree	先写朴素
方案数、可行性、能否凑出	计数/可行性 DP	State	DP-20	MOD / bitset	不要用取模值判可达
多重/二维/价值维度/至少装满背包	背包变体	Knapsack State	DP-24	DP-06/07/08	先判至多/恰好/至少
斜率、分治、Knuth、SOS、轮廓线等高阶 DP 信号	高阶 DP 索引	State	DP-27	先交朴素/记忆化	不满足条件别硬套
重复状态明显但不好表推	记忆化搜索	State	DFS + memo	map tuple	状态有环且无处理
求第 k、小于等于 x 个数	排名/二分	Compressor	树状数组 / SegmentTree	二分答案	静态可排序二分
最大最小值、最小化最大值	二分答案	判断函数	BinarySearch + Check	Greedy/Graph/DP check	不单调

3. 数据范围预算表

先用最大量级估算，不要被样例骗。

数据范围	目标复杂度	常见可用模块	考场口令
n <= 8..10	$O(n!)$	全排列、爆搜	可以先全排列拿分
n <= 18..22	$O(2^n n)$	状压 DP、子集枚举	看到集合访问先想 mask
n <= 35..45	$O(2^{(n/2)})$	折半枚举	一分为二，排序合并
n <= 300	$O(n^3)$	Floyd、区间 DP	三重循环可接受
n <= 500	勉强 $O(n^3)$	Floyd、小规模 DP	常数要小
n <= 3000	$O(n^2)$	双序列 DP、普通 DP	双重循环可试
n <= 5000	$O(n^2)$ 边缘	DP、枚举优化前版本	C++ 勉强，注意常数
n, q <= 2e5	$O((n+q)\log n)$	树状数组、SegmentTree、Dijkstra、排序	默认 log 结构
n, m <= 2e5	$O((n+m)\log n)$	BFS、Dijkstra、Kruskal、Topo	图用邻接表
n <= 1e6	$O(n)$ 或 $O(n \log n)$	前缀、差分、双指针、筛法	少开二维
值域 1e9/1e18，数量 2e5	$O(n \log n)$	Compressor + 树状数组	坐标压缩
网格 n*m <= 2e5	$O(nm)$	Grid BFS/DP	二维直接做
网格 n, m <= 1000	$O(nm)$	BFS、前缀、DP	内存约百万级
多测试总和给出	按总和	清空容器	不要按单测最大乘 T
多测试无总和	保守	暴力分档/剪枝	防止总量爆炸

粗算：

1e8 次简单操作已经危险。

2e8 次以上不要赌，除非数据极小且时间宽。

vector<vector<ll>>(5000,5000) 约 200MB，通常危险。

4. 操作类型路由表

操作组合	第一选择	接口	复杂度	部分版本	升级方向
静态区间和	PrefixSum	build/query	$O(n+q)$	每问循环 $O(nq)$	无需升级
静态区间	SparseTable	build/query	$O(n\log n+q)$	每问循环	SegmentTree 若有修改
min/max/gcd	树状数组	add/prefix/query	$O(\log n)$	直接改数组再循环	SegmentTree
单点加 + 前缀/区间和	树状数组	add(pos, new-old)	$O(\log n)$	数组暴力	SegmentTree
单点赋值 + 区间最值	SegmentTree	setv/query	$O(\log n)$	数组暴力	无
区间加 + 最后输出	Difference	diff[l]+=x, diff[r+1]-=x	$O(n+q)$	每次循环加	差分树状数组
区间加 + 单点查	差分树状数组	range_add/at	$O(\log n)$	差分离线	LazySegTree
区间加 + 区间和	LazySegTree / 双树状数组	range_add/query	$O(\log n)$	每次循环	LazySegTree
第 k 小 / 动态排名	Compressor + 树状数组	add/prefix + 二分	$O(\log^2 n)$	排序数组重算	权值线段树
连通块合并	DSU	find/unite	近似 $O(1)$	DFS 每次判连通	无删除边
加边判是否成环	DSU	unite 返回真假	近似 $O(1)$	DFS	Kruskal
无权单源最短路	BFS	bfs(G,s)	$O(n+m)$	DFS 不保证最短	Dijkstra 若有权
非负权单源最短路	Dijkstra	dijkstra(G,s)	$O((n+m)\log n)$	Bellman-Ford 小数据	无负边
任意两点小图最短路	Floyd	dist[i][j]	$O(n^3)$	每次 BFS/Dijkstra	Johnson 不作为优先
有向无环依赖	Topo	topo_sort(G)	$O(n+m)$	DFS 记忆化	DAG DP
树上路径距离	DFS + LCA	lca.query(u,v)	$O(\log n)$	每问 BFS/爬父亲	树剖低优先
容量选择最优	背包 DP	dp[j]	$O(nW)$	DFS 枚举	优化循环/滚动
复杂状态最优	Memo DFS	dfs(state)	状态数 * 转移	暴力 DFS	表推 DP

5. 路由冲突时怎么判

有修改优先看操作类型，不要直接用静态算法。

有边权优先看权值符号：无权 BFS，非负 Dijkstra，负边 Bellman-Ford/SPFA。

是树先确认 $m = n - 1$ 且连通；否则按一般图处理。

有 DAG 字样但可能有环，先 topo 检查 `order.size() == n`。

值域大但只出现 $2e5$ 个数，先 Compressor。

能先拿部分分时，先写暴力版本，保留函数名再升级。

C++17 主骨架与输入输出

模块编号：CPP-001

模块名称：C++17 主骨架与输入输出

标签：C++17、标准输入输出、多组数据、EOF、getline、统一类型

一句话用途：每道题先抄这一页，得到稳定的 main + solve 外壳和常用读入方式。

题面触发词：多组数据、直到输入结束、每行一个字符串、字符串可能含空格、输出每组答案、标准输入输出。

什么时候用：所有 C++17 题目开写前；需要统一 long long、换行、数组 1-index 和多组数据结构时。

不要什么时候用：交互题需要按题面及时刷新输出；题面明确给出完全不同的框架时。

复杂度：读入输出本身是 $O(\text{输入规模} + \text{输出规模})$ 。

数据范围参考：数值、权值、答案、计数可能到 $1e18$ 时用 long long；下标、点数、边数通常用 int。

依赖的标准容器：vector、string、pair；全卷统一 ll = long long。

输入如何整理：数组默认整理成 `vector<ll> a(n + 1)`，使用下标 $1..n$ ；字符串保留 C++ 默认 $0..len-1$ 。

接口：

- void solve(): 处理一组数据。
- int main(): 只负责加速 IO、选择单组/多组/EOF 模式、调用 solve()。
- read_case(): 遇到 EOF 多组时可写成返回 bool 的读入函数。

输出能力: 使用 `cout << ans << '\n'`; 大量输出时也用 `'\n'`, 避免频繁强制刷新。

下游可接: 所有算法模块、数据结构模块、图论模块、DP 模块。

可拼接模块: C++-002 基础容器、C++-008 整数溢出、C++-009 语言坑。

模板代码:

考场默认可以用 GNU g++ 时, 第一行写:

```
#include <bits/stdc++.h>
```

如果现场环境不支持 `bits/stdc++.h`, 把它替换成下面这个“标准头文件安全包”。这组头文件覆盖本资料中常用的 STL 容器、算法、数学、格式化、字符串流、快读快写和断言。

```
#include <algorithm>
#include <array>
#include <bitset>
#include <cassert>
#include <cctype>
#include <climits>
#include <cmath>
#include <complex>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <deque>
#include <functional>
#include <iomanip>
#include <iostream>
#include <iterator>
#include <limits>
#include <list>
#include <map>
#include <numeric>
#include <queue>
#include <set>
#include <sstream>
#include <stack>
#include <string>
#include <tuple>
#include <type_traits>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>
```

头文件速查:

用到的东西	标准头
<code>cin/cout/ios</code>	<code><iostream></code>
<code>printf/scanf/getchar/putchar</code>	<code><cstdio></code>
<code>vector/string/array/list/deque</code>	<code><vector></code> 、 <code><string></code> 、 <code><array></code> 、 <code><list></code> 、 <code><deque></code>
<code>queue/stack/priority_queue</code>	<code><queue></code> 、 <code><stack></code>
<code>set/map/unordered_map/unordered_set</code>	<code><set></code> 、 <code><map></code> 、 <code><unordered_map></code> 、 <code><unordered_set></code>
<code>sort/lower_bound/unique/min/max</code>	<code><algorithm></code>
<code>iota/accumulate/partial_sum/gcd/lcm</code>	<code><numeric></code>
<code>pair/tuple</code>	<code><utility></code> 、 <code><tuple></code>
<code>greater/function/hash</code>	<code><functional></code>
<code>bitset</code>	<code><bitset></code>
<code>numeric_limits/LLONG_MAX/INT_MAX</code>	<code><limits></code> 、 <code><climits></code>
<code>setw/setfill/setprecision/fixed</code>	<code><iomanip></code>

用到的东西	标准头
stringstream	<sstream>
isdigit/tolower	<cctype>
sqrt/fabs	<cmath>
assert	<cassert>
make_unsigned 快写模板	<type_traits>

完整主骨架:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 1000000000;
const ll LINF = 4'000'000'000'000'000'000LL;

void solve() {
    int n;
    cin >> n;

    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    ll sum = 0;
    for (int i = 1; i <= n; i++) sum += a[i];

    cout << sum << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();          // 单组数据

    return 0;
}
```

调用示例:

// 题面写“第一行 T , 表示 T 组数据”时, 把 `main` 中的 `solve()` 换成:

```
int T;
cin >> T;
while (T--) {
    solve();
}
```

// 题面写“直到输入结束”时, 常用写法:

```
int n, m;
while (cin >> n >> m) {
    // 处理这一组  $n, m$ 
}
```

// 题面有整行字符串且前面刚用过 `cin >> x`:

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string line;
getline(cin, line);
```

常见坑:

- 如果 `#include <bits/stdc++.h>` 不可用, 不要现场猜头文件; 直接替换成上面的标准头文件安全包。

- 题面没有 T 时不要强读 T。
- `cin >> s` 读不到空格；含空格必须用 `getline(cin, s)`。
- `cin >> x` 后立刻 `getline`，要先用 `ignore` 吃掉行尾换行。
- `endl` 会刷新缓冲，大量输出时可能慢；普通换行用 `'\n'`。
- 不要依赖本地文件输入，OJ 默认使用标准输入输出。
- 不要写编译器私有优化指令；纸质模板以稳为主。
- 不要用宏把所有 `int` 替换成 `long long`，下标和容器大小仍用 `int` 更稳。

暴力/部分替代：先把读入和输出跑通；不会算法时也先输出合法格式，避免格式错误。

升级方向：把 `solve()` 内部替换为暴力、记忆化、DP、图论或数据结构正解；`main` 通常不动。

最小测试样例：

输入

```
3
10 20 30
```

输出

```
60
```

CPP-10：输入输出与格式化

模块编号：CPP-10

模块名称：输入输出与格式化

标签：[C++17][输入输出][格式化][快读快写][小数][对齐]

一句话用途：把 `cin/cout`、`scanf/printf`、快读快写和常见输出格式统一成考场可抄片段，避免格式错误丢分。

题面触发词：

- 多组数据、读到 EOF。
- 保留小数、四舍五入、允许误差。
- 右对齐、左对齐、补零、表格输出。
- 输入含空格字符串、整行、字符网格。
- 输入里有括号、逗号、冒号等固定标点。

什么时候用：

- 每道题开写前确认输入输出形态。
- 样例输出有固定小数位、列宽、补零或特殊标点。
- 数据量很大，普通 `cin/cout` 可能偏慢。
- 需要读整行或混合读数字和字符串。

不要什么时候用：

- 已经使用关闭同步的 `cin/cout` 时，不要再混用 `scanf/printf`。
- 普通数据量题不要先写复杂快读，`cin/cout` 关闭同步更稳。
- 字符串中包含空格时，不要用 `cin >> s` 期待读整行。

复杂度：

- 输入输出本身按字符数线性。
- 快读快写常数更小，但代码更长。

依赖的标准容器：

- `string`、`stringstream`、`vector`。
- 格式化输出使用 `<iomanip>`，`bits/stdc++.h` 已包含。

接口：

```
cin/cout: ios::sync_with_stdio(false); cin.tie(nullptr)
scanf/printf: %d, %lld, %lf, %.2f, %04d
readInt(x): 快读整数, 读到 EOF 返回 false
writeInt(x, end): 快写整数
getline(cin, line): 读整行
```

常见坑：

- endl 会刷新，普通换行用 '\n'。
- fixed << setprecision(2) 是小数点后 2 位，单独 setprecision(2) 是总有效数字。
- setw 只影响下一个输出项，setfill/left/right 会持续生效。
- scanf 读 double 用 %lf，printf 输出 double 用 %f。
- getline 前如果刚用过 cin >> x，要先 ignore 掉行尾换行。

暴力/部分替代：

- 复杂格式看不懂时，先整行 getline，再用 stringstream 或手动扫描。
- 输出格式不确定时，优先照样例保持空格和换行，避免额外调试输出。

考场目标

输入输出的目标不是优雅，是稳定拿分：

选一套 IO。

多组和 EOF 判断写对。

小数位数按题面。

空格、换行、补零、对齐按样例。

不要 freopen，不要文件 IO，不要 #pragma。

本模块默认 C++17、ACM/机考标准输入输出，允许 using namespace std。

1. 先选 IO 套路

场景	推荐
普通题、字符串多	cin/cout + 关闭同步
传统格式化多、全是基础类型	scanf/printf
极大整数输入输出	自写快读快写
要读整行、含空格字符串	getline
复杂标点格式	char 吃掉标点，或整行后 stringstream

口令：

不要把关闭同步后的 cin/cout 和 scanf/printf 混用。

endl 会刷新，普通换行用 '\n'。

setw 只影响下一个输出项，setfill/left/right 会持续生效。

1A. 分隔方式总表

最重要的判断：题目输入到底是“token 流”，还是“每一行有特殊含义”。

输入形态	C++ 推荐	说明
整数/单词由空格、换行、Tab 任意分隔	cin >> x	>> 会自动跳过所有空白，空格和换行等价
固定个数数组，但可能跨多行	for (...) cin >> a[i]	不要按行读，直接按 token 读最稳
第一行一个句子，含空格	getline(cin, line)	cin >> s 只能读到第一个空格
前面读过数字，后面马上读整行	cin.ignore(..., '\n'); getline(cin, line)	吃掉上一行末尾换行
允许跳过空行和前导空白后读一行	getline(cin >> ws, line)	会丢掉行首空格；题目要保留行首空格时不要用
一行里有不定个整数	getline + stringstream	行边界有意义时用
逗号/冒号/括号固定格式	char 接标点，或整行扫描	如 (12,34)、key: value
CSV/JSON/脚本等半结构化文本	整段 getline/读全文	翻 SIM-03/04/05

口令：

只要题面说“若干整数/字符串”，且没说每行含义不同，就用 cin >>。

只有行本身有意义，或字符串可能含空格，才用 getline。

空格和换行等价的例子

下面两份输入对 `cin >> n >> a[1] >> a[2] >> a[3]` 完全一样:

```
3
10 20 30

3 10
20
30
```

代码:

```
int n;
cin >> n;
static int a[100005];
for (int i = 1; i <= n; i++) cin >> a[i];
```

行边界有意义的例子

每一行一个表达式、日志、名字、句子时, 不能只用 `cin >>`。

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

for (int i = 1; i <= n; i++) {
    string line;
    getline(cin, line); // line 可以包含空格, 也可以是空行
    // process line
}
```

2. cin/cout 常用片段

快速 cin/cout 骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x;
        cin >> x;
        sum += x;
    }

    cout << sum << '\n';
    return 0;
}
```

多组数据

题面给 T:

```
int T;
cin >> T;
while (T--) {
    solve();
}
```

题面没给 T, 读到 EOF:

```
int n, m;
while (cin >> n >> m) {
    // write one case here
}
```

EOF 读到输入末尾就停止

EOF 题的口令:

不要先判断 `cin.eof()`。

直接尝试读取; 读成功就处理, 读失败就停止。

读未知个整数, 直到输入结束:

```
long long x;
while (cin >> x) {
    // use x
}
```

每组两个数, 直到输入结束:

```
int n, m;
while (cin >> n >> m) {
    cout << (n + m) << '\n';
}
```

每组结构复杂时, 写 `read_case()`:

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100000 + 5;
const int MAXM = 200000 + 5;

int n, m;
int a[MAXN], u[MAXM], v[MAXM];

bool read_case() {
    if (!(cin >> n >> m)) return false;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= m; i++) cin >> u[i] >> v[i];
    return true;
}

void solve_one_case() {
    long long sum = 0;
    for (int i = 1; i <= n; i++) sum += a[i];
    cout << sum << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    while (read_case()) {
        solve_one_case();
    }
    return 0;
}
```

注意: `read_case()` 里用到的 `n/m/a/u/v` 可以是全局变量, 也可以改成传引用。关键是第一句先判断这一组是否真的存在。

如果题面给的是“第一行 T”, 不要写 EOF 循环; 如果题面没给 T, 不要强读 T。

小数保留位数

```
double x;
cin >> x;

cout << fixed << setprecision(2) << x << '\n'; // 保留 2 位小数
```

提醒:

fixed + setprecision(2): 小数点后 2 位。
只有 setprecision(2): 总有效数字 2 位。

右对齐、左对齐、补零

```
int x = 7;
string name = "Tom";

cout << setw(5) << x << '\n'; // 默认右对齐: 7
cout << right << setw(5) << x << '\n'; // 右对齐
cout << left << setw(10) << name << "!" << '\n'; // 左对齐

cout << right << setfill('0') << setw(4) << x << '\n'; // 0007
cout << setfill(' '); // 记得恢复空格填充
```

表格输出:

```
cout << left << setw(12) << "name"
    << right << setw(5) << "score" << '\n';

cout << left << setw(12) << name
    << right << setw(5) << 98 << '\n';
```

3. scanf/printf 常用片段

基础骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x;
        if (scanf("%lld", &x) != 1) return 0;
        sum += x;
    }

    printf("%lld\n", sum);
    return 0;
}
```

常用格式符

类型	scanf	printf
int	%d	%d
long long	%lld	%lld
double	%lf	%f
char	%c	%c
C 字符串 char[]	%100s	%s

scanf 读 char[] 时要给宽度, 例如 char s[105]; scanf("%100s", s);。裸 %s 在输入过长时可能越界; 考场更推荐 string s; cin >> s;。

小数、宽度、补零

```
double x = 3.14159;
int a = 7;

printf("%.2f\n", x); // 3.14
printf("%5d\n", a); // 宽度 5, 右对齐:      7
printf("%04d\n", a); // 宽度 4, 前面补 0: 0007
```

多组数据:

```
int T;
scanf("%d", &T);
while (T--) {
    solve();
}
```

直到 EOF:

```
int n, m;
while (scanf("%d%d", &n, &m) == 2) {
    // write one case here
}
```

4. 自写快读快写

只在输入输出量特别大、且主要是整数时使用。用了它就全程用它, 不要再混 cin。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

template <class T>
bool readInt(T &x) {
    int c = getchar();
    if (c == EOF) return false;

    while (c != '-' && (c < '0' || c > '9')) {
        c = getchar();
        if (c == EOF) return false;
    }

    int neg = 0;
    if (c == '-') {
        neg = 1;
        c = getchar();
    }

    using U = make_unsigned_t<T>;
    U y = 0;
    while (c >= '0' && c <= '9') {
        y = y * 10 + (U)(c - '0');
        c = getchar();
    }

    if (neg) x = (T)(U(0) - y);
    else x = (T)y;
    return true;
}

template <class T>
void writeInt(T x, char end = '\n') {
```

```

    if (x == 0) {
        putchar('0');
        putchar(end);
        return;
    }

    using U = make_unsigned_t<T>;
    U y;
    if (x < 0) {
        putchar('-');
        y = U(0) - (U)x; // 避免 LLONG_MIN 取负溢出
    } else {
        y = (U)x;
    }

    char s[50]; // Long Long 足够; __int128 请用 CPP-008 的专门打印模板
    int top = 0;
    while (y > 0) {
        s[top++] = char('0' + y % 10);
        y /= 10;
    }
    while (top--) putchar(s[top]);
    putchar(end);
}

int main() {
    int n;
    if (!readInt(n)) return 0;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x = 0;
        if (!readInt(x)) return 0;
        sum += x;
    }

    writeInt(sum);
    return 0;
}

```

EOF 多组:

```

int n, m;
while (readInt(n) && readInt(m)) {
    // write one case here
}

```

scanf 版 EOF:

```

int n, m;
while (scanf("%d%d", &n, &m) == 2) {
    printf("%d\n", n + m);
}

```

scanf 返回成功读到的变量个数; 读两个整数就检查是否等于 2。不要只写 != EOF, 因为输入残缺时可能只读到 1 个。

5. 字符、整行与换行处理

读一个非空白字符

cin >> c 会跳过空格和换行:

```

char c;
cin >> c;

```

scanf(" %c", &c) 前面的空格会跳过所有空白:

```
char c;
scanf(" %c", &c);
```

读下一个原始字符

需要读空格或换行本身:

```
char c;
cin.get(c);

char c;
scanf("%c", &c);
```

读整行

```
string line;
getline(cin, line);
```

如果前面刚用过 `cin >> n`, 要先吃掉本行剩余换行:

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
string line;
getline(cin, line);
```

读到 EOF 的整行:

```
string line;
while (getline(cin, line)) {
    // process line
}
```

空行不是 EOF。getline 读到空行时 `line == ""` 但循环仍然成立; 只有真的没有下一行时循环才结束。

读完整剩余输入, 包括换行和空格:

```
string text((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
```

如果前面刚读过一个模式名, 还要读后面整段文本:

```
string mode;
cin >> mode;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
string text((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
```

6. 复杂格式处理

标点固定: 用 char 接住

输入形如 (12,34):

```
char l, comma, r;
int x, y;
cin >> l >> x >> comma >> y >> r;
```

或者:

```
int x, y;
scanf(" (%d,%d)", &x, &y);
```

一行里有不定个整数

```
string line;
while (getline(cin, line)) {
    stringstream ss(line);
    int x;
    while (ss >> x) {
        // use x
    }
}
```

```

    }
}

```

简单逗号分隔

只适用于题目保证没有引号包裹、字段中不含逗号的简单格式。真正 CSV 翻 SIM-04。

```

string line;
getline(cin, line);

stringstream ss(line);
string cell;
vector<string> cells(1); // 1-index
while (getline(ss, cell, ',')) {
    cells.push_back(cell);
}

```

key=value 或 key: value

```

string line;
getline(cin, line);

int p = line.find('=');
if (p != (int)string::npos) {
    string key = line.substr(0, p);
    string value = line.substr(p + 1);
}

```

冒号同理把 '=' 改成 ':'。若要去掉两侧空格，翻 STR-01 或手写 trim。

读带空格的名字和值

输入每行形如 Tom Hanks 98，最后一个分数：

```

string line;
getline(cin, line);

stringstream ss(line);
vector<string> parts;
string word;
while (ss >> word) parts.push_back(word);
if (!parts.empty()) {
    int score = stoi(parts.back());
    parts.pop_back();

    string name;
    for (int i = 0; i < (int)parts.size(); i++) {
        if (i) name += ' ';
        name += parts[i];
    }
}

```

网格读入

无空格字符网格：

```

int n, m;
cin >> n >> m;
vector<string> g(n + 1);
for (int i = 1; i <= n; i++) {
    string row;
    cin >> row;
    g[i] = " " + row; // 之后访问 g[i][j], i=1..n, j=1..m
}

```

有空格分隔的字符网格：

```

const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static char g[MAXN][MAXM];

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        cin >> g[i][j];
    }
}

```

7. 提交前格式检查

题面有没有 T? 没有就别强读 T。
 EOF 题 while 条件是否判断了读入成功?
 小数是保留几位, 还是允许误差?
 每个答案后有没有换行?
 多个数之间是空格还是换行?
 printf 的 long long 是否用 %lld?
 scanf 读 double 是否用 %lf?
 getline 前是否处理了上一次 cin 留下的换行?
 setfill('0') 后是否恢复 setfill(' ')?
 有没有 endl、调试输出、freopen、文件 IO、#pragma?

CPP-013 STL 容器成员函数速查

模块编号: CPP-013

模块名称: STL 容器成员函数速查

标签: STL、vector、deque、list、array、string、stack、queue、priority_queue、set、multiset、map、multimap、unordered_map、unordered_set、pair、tuple、iterator

一句话用途: 在考场上快速确认 STL 容器该用哪个成员函数、复杂度大致是多少、哪些写法安全可抄。

题面触发词: 数组、队列、栈、堆、集合、映射、去重、计数、前驱后继、哈希表、字符串、二元组、多元组、遍历删除。

什么时候用: 已经知道算法方向, 但卡在容器接口、删除写法、堆排序规则、哈希表预留空间或 pair/tuple 写法时。

不要什么时候用: 需要自己实现线段树、树状数组、DSU、图算法主体时, 本模块只提供 STL 接口速查, 不替代算法模块。

复杂度: 顺序容器按位置访问/插入删除不同; 红黑树容器 $O(\log n)$; 哈希容器平均 $O(1)$; 堆 push/pop $O(\log n)$ 、top $O(1)$ 。

数据范围参考: $n \leq 2e5$ 时 STL 容器通常可用; 哈希表大量插入前建议 reserve 和 max_load_factor; 需要稳定最坏复杂度时优先有序容器。

依赖的标准容器: vector、deque、list、array、string、stack、queue、priority_queue、set、multiset、map、multimap、unordered_map、unordered_set、pair、tuple。

输入如何整理:

- 连续下标、DP 表、邻接表: 优先 vector。
- 两端进出、单调队列、0-1 BFS: 优先 deque。
- 先进先出: queue; 后进先出: stack。
- 每次取最大/最小: priority_queue。
- 自动排序、去重、前驱后继: set/map。
- 只按 key 快速查找/计数: unordered_map/unordered_set。
- 多字段排序或存状态: pair/tuple, 字段多且含义复杂时改 struct。

接口:

- 通用查询: c.empty()、c.size()。
- 端点访问: c.front()、c.back(); 栈和堆用 c.top()。
- 清空: c.clear(); 注意 array 没有 clear()。
- 改大小: vector/string/deque 常用 resize(); array 大小固定。
- 替换内容: vector/string/deque/list 可用 assign()。
- 预留容量: vector/string/unordered_map/unordered_set 常用 reserve()。

输出能力：输出容器元素、计数结果、映射值、堆顶、集合最小/最大值、排序后的记录字段。

下游可接：排序、二分、前缀和、BFS、Dijkstra、贪心、DP、离散化、扫描线、记忆化搜索。

可拼接模块：CPP-002 基础容器、CPP-003 排序二分、CPP-004 队列栈堆、CPP-005 关联容器、CPP-007 坐标压缩、CPP-009 常见 RE/WA。

顺序容器简表

容器	常用用途	常用接口	关键提醒
vector<T>	数组、表、邻接表、DP	push_back、pop_back、back、resize、reserve、assign、clear	支持 a[i]；中间插删慢；扩容会使旧迭代器/引用/指针失效
deque<T>	两端队列、单调队列、0-1 BFS	push_front、push_back、pop_front、pop_back、front、back	支持 dq[i]；两端快，中间插删仍不适合大量使用
list<T>	已知迭代器位置的频繁插删	push_back、push_front、insert、erase、sort、splice	不支持 a[i]；遍历只能用迭代器；算法题较少用
array<T, N>	固定小数组、方向数组	fill、front、back、begin、end、size	大小编译期固定；没有 push_back/clear/resize
string	字符串、字符数组	push_back、pop_back、substr、find、resize、reserve、clear	size() 是无符号类型；空串不能 s[0]、front、back

通用成员函数速查

函数	作用	可抄写法	注意
empty() size() clear()	是否为空 元素个数 清空元素	if (c.empty()) int n = (int)c.size(); c.clear();	访问端点前先判空 和 int 比较时常强转 vector 容量通常不变；array 没有
front()	第一个元素	int x = c.front();	vector/deque/list/array/string/queue 支持，非空才可用
back() top()	最后一个元素 栈顶/堆顶	int x = c.back(); int x = st.top();	queue 也有 back() 只用于 stack/priority_queue
resize(n) resize(n, v) reserve(n)	改变元素个数 改大小并指定新增值 预留容量	a.resize(n + 1); a.resize(n, -1); a.reserve(n);	变大补默认值，变小删除尾部 只影响新增元素，不会重置旧元素 不改变 size()，不能直接访问新位置
assign(n, v) begin()/end()	替换为 n 个 v 半开区间迭代器	a.assign(n + 1, 0); for (auto it = c.begin(); it != c.end(); ++it)	会清除原内容 end() 不是最后一个元素

vector/deque/list/array/string 常用模板

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> a;
    a.reserve(n); // 只预留容量, size 仍是 0
    for (int i = 0; i < n; i++) {
        int x;
```

```

    cin >> x;
    a.push_back(x);
}

a.resize(n + 1, 0);           // size 变为 n + 1, 新增位置补 0
a.assign(n + 1, -1);         // 整个 vector 变成 n + 1 个 -1
a.clear();                    // size 变 0

deque<int> dq;
dq.push_back(1);
dq.push_front(2);
if (!dq.empty()) {
    cout << dq.front() << ' ' << dq.back() << '\n';
}

list<int> ls = {3, 1, 2};
ls.sort();                    // List 不能用 sort(ls.begin(), ls.end())

array<int, 4> dir = {0, 1, 0, -1};
dir.fill(0);

string s = "abc";
s.push_back('d');
cout << s.substr(1, 2) << '\n'; // 从下标 1 开始取 2 个字符, 输出 bc

return 0;
}

```

stack/queue/priority_queue

容器	用途	插入	查看	删除	备注
stack<T>	后进先出	push(x)	top()	pop()	括号、DFS 模拟、撤销
queue<T>	先进先出	push(x)	front()	pop()	BFS; 也可 back() 看队尾
priority_queue<T> 默认最大堆		push(x)	top()	pop()	每次取最大
priority_queue<T, vector<T>, greater<T>>	最小堆	push(x)	top()	pop()	每次取最小

```

stack<int> st;
st.push(1);
if (!st.empty()) {
    int x = st.top();
    st.pop();
}

queue<int> q;
q.push(1);
if (!q.empty()) {
    int u = q.front();
    q.pop();
}

priority_queue<int> max_heap;
max_heap.push(5);
max_heap.push(2);
cout << max_heap.top() << '\n'; // 5

priority_queue<int, vector<int>, greater<int>> min_heap;
min_heap.push(5);

```



```
min_heap.push(2);
cout << min_heap.top() << '\n'; // 2
```

priority_queue 存 pair 与自定义排序

```
using pii = pair<int, int>;

// pair 默认字典序: 先比较 first, 再比较 second.
// 最小堆常用于 Dijkstra: {距离, 点}
priority_queue<pii, vector<pii>, greater<pii>> pq;
pq.push({0, 1});
while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
}

struct Node {
    int dist, id;
};

struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        if (a.dist != b.dist) return a.dist > b.dist; // dist 小的优先
        return a.id > b.id; // id 小的优先
    }
};

priority_queue<Node, vector<Node>, Cmp> heap;
heap.push({3, 2});
heap.push({1, 5});
```

set/multiset

容器	特点	常用接口	适合场景
set<T>	自动排序, 自动去重	insert、erase、find、count、lower_bound、upper_bound	去重、有序遍历、前驱后继
multiset<T>	自动排序, 允许重复	同 set	动态维护一堆数, 重复值要保留

```
set<int> s;
s.insert(3);
s.insert(1);

if (s.count(3)) {
    cout << "exist\n";
}

auto it = s.lower_bound(2); // 第一个 >= 2
if (it != s.end()) cout << *it << '\n';

// 前驱: 严格小于 x 的最大值
int x = 3;
auto p = s.lower_bound(x);
if (p != s.begin()) {
    --p;
    cout << *p << '\n';
}

multiset<int> ms;
ms.insert(5);
ms.insert(5);

// multiset 只删除一个 5
```

```

auto one = ms.find(5);
if (one != ms.end()) {
    ms.erase(one);
}

```

// ms.erase(5) 会删除所有 5

map/multimap

容器	特点	常用接口	适合场景
map<K, V>	key 自动排序且唯一	mp[k]、insert、find、count、erase、lower_bound	映射、计数、有序 key 查询
multimap<K, V>	key 自动排序且可重复	insert、equal_range、find、erase	一个 key 对应多条记录且需要有序

```

map<string, int> cnt;
cnt["alice"]++;

if (cnt.find("bob") == cnt.end()) {
    cout << "bob not found\n";
}

for (auto [key, value] : cnt) {
    cout << key << ' ' << value << '\n';
}

multimap<int, string> mm;
mm.insert({90, "alice"});
mm.insert({90, "bob"});

auto range = mm.equal_range(90);
for (auto it = range.first; it != range.second; ++it) {
    cout << it->first << ' ' << it->second << '\n';
}

```

unordered_map/unordered_set

容器	特点	常用接口	适合场景
unordered_set<T>	哈希集合，不保证顺序	insert、erase、find、count、reserve、max_load_factor	快速判重、存在性
unordered_map<K, V>	哈希映射，不保证顺序	mp[k]、find、count、erase、reserve、max_load_factor	快速计数、快速查值

```

int n;
cin >> n;

unordered_map<long long, int> cnt;
cnt.max_load_factor(0.7);
cnt.reserve(n * 2 + 1);

for (int i = 1; i <= n; i++) {
    long long x;
    cin >> x;
    cnt[x]++;
}

unordered_set<int> seen;
seen.max_load_factor(0.7);
seen.reserve(n * 2 + 1);

```

```

seen.insert(10);
if (seen.find(10) != seen.end()) {
    cout << "seen\n";
}

```

pair/tuple

```

using pii = pair<int, int>;
using tiii = tuple<int, int, int>;

pii p = {3, 5};
cout << p.first << ' ' << p.second << '\n';

auto [x, y] = p;

tuple<int, int, long long> state = {1, 2, 100LL};
auto [i, j, val] = state;

vector<pair<int, int>> v = {{2, 3}, {1, 9}, {1, 4}};
sort(v.begin(), v.end());           // 默认先 first 升序, 再 second 升序

// 自定义排序: first 升序; first 相同时 second 降序
sort(v.begin(), v.end(), [](const pii& a, const pii& b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
});

```

迭代器遍历与 erase 安全写法

```

// vector/string/deque/list/set/map/unordered_* 通用: erase 返回下一个合法迭代器
for (auto it = c.begin(); it != c.end(); ) {
    if (need_delete(*it)) {
        it = c.erase(it);
    } else {
        ++it;
    }
}

// map/unordered_map 删除时, 元素是 pair<const K, V>
for (auto it = mp.begin(); it != mp.end(); ) {
    if (it->second == 0) {
        it = mp.erase(it);
    } else {
        ++it;
    }
}

// 不要在 range-for 中直接 erase 当前容器
// 错误示例:
// for (int x : v) {
//     if (x < 0) v.erase(...);
// }

// vector 按条件删除的可靠写法
v.erase(remove(v.begin(), v.end(), value), v.end());

v.erase(remove_if(v.begin(), v.end(), [](int x) {
    return x < 0;
}), v.end());

```

常用完整模板

```

#include <bits/stdc++.h>
using namespace std;

```

```

using ll = long long;
using pii = pair<int, int>;
const int MAXN = 200000 + 5;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    static int sorted[MAXN];

    unordered_map<int, int> cnt;
    cnt.max_load_factor(0.7);
    cnt.reserve(n * 2 + 1);

    priority_queue<pii, vector<pii>, greater<pii>> min_heap;
    set<int> unique_values;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        sorted[i] = x;
        cnt[x]++;
        unique_values.insert(x);
        min_heap.push({x, i});
    }

    sort(sorted + 1, sorted + n + 1);

    while (!min_heap.empty()) {
        auto [value, id] = min_heap.top();
        min_heap.pop();
        cout << value << ' ' << id << '\n';
    }

    return 0;
}

```

调用示例:

```

// 1-index 数组: 上限明确时优先静态数组
static int a[MAXN];
for (int i = 1; i <= n; i++) cin >> a[i];

// 邻接表: 上限明确时直接静态数组, 每个点一个 vector
static vector<int> g[MAXN];
g[u].push_back(v);

// 固定方向数组
array<int, 4> dx = {-1, 0, 1, 0};
array<int, 4> dy = {0, 1, 0, -1};

// map 的 lower_bound: 第一个 key >= x
auto it = mp.lower_bound(x);
if (it != mp.end()) {
    cout << it->first << ' ' << it->second << '\n';
}

```

常见坑:

- front/back/top 前必须先确认非空。
- reserve(n) 只改容量, 不改元素个数; 写完 reserve 后不能用 a[i] = x 填新元素。

- `resize(n, v)` 只给新增位置填 `v`，旧位置不会被重置。
- `assign(n, v)` 会替换整个容器内容，旧内容全部消失。
- `clear()` 后 `size()` 为 0，但 `vector/string` 的容量通常还在。
- `array` 大小固定，没有 `push_back`、`pop_back`、`clear`、`resize`。
- `priority_queue` 默认最大堆；小根堆写 `priority_queue<T, vector<T>, greater<T>>`。
- `pair` 默认排序是字典序，先 `first` 后 `second`。
- `set/map` 的 `lower_bound` 是按有序 `key` 查；`unordered_map/unordered_set` 没有 `lower_bound`。
- `unordered_map/unordered_set` 大量插入前，推荐先 `max_load_factor(0.7)` 再 `reserve(n * 2 + 1)`。
- `mp[key]` 会在 `key` 不存在时创建默认值；只判断存在用 `find` 或 `count`。
- `multiset.erase(x)` 删除所有等于 `x` 的元素；只删一个要先 `find`，存在再 `erase(it)`。
- 遍历时删除元素要写 `it = c.erase(it)`，不要在 `range-for` 中删除当前容器。
- `string::find()` 找不到时返回 `string::npos`，不要拿它和 `-1` 混用。
- `list` 不支持随机访问，也不能用普通 `sort(begin, end)`，要用成员函数 `ls.sort()`。

暴力/部分替代：小数据可以用 `vector` 存全部元素，每次线性扫描、排序或手动删除；确认思路后再替换成 `set/map/unordered_map/priority_queue`。

升级方向：哈希计数接记忆化搜索；`set/map` 接扫描线和离线查询；`priority_queue` 接 Dijkstra、贪心合并；`vector` 接前缀和、树状数组、线段树和 DP。

最小测试样例：

输入

```
3
5 2 5
```

输出

```
2 2
5 1
5 3
```

BRUTE-01：复杂度与数据范围速查

模块编号：BRUTE-01

模块名称：复杂度与数据范围速查

标签：复杂度、数据范围、算法选择、时间预算

一句话用途：用数据范围快速判断暴力、记忆化、折半或正式算法是否值得写。

题面触发词：

- `1 <= n <= ...`
- `T` 组数据。
- 时间限制 `1s/2s/3s`。
- 子任务中出现小范围。

适用场景：

- 写代码前判断版本能拿哪一档分。
- TLE 后判断是算法复杂度问题还是常数问题。
- 在暴力和记忆化之间做取舍。

什么时候用：

- 每道题读完输入范围后立刻用。
- 每次新增一层循环或一维状态时用。

不要什么时候用：

- 不要只看 `n`，还要看 `T`、边数 `m`、值域 `W`、状态转移数量。
- 不要把 $O(2^n)$ 当作永远不能写，小数据子任务正是它的目标。

复杂度：

本模块本身无运行复杂度；用于估算其他模块。

数据范围参考：

估计操作数	通常可行性
$\leq 1e6$	很稳
$1e7$	通常可过
$1e8$	C++ 勉强, 常数要小
$> 1e8$	大概率 TLE

数据范围	常见可用方案
$n \leq 10$	$n!$ 、复杂回溯
$n \leq 20$	2^n 、状压、子集枚举
$n \leq 25$	DFS + 剪枝 / memo
$n \leq 40$	折半枚举 $2^{(n/2)}$
$n \leq 300$	$O(n^3)$
$n \leq 3000$	$O(n^2)$
$n \leq 2e5$	$O(n \log n)$ 或 $O(n)$

依赖的标准容器:

- 无强依赖。
- 估算状态时常配合 `vector`、`map`、`unordered_map`。

输入如何整理:

先在草稿纸上列出:

$n =$
 $m =$
 $T =$
 值域/容量 $W =$
 状态维度 $=$
 每个状态转移数 $=$

接口:

// 估算总操作数: 状态数 * 每状态转移数 * 测试组数。

```
__int128 estimate_ops(long long states, long long transitions, long long T) {
    return (__int128)states * transitions * T;
}
```

输出能力:

- 判断某个版本是否适合提交。
- 确定优先写暴力、memo、折半还是换模型。

下游可接:

- 所有 BRUTE 模块。
- DP 卷的数据范围路由。
- 图论卷的最短路、连通性、拓扑模块。

可拼接模块:

- BRUTE-00 部分分总策略。
- BRUTE-07 记忆化搜索总论。
- BRUTE-12 折半枚举。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

string judge_ops(long double ops) {
    if (ops <= 1e6) return "very_safe";
    if (ops <= 1e7) return "safe";
    if (ops <= 1e8) return "maybe";
    return "danger";
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ll states, transitions, T;
    cin >> states >> transitions >> T;
    long double ops = (long double)states * transitions * T;
    cout << judge_ops(ops) << '\n';
    return 0;
}
```

调用示例:

```
// 0/1 背包 memo: n * W 个状态, 每个状态 2 个转移。
long double ops = (long double)n * W * 2;
if (ops <= 1e8) {
    // vector memo 可以尝试
}
```

常见坑:

- 忘记乘 T。
- 忘记每个状态还要枚举 k 或邻接边。
- map / unordered_map 常数比数组大很多。
- 2^{25} 看起来不大, 但转移复杂时会爆。
- 递归深度和内存也要算, 不只算时间。

暴力/部分替代:

- 若 $n \leq 20$ 子任务存在, 先写子集或 DFS。
- 若 $n \leq 40$, 优先考虑折半。
- 若 $n * W$ 可承受, 优先写 vector memo。
- 若状态不清晰, 用 map<tuple,...> 先拿中档。

升级方向:

- $O(n!)$ -> 回溯剪枝 / 状压 DP。
- $O(2^n)$ -> 折半 / 状压 DP / 记忆化。
- $O(n^3)$ -> 前缀和 / 单调队列 / 数据结构优化。
- map memo -> vector memo 或编码 unordered_map。

最小测试样例:

输入:

1000 1000 1

输出:

very_safe

BRUTE-07: 记忆化搜索总论

模块编号: BRUTE-07

模块名称: 记忆化搜索总论

标签: 记忆化、DFS、状态、DP 入口、核心章节

一句话用途: 把暴力 DFS 中重复计算的相同状态缓存起来, 用最小改动得到中档甚至满分版本。

题面触发词:

- 每一步有选择。
- 暴力 DFS 能写出来。
- 同一个局面会从不同路径到达。
- 表推 DP 循环顺序想不清。
- $n * W$ 、mask * last、pos * tight * state 等状态数可估算。

适用场景:

- 背包、区间、树、DAG、数位、状压等 DP 的递归写法。
- 递归参数能完整描述后续问题。
- 状态总数远小于搜索树节点数。

什么时候用：

- 已经能写出 `dfs(...)`。
- `dfs` 的返回值只由参数决定。
- 相同参数组合会重复出现。
- 状态数量可承受。

不要什么时候用：

- `dfs` 返回值依赖没有写进参数的全局变量。
- 相同参数下，后续答案还会因为路径不同而不同。
- 状态有环且没有环检测。
- 状态几乎不重复，memo 只会增加常数。
- 递归深度可能爆栈且无法改写。

复杂度：

记忆化复杂度 = 状态数 * 每个状态的转移数

空间复杂度 = 状态数

数据范围参考：

- $n * W \leq 1e7$ ：数组/vector memo 可尝试。
- 状态数量 $\leq 1e5 \sim 1e6$ ：map<tuple> 可作为稳妥版。
- 状态数量较多且能安全编码：unordered_map。
- mask 状态通常要求 $n \leq 20$ 。

依赖的标准容器：

- vector
- map
- unordered_map
- tuple

输入如何整理：

- 把 DFS 参数尽量整理成整数：i, rest, last, mask, cnt。
- 若参数有负数，用 OFFSET 平移，或改用 map<tuple,...>。
- 若参数是集合，用 mask 表示。

接口：

```
// 固定句式：
// dfs(状态参数) 返回：从这个状态继续走，能得到的答案。
long long dfs(int i, int rest);
```

输出能力：

- 最大值。
- 最小值。
- 方案数。
- 可行性。
- 也可保存选择用于还原方案。

下游可接：

- DP 卷：把 DFS 参数变成 DP 下标。
- BRUTE-08 vector memo。
- BRUTE-09 map memo。
- BRUTE-10 unordered_map 编码 memo。

可拼接模块：

- BRUTE-04 组合 DFS。
- BRUTE-05 子集枚举。
- BRUTE-06 回溯剪枝。
- BRUTE-15 常见坑。

模板代码：


```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);

int n, W;
vector<int> cost;
vector<ll> value;
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int i, int rest) {
    if (rest < 0) return -LINF; // 1. 先判非法, 防止数组越界
    if (i == n + 1) return 0; // 2. 再判终止

    if (vis[i][rest]) return memo[i][rest]; // 3. 查缓存
    vis[i][rest] = 1;

    ll ans = -LINF;
    ans = max(ans, dfs(i + 1, rest)); // 不选
    if (cost[i] <= rest) {
        ans = max(ans, value[i] + dfs(i + 1, rest - cost[i])); // 选
    }

    return memo[i][rest] = ans; // 4. 存缓存
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    memo.assign(n + 2, vector<ll>(W + 1, 0));
    vis.assign(n + 2, vector<int>(W + 1, 0));

    cout << dfs(1, W) << '\n';
    return 0;
}

```

调用示例:

```

// 暴力版本:
// ll dfs(int i, int rest);
// 记忆化版本只加三件事:
// 1. memo/vis 容器
// 2. if (vis[state]) return memo[state]
// 3. return memo[state] = ans
cout << dfs(1, W) << '\n';

```

状态能否缓存判断:

1. dfs 参数是什么?
2. dfs 返回值是什么?
3. 给定这些参数后, 未来答案是否唯一?
4. 是否还依赖当前路径、已选集合、上一个元素、剩余次数、颜色、方向?
5. 如果依赖, 把它加入参数; 如果无法加入, 不要缓存。

可缓存例子:

dfs(i, rest) = 从第 i 个物品开始, 剩余容量 rest 的最大价值。
 给定 i 和 rest 后, 前面怎么选不影响后面, 所以可缓存。

不可缓存例子:

`dfs(i, sum)` 里还用全局 `vector<int> chosen` 判断相邻冲突。
如果 `chosen` 没有进入参数, 相同 `i` 和 `sum` 可能有不同后续, 不能缓存。

返回值四件套:

最大值: `ans = -LINf`; 非法返回 `-LINf`; 转移用 `max`。
最小值: `ans = LINf`; 非法返回 `LINF`; 转移用 `min`。
方案数: `ans = 0`; 成功边界返回 `1`; 非法返回 `0`; 转移用加法取模。
可行性: `ans = false`; 成功边界返回 `true`; 非法返回 `false`; 转移用 `||`。

有环状态风险:

普通记忆化默认状态依赖是无环的。如果 `dfs(a)` 可能还没算完又调用回 `dfs(a)`, 只用 `vis` 会出错。

```
// 0 = 未访问, 1 = 正在访问, 2 = 已完成
vector<int> color;

bool dfs_cycle(int u) {
    if (color[u] == 1) return false; // 发现环, 按题意处理
    if (color[u] == 2) return true;
    color[u] = 1;
    // for (int v : g[u]) if (!dfs_cycle(v)) return false;
    color[u] = 2;
    return true;
}
```

常见坑:

- 非法状态晚于 `memo` 查询, 导致数组下标越界。
- 漏掉 `last`、`mask`、`cnt` 等影响未来的参数。
- 多测不清空 `memo`。
- 最大值题初始化为 `0`, 全负数时错。
- 计数题忘记取模。
- 有环状态直接递归, 死循环。
- `vis=1` 表示“算完”, 不要在状态仍在计算中就当答案可用。

暴力/部分分替代:

- 没有重复状态: 保留 DFS + 剪枝。
- 状态范围不清: 先用 `map<tuple, ...>`。
- 状态范围清楚: 换 `vector`。
- 表推顺序不会: 保留记忆化提交。

升级方向:

暴力 DFS -> 记忆化 DFS -> 表推 DP -> 滚动数组/数据结构优化

最小测试样例:

输入:

```
3 5
3 10
4 20
2 8
```

输出:

```
20
```

DP-00: DP 总流程

模块编号: DP-00

模块名称: DP 总流程

标签: DP、路由、状态设计、部分分、记忆化

一句话用途: 遇到疑似 DP 题时, 按固定步骤从题面信号走到暴力 DFS、记忆化或表推 DP。

题面触发词：

- “最大/最小/方案数/是否存在”
- “前 i 个” “走到第 i 个” “容量/预算/次数限制”
- “两个序列” “区间合并” “树上选择” “集合访问” “数字范围 $1..N$ ”
- 数据范围不像暴力，但状态有重复

什么时候用：

- 题目要求全局最优、计数或可行性。
- 每一步有选择，且后续答案只依赖少量状态。
- 暴力搜索会反复到达相同局面。

不要什么时候用：

- 最少步数且每条边代价相同，优先 BFS。
- 明显是排序、贪心或数据结构维护，不需要记录历史状态。
- 状态会形成正权/负权环，不能直接套普通 DP。

复杂度：

- 估算公式：状态数 * 每个状态转移数。
- 表推 DP 与记忆化 DFS 的理论状态数通常一致，常数不同。

数据范围参考：

数据范围	优先考虑
$n \leq 20$	DFS、记忆化、状压 DP
$n \leq 40$	折半枚举、状态压缩、小维 DP
$n \leq 300$	区间 DP、Floyd 类 $O(n^3)$
$n \leq 3000/5000$	$O(n^2)$ 线性 DP、LCS、朴素 LIS
$n \leq 2e5$	$O(n \log n)$ 、滚动数组、DP + 数据结构优化

依赖的标准容器：

- `vector<ll>`、`vector<vector<ll>>`
- `map<tuple<...>, ll>`：状态复杂时先用它保命。
- `queue<int>`：DAG 拓扑或 BFS 状态搜索。
- Graph：树形 DP、DAG DP。

输入如何整理：

- 序列：整理成 `vector<ll> a(n + 1)`，默认 1-index。
- 字符串：默认 0-index；做 LCS/编辑距离时常用 `s[i - 1]`。
- 图/树：优先使用统一 Graph；若为了速抄写局部 `g`，必须注明它是从 `G.g` 抽出的简化邻接表。
- 区间代价：优先预处理前缀和或 `cost(l, r)`。

接口：

```
using ll = long long;
const ll LINF = 4'000'000'000'000'000'000LL; // 与主骨架统一，作为 Long Long 无穷大哨兵

// 最大值题：不可达用 -LINF
// 最小值题：不可达用 LINF
// 计数题：初始 0，成功边界 1，必要时取 MOD
// 可行性题：bool / char
```

输出能力：

- 最大值/最小值
- 方案数
- 是否可行
- 可选：记录路径或选择方案

下游可接：

- DP-01 路由表
- DP-02 状态句式库
- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-03B/DP-26 状态增维/升维：状态有后效性时使用。

- 各 DP 模型卡片

可拼接模块：

- BRUTE 记忆化搜索
- PrefixSum / 树状数组 / SegmentTree
- Graph / Topo / Tree
- Compressor
- Floyd / Dijkstra

考场总流程

1. 看数据范围
-> 判断能不能暴力、能不能 $O(n^2)$ 、是否需要优化。
2. 看处理对象
-> 序列 / 背包 / 两个序列 / 网格 / 区间 / 树 / DAG / 集合 / 数字位。
3. 查 DP 路由表
-> 先匹配现成模型，不空想。
4. 套状态句式
-> “处理到哪里 + 还记住什么 + dp 值代表什么”。
5. 写初始化
-> 边界状态是什么，不合法状态用什么值。
6. 写转移
-> 当前状态从哪里来，或当前状态能去哪里。
7. 确认答案位置
-> $dp[n]$ / $\max(dp)$ / $dp[n][m]$ / $dp[full][last]$ / $dfs(0, \dots)$ 。
8. 先交能写对的版本
-> 暴力 DFS -> 记忆化 -> 表推 -> 优化。

五问检查卡

状态表示什么？
初始状态是什么？
转移从哪里来？
循环顺序是否保证依赖已计算？
答案从哪里取？

建模 8 问：从题面到代码前必须过一遍

问题	要写下来的答案	常见例子
1. 处理对象是什么？	序列、两个字符串、网格、区间、树、集合、数字位	背包是物品序列，编辑距离是两个字符串
2. 进度维度是什么？	$i, i, j, l, r, u, \text{mask}, \text{pos}$	LCS 是 i, j ，区间 DP 是 l, r
3. 还要记住什么？	容量、数量、上一状态、是否用过机会、余数	有后效性就去 DP-03B/DP-26 升维
4. dp 值是什么类型？	最大值、最小值、方案数、可行性	决定初值是 $-\text{LINF}/\text{LINF}/0/\text{false}$
5. base case 是什么？	空前缀、容量 0、起点、叶子、空集合	背包 $dp[0]=0/1/\text{true}$
6. 状态从哪里来？	前驱状态 pull，或当前状态 push 到后继	网格从上/左来，背包从旧容量来
7. 计算顺序是什么？	小到大、长度递增、拓扑序、DFS memo	区间按长度，DAG 按拓扑
8. 答案在哪里？	$dp[n]$ 、 $\max(dp)$ 、 $dp[n][m]$ 、 $dp[full][last]$ 、 $\text{sum}(dp)$	LIS 是 $\max(dp[i])$ ，LCS 是 $dp[n][m]$

小例子：不相邻选数最大和。

处理对象：一个序列 $a[1..n]$

进度维度：处理到第 i 个数

关键历史：第 i 个数选没选，会影响 $i+1$ 能不能选

状态定义: $dp[i][0/1]$ = 处理完前 i 个数, i 不选/选时的最大和
 base: $dp[0][0]=0, dp[0][1]=-INF$
 转移: $dp[i][0]=\max(dp[i-1][0], dp[i-1][1]); dp[i][1]=dp[i-1][0]+a[i]$
 答案: $\max(dp[n][0], dp[n][1])$

这个例子对应“有后效性就升维”: 只写 $dp[i]$ 不够, 因为不知道第 i 个是否被选; 把 选/不选 放进状态后, 未来就只看当前状态, 不再回头问完整历史。

如果第 3 问答不出来, 先写暴力 DFS, 把所有影响未来的量放进函数参数, 再决定哪些参数能成为 DP 下标。

初始化常用表

目标	初值	合法起点
最大值	-LINF	起点设 0 或题意值
最小值	LINF	起点设 0 或题意值
方案数	0	起点设 1
可行性	false	起点设 true

初始化与答案位置总表

模型	常见初始化	答案位置	易错点
线性 DP	$dp[0]=0$ 或 $dp[1]=a[1]$	$dp[n]$ 或 $\max(dp)$	必须以 i 结尾时答案常是 $\max$
0/1 背包不要求装满	$dp[j]=0$	$dp[w]$	全 0 表示可以不选
恰好装满最大值	$dp[0]=0$, 其他 -LINF	$dp[w]$	不可达不能参与转移
完全背包最少件数	$dp[0]=0$, 其他 LINF	$dp[target]$	不能初始化全 0
LCS/编辑距离	第 0 行/列	$dp[n][m]$	字符访问用 $i-1/j-1$
LIS 朴素	$dp[i]=1$	$\max(dp[i])$	不是 $dp[n]$
网格 DP	起点 $dp[1][1]$	$dp[n][m]$	起点/终点障碍要特判
区间 DP	单点或空区间	$dp[1][n]$	先枚举长度
树形 DP	叶子/子树初值	$dp[root][state]$ 或取 $\max/\min$	父子状态约束别漏
状压 DP	$dp[\text{初始 mask}][start]$	$dp[full][last]$ 聚合	mask 和 last 常常都要

转移顺序心法: pull、push 与依赖图

pull: 当前状态从哪些前驱状态汇总而来。

push: 当前状态向哪些后继状态更新。

考场选择:

- 网格、LCS、编辑距离: pull 通常更自然。
- 背包、状压、DAG: push/pull 都可, 选不容易写错的。
- 记忆化 DFS: 不用手排循环顺序, 但必须保证状态无环或能处理环。

滚动数组判断:

如果 $dp[i]$ 只依赖 $i-1$, 可以滚动。

如果一维压缩后会在同一轮重复使用当前物品, 循环方向必须调整。

0/1 背包倒序: 防止一个物品用多次。

完全背包正序: 允许同一种物品用多次。

复杂度预算卡

模型	状态数	每状态转移	常见复杂度
编辑距离/LCS	$n*m$	$O(1)$	$O(nm)$
P1874 快速求和	$len*target$	枚举下一段 $O(len)$	$O(len^2*target)$
0/1 背包	$n*W$	$O(1)$	$O(nW)$
区间 DP	n^2	枚举断点 $O(n)$	$O(n^3)$
状压 TSP	2^n*n	枚举下一点 $O(n)$	$O(2^n*n^2)$
树上背包	子树容量状态	合并容量	常见 $O(nK^2)$ 或优化到 $O(nK)$

内存估算:

int 数组: 状态数 * 4 字节
long long 数组: 状态数 * 8 字节
1e7 个 int 约 40MB
1e7 个 long long 约 80MB

如果估算爆了:

- 先看能否滚动数组。
- 再看能否换维度, 例如容量太大但价值和小。
- 再看是否只访问少量状态, 用 map/unordered_map 记忆化拿部分分。

最小表推骨架

```
vector<ll> dp(n + 1, -LINF);  
dp[0] = 0;  
  
for (int i = 1; i <= n; i++) {  
    // 在这里按题意写 dp[i] 的转移  
}  
  
cout << dp[n] << '\n';
```

暴力/部分分替代

- $n \leq 20$: 直接 DFS 枚举选择。
- 状态参数明确: 加 memo, 通常能从小数据升到中档。
- 只会部分模型: 先写最接近的记忆化版本, 不强行表推。

升级方向:

暴力 DFS

- > 参数完整后加 memo
- > 状态范围清楚后改数组 memo
- > 依赖顺序清楚后改表推 DP
- > 转移太慢时加前缀和 / 树状数组 / 线段树 / 单调队列

常见坑:

- dp 初值与目标不匹配: 最大值题不能默认 0, 可能全负。
- 答案位置错: 有些题是 $\max(dp[i])$, 不是 $dp[n]$ 。
- 循环顺序错: 0/1 背包容量要倒序, 完全背包容量要正序。
- 记忆化状态漏参数: 例如漏 last、mask、tight。
- 多测没有清空 dp/memo/vis。
- 同样下标未来不唯一, 说明状态有后效性, 要去 DP-03B/DP-26 升维。

DP 调试口令

样例不过, 按顺序查:

1. 状态句子是否唯一?
2. 初值是否和最大/最小/计数/可行性匹配?
3. 不可达状态有没有参与转移?
4. 循环顺序是否保证依赖已算好?
5. 一维压缩方向是否正确?
6. 答案位置是 $dp[n]$ 、max 还是求和?
7. 下标是否 0/1-index 混乱?
8. 取模和 long long 是否处理?
9. 多组数据是否清空?

最小测试样例:

n=1
资源 =0
所有选择都非法
所有选择都合法
答案可能为负
存在重复状态

DP-01: DP 路由表

模块编号: DP-01

模块名称: DP 路由表

标签: DP、模型识别、题面触发词、复杂度

一句话用途: 把题面信号直接路由到可复用的 DP 模型卡片。

模板代码: 本模块是 DP 路由表, 不放完整代码; 具体可抄模板按路由结果去 DP-04 到 DP-27。

题面触发词:

- “每个物品选或不选”
- “容量/预算/重量/费用”
- “两个字符串/两个序列”
- “区间合并/删除/两端取”
- “树上选点/覆盖/染色”
- “集合访问/所有点/钥匙状态”
- “上界很大, 问 $1..N$ 中满足条件的数”
- “字符串切成若干数字段, 段和/代价达到目标”

什么时候用:

- 读题 1 分钟内无法直接写转移时, 先查表。
- 数据范围暗示不能纯暴力, 但题面有典型结构。

不要什么时候用:

- 题目只是单次求和、排序、最短路、并查集连通性。
- 题面要求在线修改与查询, 优先看数据结构卷。

复杂度:

- 本表只负责路由; 真正复杂度见对应模型。

数据范围参考:

- $n \leq 20$ 且集合: 状压 DP。
- $n \leq 300$ 且区间: 区间 DP。
- $n, m \leq 5000$ 且两个序列: LCS/编辑距离。
- $W \leq 1e5$ 且容量: 背包。
- $n \leq 2e5$: 考虑优化 DP 或非 DP 模型。

依赖的标准容器:

- vector
- `map<tuple<...>, ll>`
- Graph

输入如何整理:

- 先标出对象: 序列、物品、网格、区间、树、DAG、集合、数字。
- 再标出目标: 最大、最小、方案数、可行性。
- 最后标出约束: 容量、次数、相邻关系、顺序限制。

接口:

题面触发词 + 数据范围 -> 模型编号 -> 状态句式 -> 三版本路径

输出能力:

- 模型候选。
- 优先使用的状态句式。
- 可先交的部分分版本。

下游可接:

- DP-02 状态句式库
- DP-03 升级图
- DP-04 到 DP-27 模型卡片

可拼接模块:

- BRUTE-07/08/09 记忆化搜索

- DS PrefixSum/树状数组/SegmentTree
- GRAPH Topo/Tree/Floyd

一眼路由表

看到题面信号	优先模型	状态句式入口	注意
前 i 个、最大收益、最小代价 每个元素选或不选 容量、重量、预算、价值	DP-04 线性 DP DP-05 选/不选 DP DP-06/07/08/24 背包	dp[i] dfs(i, state) 或 dp[i][j] dp[j]	答案可能是 max(dp) 小数据先 DFS 看能否重复选、是否分组/多重/多维
每个物品最多一次 每种物品无限个 每组最多选一个 每种物品有限个/多重/二维/价值维度	DP-06 0/1 背包 DP-07 完全背包 DP-08 分组背包 DP-24 背包变体	dp[j] dp[j] dp[j] + ndp dp[j] / dp[value]	容量倒序 容量正序 组内不要相互影响 先判循环方向和初始化
两个序列，公共子序列 两个序列，公共，且要求上升/递增 插入、删除、替换	DP-09 LCS / DP-23 变体 DP-23 LCIS DP-10 编辑距离 / DP-22 建模例题	dp[i][j] dp[j] dp[i][j]	子序列和子串别混 不是先 LCS 再 LIS，扫描第二个序列维护 best 初始化第一行/列
最长递增/不下降子序列	DP-11 LIS / DP-23 变体	dp[i] 或 d[len]	lower_bound/upper_bound 区分
网格只能右/下走 同一位置但历史不同，后续合法选择不同	DP-12 网格 DP DP-03B/DP-26 状态升维	dp[i][j] dp[位置][关键历史]	障碍格初始化 有后效性就把关键历史进状态
区间合并、括号、回文、两端取 树上选择、覆盖、染色 树上选 K 个、子树容量合并 每个点都可能作为根/中心 依赖关系、有向无环、路径计数 n <= 20, 集合、访问所有点 1..N、数位限制、上界很大 dp[i]=min/max dp[j]+cost 且查询范围	DP-13 区间 DP DP-14 树形 DP DP-14 + DP-19 树上背包 TREE-02 换根 DP DP-15 DAG DP DP-16 状压 DP DP-17 数位 DP DP-18 DP+ 数据结构优化	dp[l][r] dp[u][state] dp[u][k] down/up 或二次 DFS dp[u] dp[mask][last] dfs(pos,tight,leading,state) dp[i] + 查询结构	按长度枚举 后序 DFS 子树合并时容量倒序 第一遍子树，第二遍换根 先拓扑排序 内存 $2^n * n$ tight 状态通常不缓存 先写 $O(n^2)$
朴素 DP 会写但超时，出现 SOS/斜率/分治/Knuth/轮廓线	DP-27 高阶 DP 索引	先写朴素式	只作为索引，不要硬套
字符串切分成数字段，凑目标和/最小切分代价	DP-21 P1874 建模例题	dp[i][sum]	这不是数位 DP，是前缀切分 DP
DP 方程想不出，但 DFS 能写	DP-25 DFS+ 记忆化例题	dfs(可变参数)	函数参数就是状态

背包细分表

题面	模型	容量循环
每个物品只能选一次	0/1 背包	for j=W..w[i]
每个物品可以选无限次	完全背包	for j=w[i]..W
物品分成若干组，每组选 0 或 1 个	分组背包	每组用 ndp 或组内倒序小心写
每种物品最多 cnt[i] 个	多重背包	DP-24: 二进制拆分成 0/1
两个容量/费用限制	二维背包	DP-24: 两个容量都倒序
容量很大但价值和小	价值维度背包	DP-24: dp[value]= 最小重量
问能否凑出容量	可行性背包	bool dp[j]
问方案数	计数背包	dp[j] += dp[j-w]
问至少达到容量	至少装满背包	DP-24: 容量封顶或扩容
要输出选了哪些物品	背包路径恢复	初学优先二维 take

不是 DP 的常见误判

题面	更可能是
最少操作次数，每一步代价相同	BFS
边权非负最短路	Dijkstra
连通性、合并集合	DSU
区间修改查询	树状数组/SegmentTree
“每次选当前最小/最大”且有明确局部规则	GREEDY-00/01 贪心 + 堆
只是最大/最小目标，但选择影响容量/历史	DP 或记忆化，先看 GREEDY-01 判别卡

暴力/部分替代：

- 路由不确定时，先写 DFS，把状态参数写全。
- 如果 DFS 有重复状态，加 `map<tuple<...>, ll>`。
- 能跑小数据后再翻对应模型改表推。

升级方向：

路由表候选 1 个 -> 直接套模型

路由表候选多个 -> 对照数据范围和“是否有容量/区间/集合”

完全匹配不上 -> DP-03 从 DFS 造模型

常见坑：

- 只看关键词不看数据范围：比如 $n \leq 20$ 的“选/不选”可能是状压或 DFS。
- “最长路径”在一般有环图不是普通 DP，DAG 才能拓扑 DP。
- “区间查询”不等于区间 DP，可能只是数据结构。

最小测试样例：

拿题面摘要，不写代码，只做：

1. 标出触发词
2. 标出数据范围
3. 选 1 个主模型
4. 写一句状态句式

DP-02：状态句式库

模块编号：DP-02

模块名称：状态句式库

标签：DP、状态设计、句式模板

一句话用途：不会设状态时，直接套“处理到哪里 + 记住什么 + dp 值含义”的固定句式。

模板代码：本模块是状态定义句式库，不放完整代码；定好状态后去对应模型模块抄循环或记忆化模板。

题面触发词：

- “前 i 个”
- “剩余容量/已用容量”
- “最后一个是谁”
- “区间 $[l, r]$ ”
- “子树”
- “集合 mask”
- “数字位 pos”

什么时候用：

- 已经选出 DP 模型，但不知道 dp 下标怎么解释。
- 写转移前，需要确认状态是否完整。

不要什么时候用：

- 状态含义无法排除路径历史影响时，不要硬套；先回到 DFS 参数。
- 有环依赖时，不要把句式当成拓扑顺序。

复杂度：

- 状态句式不决定复杂度；复杂度由状态数量和转移数量决定。

数据范围参考：

- 一维状态通常支持更大 n 。
- 二维 $n*m$ 要看 $n, m \leq 5000$ 左右是否能接受内存。
- 2^n 状态通常要求 $n \leq 20$ 。

依赖的标准容器：

- `vector`
- `vector<vector<ll>>`
- `map<tuple<...>, ll>`

输入如何整理：

- 把题面中的对象改名为统一变量： $i, j, l/r, u, mask, pos$ 。
- 把目标改成四类之一：最大值、最小值、方案数、可行性。

接口：

$dp[\text{下标}]$ 表示：处理范围 + 附加记忆 + 值的类型。

输出能力：

- 直接可写到题解或代码注释里的状态定义。
- 帮助检查是否漏掉必要参数。

下游可接：

- 全部 DP 模型。
- DP-03B/DP-26：如果状态句式写完后，同样下标下未来不唯一，需要状态增维/升维。

可拼接模块：

- BRUTE 记忆化状态卡片。

句式 1：线性序列

$dp[i]$ 表示：处理完前 i 个元素时，_____ 的最大值/最小值/方案数/是否可行。

常见填空：

- 以第 i 个结尾的最大收益。
- 前 i 个中的最优答案。
- 到达位置 i 的最小代价。

句式 2：选/不选

$dp[i][j]$ 表示：处理完前 i 个元素，并且资源/数量/状态为 j 时，_____。

如果表推顺序不清，先写：

$dfs(i, j)$ 表示：从第 i 个元素开始，当前资源/数量/状态为 j 时，后续能得到的答案。

句式 3：背包

$dp[j]$ 表示：当前已处理若干物品，容量/花费为 j 时的最大价值/方案数/是否可行。

初始化句：

$dp[0] = 0/1/true$ ：容量为 0 时什么都不选是合法起点。

其他为 $-LINF/0/false$ ：表示暂时不可达。

句式 4：两个序列

$dp[i][j]$ 表示：只考虑 A 的前 i 个和 B 的前 j 个时，_____。

用于：

- LCS：最长公共子序列长度。
- 编辑距离：把前 i 个变成前 j 个的最小操作数。

句式 5: 网格

$dp[i][j]$ 表示: 走到格子 (i, j) 时, ____。

常见填空:

- 路径数量。
- 最小路径和。
- 最大收益。

句式 6: 区间

$dp[l][r]$ 表示: 只考虑闭区间 $[l, r]$ 时, ____ 的最优值/方案数。

遍历句:

先枚举区间长度 len , 再枚举左端点 l , 右端点 $r = l + len - 1$ 。

句式 7: 树形

$dp[u][state]$ 表示: 只考虑 u 的子树, 并且 u 自己处于 $state$ 状态时, ____。

常见 $state$:

- 0/1: 不选/选 u 。
- 0/1/2: 被父亲覆盖、被儿子覆盖、未覆盖。
- 颜色编号。

句式 8: DAG

$dp[u]$ 表示: 到达 u 的最优值/方案数。

或:

$dp[u]$ 表示: 从 u 出发能得到的最优值/方案数。

选哪一个取决于转移方向是否顺手。

句式 9: 状压集合

$dp[mask][last]$ 表示: 已经选择/访问的集合是 $mask$, 并且最后停在 $last$ 时, ____。

如果不需要最后位置:

$dp[mask]$ 表示: 已经选择集合 $mask$ 时, ____。

句式 10: 数位

$dfs(pos, tight, leading, state)$ 表示: 当前处理到第 pos 位, 是否贴上界 $tight$, 是否仍是前导零 $leading$, 并且附加状态为 $state$ 时, 后

常见附加状态:

- 各位和。
- 上一位数字。
- 是否出现某数字。
- 余数。

状态完整性检查

给定这些下标后, 未来答案是否唯一?

如果同样下标但因为“上一位/最后位置/已选集合/剩余次数”不同导致答案不同, 就必须把它加入状态。

这就是“无后效性”检查:

同样状态 $\rightarrow$ 后续答案唯一: 状态可以用。

同样状态 $\rightarrow$ 后续答案不唯一: 状态有后效性, 去 DP-03B/DP-26 升维。

暴力/部分分替代:

- 状态句式写不顺时, 先把句式改成 $dfs(\dots)$ 表示....
- DFS 参数一旦完整, 直接加 memo。

升级方向:

- `dfs(i, rest) -> memo[i][rest] -> dp[i][rest]` 或滚动 `dp[rest]`。
- `dfs(l, r) -> dp[l][r]`, 按长度表推。
- `dfs(u, state) ->` 树形后序 DP。
- `dfs(i, last/used/phase) ->` DP-03B/DP-26 状态升维。

常见坑:

- `dp[i]` 同时想表达 “前 i 个最优” 和 “必须选 i ”, 转移会混乱。
- 方案数和最优值初值不同, 不能混用。
- 字符串 0-index 和 DP 的 i/j 前缀长度容易差 1。

最小测试样例:

任选一个模型, 先只写状态句式, 不写代码。

检查三个问题:

处理到哪里?

还记住什么?

`dp` 值代表最大、最小、方案数, 还是可行性?

DP-03: DFS -> 记忆化 -> 表推升级图

模块编号: DP-03

模块名称: DFS -> 记忆化 -> 表推升级图

标签: DP、暴力、记忆化、表推、升级路径

一句话用途: 模型匹配不上时, 先用 DFS 表达选择, 再把 DFS 参数升级成 memo 和 DP 表。

题面触发词:

- “每一步有多个选择”
- “同一局面会重复出现”
- “表推顺序想不清”
- “想先拿部分分”

什么时候用:

- 题面不像标准背包/LCS/区间, 但能写出递归枚举。
- 想从可运行的小数据暴力开始, 逐步升级。

不要什么时候用:

- 状态图存在环且没有拓扑顺序或最短路性质。
- 递归深度可能到 $2e5$ 且无法改迭代。
- DFS 参数没有包含完整历史信息。

复杂度:

- 暴力: 选择数的指数级。
- 记忆化: 不同状态数 * 每状态选择数。
- 表推: 状态表大小 * 转移数。

数据范围参考:

- $n \leq 20$: 暴力或状压。
- 状态数 $\leq 1e7$ 且转移少: 数组 memo/表推。
- 状态复杂且稀疏: `map<tuple<...>, ll>` 先保命。

依赖的标准容器:

- `map<tuple<...>, ll>`
- `vector<vector<ll>>`
- `vector<vector<int>> vis`

输入如何整理:

- 先把每一步的选择列出来。
- 再把 “未来答案需要知道的东西” 列为 DFS 参数。

接口:

```
11 dfs(State s);
```

输出能力:

- 暴力小数据。
- 记忆化中档分。
- 表推满分或接近满分。

下游可接:

- 全部 DP 模型。

可拼接模块:

- BRUTE-07 记忆化总论
- BRUTE-09 `map<tuple, ...>` 记忆化
- DP-18 优化 DP

升级图

题面选择

- > 写 `dfs`(当前进度, 资源, 必须记住的历史)
- > 检查参数是否完整
- > 加 `memo`: 同参数直接返回
- > 把 `dfs` 参数变成 `dp` 下标
- > 把 `dfs` 终止条件变成初始化
- > 把 `dfs` 枚举选择变成转移
- > 找到依赖顺序后表推

什么时候可以直接加 memo

记忆化的判断口令:

同样的 `dfs` 参数 -> 未来可选集合完全一样 -> 最优答案也完全一样。

如果同样的参数下, 未来还会被“没放进参数里的历史”影响, 就不能直接 `memo`, 必须升维。常见漏掉的历史:

`last`: 上一次选了谁、上一步方向、上一个颜色。

`used/mask`: 哪些点/物品已经用过。

`path property`: 当前连续次数、是否已经触发过某个限制。

反例:

`dfs(i)` 表示走到第 `i` 步的最好结果。

题目限制“不能连续向下走两步”。

如果不把 `last_direction` 放进参数, `dfs(i)` 不知道上一步是不是向下, 后续选择集合不同, `memo` 会错。

正确状态应类似 `dfs(i, last_direction)`。

通用暴力 DFS 版本

```
11 dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    11 ans = -LINF;
    ans = max(ans, dfs(i + 1, rest)); // 不选
    ans = max(ans, val[i] + dfs(i + 1, rest - cost[i])); // 选
    return ans;
}
```

通用记忆化版本

```
vector<vector<ll>> memo;
```

```
vector<vector<int>> vis;
```

```
11 dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;
    if (vis[i][rest]) return memo[i][rest];
```

```

vis[i][rest] = 1;

ll ans = -LINF;
ans = max(ans, dfs(i + 1, rest));
ans = max(ans, val[i] + dfs(i + 1, rest - cost[i]));
return memo[i][rest] = ans;
}

```

通用表推版本

```

vector<vector<ll>> dp(n + 2, vector<ll>(W + 1, -LINF));
for (int rest = 0; rest <= W; rest++) dp[n + 1][rest] = 0;

for (int i = n; i >= 1; i--) {
    for (int rest = 0; rest <= W; rest++) {
        dp[i][rest] = max(dp[i][rest], dp[i + 1][rest]);
        if (rest >= cost[i]) {
            dp[i][rest] = max(dp[i][rest], val[i] + dp[i + 1][rest - cost[i]]);
        }
    }
}

cout << dp[1][W] << '\n';

```

从 DFS 到表推的对照表

DFS 元素	表推 DP 元素
dfs 参数	dp 下标
非法状态	不转移或设 LINF/-LINF
结束状态	初始化
递归选择	状态转移
返回值	dp 值
初始调用	答案位置

map<tuple> 救场版本

```

map<tuple<int,int,int>, ll> memo;

ll dfs(int i, int rest, int last) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    auto key = make_tuple(i, rest, last);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 按题意枚举选择
    return memo[key] = ans;
}

```

什么时候不改表推

状态范围不清楚。
 依赖顺序会绕回来。
 只需要中档分，memo 已经过样例。
 表推写法比 memo 更容易错。

常见坑：

- 记忆化先查 memo 再判非法，可能数组越界。
- vis=1 代表“算完”，不能用于有环状态的“正在算”。
- 表推初始化忘记对应 DFS 的结束状态。
- 最大值题非法返回 0，导致选非法方案。

- 改成滚动数组后循环方向错。

暴力/部分分替代：

- 用 `map<tuple>` 版本提交，哪怕慢，也比空着强。
- 大数据不会时，加简单特判：空集、全相同、容量为 0、 $n=1$ 。

升级方向：

- 数组 memo 替换 `map<tuple>`。
- 二维表推替换递归。
- 滚动数组省空间。
- 用前缀和/树状数组/线段树/单调队列优化转移。

最小测试样例：

```
3 5
cost: 2 3 4
val : 3 4 5
答案：7
```

DP-03B：状态增维/升维路由

模块编号：DP-03B

模块名称：状态增维/升维路由

标签：DP、状态设计、增维、升维、无后效性、有后效性、反例、记忆化

一句话用途：看到题面里有“未来会受某个历史信息影响”的信号，就把这个信息加进状态，让新状态重新满足无后效性。

题面触发词：

- “相邻不能相同” “上一个选择影响当前”
- “已经访问/已经拥有/已经使用过”
- “恰好 k 个” “还剩几次” “容量/预算”
- “不超过 N” “前导零” “数位限制”
- “颜色/奇偶/余数/父亲/阶段”

什么时候用：

- 已经会写 `dfs(...)`，但样例或手推发现同一个位置后续答案不唯一。
- 写 memo 前，需要检查 DFS 参数是否完整。
- 表推 DP 状态太少，转移里偷偷用了全局变量、路径数组或上一轮选择。

不要什么时候用：

- 这个信息只影响当前转移，不影响以后，可以不用放进状态。
- 这个信息能由现有状态唯一推出，不要重复增维。
- 状态数乘起来爆炸时，先判断能否换模型、压缩状态或只保留必要摘要。

复杂度：

状态数 = 每个维度取值数量相乘

总复杂度 = 状态数 * 每个状态的选择数

数据范围信号：

- 多加一维 last：通常乘 值域大小。
- 多加一维 cnt/rest/mod：通常乘 $K/W/M$ 。
- 多加一维 mask：通常乘 2^n ，要求 n 小。
- 多加 tight/leading：通常只乘 $2 * 2$ ，数位 DP 常用。

依赖的标准容器：

- vector
- array
- `map<tuple<...>, ll>`：状态维度复杂时先保命。

输入如何整理：

- 把题面限制逐条画圈。

- 每条限制问一句：未来选择还需要知道它吗？
- 需要就变成 DFS 参数或 DP 下标。

接口：

题面信号 -> 必须记住的信息 -> 状态维度 -> 漏掉会错的反例

输出能力：

- 帮你判断 $dp[i]$ 是否该升级成 $dp[i][last]$ 、 $dp[i][cnt]$ 、 $dp[mask][last]$ 等。
- 帮你发现 memo 漏参数导致的 WA。

下游可接：

- DP-02 状态句式库
- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-16 状压 DP
- DP-17 数位 DP
- DP-20 计数/可行性 DP
- DP-26 有后效性例题：用“错误状态 -> 反例 -> 升维 -> 完整代码”练习。

可拼接模块：

- $\text{map}<\text{tuple}>$ 记忆化。
- Graph/Tree: parent、color、phase 常见。
- Math: parity/mod 常见。

核心原则

看到什么题面信号，就给状态加什么维度。

更准确地说：

如果两个局面“处理到的位置相同”，但因为某个历史信息不同，导致后续合法选择或答案不同，那么这个历史信息必须进入状态。

换句话说：

状态有后效性 -> 找到影响未来的关键历史 -> 升维/增维 -> 新状态无后效性。

状态句式固定写成：

$dp[\text{处理到哪里}][\text{还必须记住什么}]$ 表示：当前局面下的最优值/方案数/是否可行。

一眼增维表

题面信号	常加维度	状态句式	漏状态反例
相邻、递增、上一步影响当前	last	$dfs(i, last)$	二进制串相邻不能相同，只知道 i 不知道上一位，无法判断下一位能不能放 0
访问过、拥有钥匙、集合状态	mask	$dp[mask][last]$	同在一个房间，拿过钥匙和没拿钥匙能打开的门不同
恰好 k 个、还剩次数、容量	cnt/rest	$dp[i][cnt] / dp[i][rest]$	处理到第 i 个时，已选 0 个和已选 2 个，离“恰好 2 个”的答案不同
数字上界、不超过 N	tight	$dfs(pos, tight, \dots)$	前缀等于上界时本位最多取 $digits[pos]$ ，前缀已经小于上界时本位可取 9
前导零是否还在	leading	$dfs(pos, leading, \dots)$	统计不含数字 0 时，数字 7 前面的补位 0007 不能算出现了 0
染色、类别、模式	color	$dp[u][color]$	树染色时父亲是红色和蓝色，儿子可选颜色不同
奇偶、余数、整除	parity/mod	$dp[i][rem]$	只知道前 i 个，不知道和模 3 的余数，无法判断最后是否整除 3
树/图从哪里来	parent/prev	$dfs(u, parent)$	在无向树上到达 2，父亲是 1 和父亲是 3 时，能继续处理的子树不同

题面信号	常加维度	状态句式	漏状态反例
分阶段、冷却、持有状态	phase	dp[i][phase]	买卖股票只用 dp[i], 分不清今天结束后是手里有股票还是没股票

维度 1: last

题面信号:

相邻不能相同
 上一个数字/字符影响当前
 递增、递减、不下降
 路径最后停在哪里

状态句式:

dfs(i, last) 表示: 已经处理到第 i 位/第 i 步, 上一位或最后位置是 last 时, 后续答案。

漏状态反例:

问长度为 n 的 0/1 串数量, 要求相邻字符不同。

如果只写 dfs(i), 那么处理完第 1 位以后:

前缀 "0" 和前缀 "1" 都是 i=2,
 但前缀 "0" 下一位只能放 1, 前缀 "1" 下一位只能放 0。
 后续合法选择不同, 所以 last 必须进状态。

小模板:

```
11 dfs(int pos, int last) {
    if (pos == n) return 1;
    11 ans = 0;
    for (int x = 0; x <= 1; x++) {
        if (x == last) continue;
        ans += dfs(pos + 1, x);
    }
    return ans;
}
```

维度 2: mask

题面信号:

n <= 20
 已经访问哪些点
 已经拿到哪些钥匙
 哪些任务已经完成
 集合中的元素会影响后续选择

状态句式:

dp[mask][last] 表示: 已经访问/选择集合 mask, 最后停在 last 时的最优值/方案数。
 last 保持 1..n 的业务编号; 第 last 个元素对应 mask 的第 last-1 位。

漏状态反例:

迷宫里同一个格子 (x,y), 如果已经拿到钥匙 A, 可以打开 A 门;
 没拿到钥匙 A, 就不能打开。

只写 dp[x][y] 会把两种局面合并, 导致把不能走的门当成能走, 或把能走的路剪掉。

小模板:

```
for (int mask = 0; mask < (1 << n); mask++) {
    for (int last = 1; last <= n; last++) {
        if (dp[mask][last] == LINF) continue;
        for (int nxt = 1; nxt <= n; nxt++) {
            if (mask >> (nxt - 1) & 1) continue;
            int nmask = mask | (1 << (nxt - 1));
```

```

        dp[nmask][nxt] = min(dp[nmask][nxt], dp[mask][last] + dist[last][nxt]);
    }
}
}

```

维度 3: cnt/rest

题面信号:

恰好/至多/至少选 k 个

还剩几次机会

容量、预算、体力、时间

状态句式:

$dp[i][cnt]$ 表示: 处理完前 i 个元素, 已经选 cnt 个时的答案。

$dp[i][rest]$ 表示: 处理到第 i 个元素, 还剩 $rest$ 资源时的答案。

漏状态反例:

从 5 个数中恰好选 2 个, 使和最大。

处理到第 4 个数时, 如果只写 $dp[i]$,

不知道前面已经选了 0 个、1 个还是 2 个。

这些局面后续能不能继续选、答案是否已经合法都不同。

小模板:

```

vector<vector<ll>> dp(n + 1, vector<ll>(K + 1, -LINF));
dp[0][0] = 0;
for (int i = 1; i <= n; i++) {
    for (int cnt = 0; cnt <= K; cnt++) {
        dp[i][cnt] = max(dp[i][cnt], dp[i - 1][cnt]); // 不选
        if (cnt > 0) {
            if (dp[i - 1][cnt - 1] != -LINF) {
                dp[i][cnt] = max(dp[i][cnt], dp[i - 1][cnt - 1] + a[i]); // 选
            }
        }
    }
}
}

```

维度 4: tight

题面信号:

统计 $0..N$ 或 $1..N$

上界很大

数字逐位构造

状态句式:

$dfs(pos, tight, state)$ 表示: 处理到第 pos 位, 当前前缀是否仍贴着上界 $tight$ 。

漏状态反例:

$N = 523$ 。

处理第 2 位时:

前缀是 5, 下一位最多只能取 2;

前缀是 4, 下一位可以取 $0..9$ 。

如果不记 $tight$, 会把两种上界混在一起。

小模板:

```

int up = tight ? digits[pos] : 9;
for (int d = 0; d <= up; d++) {
    bool ntight = tight && (d == up);
    ans += dfs(pos + 1, ntight, next_state);
}

```

注意: `tight == true` 的状态通常不要缓存; 只缓存 `tight == false` 更稳。

维度 5: leading

题面信号:

数字可以有前导零补齐位数
统计是否出现某个数字
相邻数位限制
不允许数字 0 被前导零误算

状态句式:

`dfs(pos, leading, state)` 表示: 处理到第 `pos` 位, 当前是否仍没有放过非零数字。

漏状态反例:

统计 `1..100` 中 “不含数字 0” 的数。

数字 7 在三位写法里是 007。

前两个 0 是补位, 不应该算作 “出现了数字 0”。

如果不记 `leading`, 就会错误排除 7。

小模板:

```
bool nleading = leading && (d == 0);
int nlast = nleading ? 10 : d; // 10 表示还没有真实上一位
```

维度 6: color

题面信号:

染色
相邻点颜色不同
某点属于某类
父子状态限制

状态句式:

`dp[u][color]` 表示: `u` 的颜色为 `color` 时, `u` 子树的方案数/最优值。

漏状态反例:

一棵树相邻点不能同色, 有 3 种颜色。

如果只写 `dp[u]`, 父亲是什么颜色不知道。

父亲是红色时 `u` 不能选红色; 父亲是蓝色时 `u` 不能选蓝色。

合法选择不同, 所以 `color` 必须进入状态或在转移中枚举。

小模板:

```
for (int c = 0; c < C; c++) {
    dp[u][c] = 1;
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        ll sum = 0;
        for (int vc = 0; vc < C; vc++) {
            if (vc == c) continue;
            sum = (sum + dp[v][vc]) % MOD;
        }
        dp[u][c] = dp[u][c] * sum % MOD;
    }
}
```

维度 7: parity/mod

题面信号:

和为偶数/奇数

能被 K 整除

余数为 r

路径长度模 M

状态句式:

$dp[i][rem]$ 表示: 处理完前 i 个元素, 当前结果模 K 为 rem 时的方案数/是否可行。

漏状态反例:

问选若干数, 使总和能被 3 整除。

处理到同一个 i 时, 当前和模 3 为 0、1、2 的后续完全不同。

只写 $dp[i]$ 无法知道再加一个数后会到哪个余数。

小模板:

```
vector<vector<ll>> dp(n + 1, vector<ll>(K, 0));
dp[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int r = 0; r < K; r++) {
        dp[i][r] = (dp[i][r] + dp[i - 1][r]) % MOD;
        int nr = (r + a[i]) % K;
        dp[i][nr] = (dp[i][nr] + dp[i - 1][r]) % MOD;
    }
}
```

维度 8: parent/prev

题面信号:

无向树 DFS

从哪个方向进入会影响可走边

不能立刻走回上一点

换根或路径状态

状态句式:

$dfs(u, parent)$ 表示: 当前在 u , 父亲是 $parent$, 只处理不走向 $parent$ 的部分。

漏状态反例:

链 1-2-3。

如果写 $dfs(2)$:

当 2 的父亲是 1 时, 2 的子树应该继续处理 3;

当 2 的父亲是 3 时, 2 的子树应该继续处理 1。

只用 u 不能区分这两种方向。

小模板:

```
void dfs(int u, int parent) {
    for (int v : g[u]) {
        if (v == parent) continue;
        dfs(v, u);
    }
}
```

注意: 固定根的树形 DP 中, $parent$ 常作为 DFS 参数, 不一定作为 $dp[u][\dots]$ 的下标。只有同一个 u 可能从不同方向进入并缓存时, 才需要把 $parent/prev$ 放进 memo key。

维度 9: phase

题面信号:

买入/卖出/冷却

已经使用优惠券/尚未使用

正在匹配模式串第几步

第几阶段任务

状态句式:

`dp[i][phase]` 表示: 处理完第 `i` 天/第 `i` 个元素, 当前处于 `phase` 阶段时的最优值。

漏状态反例:

股票买卖题。

处理完第 `i` 天时, 手里有股票和手里没股票的后续选择不同:

有股票可以卖, 没股票可以买。

只写 `dp[i]` 会把两种状态混掉。

小模板:

```
vector<array<ll, 2>> dp(n + 1);
dp[0][0] = 0;          // 0: 没持有
dp[0][1] = -LINF;     // 1: 持有
for (int i = 1; i <= n; i++) {
    dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] + price[i]); // 不动或卖出
    dp[i][1] = max(dp[i - 1][1], dp[i - 1][0] - price[i]); // 不动或买入
}
```

map<tuple> 完整状态救场

状态维度一多, 先用 `map<tuple>` 写对:

```
map<tuple<int,int,int,int>, ll> memo;

ll dfs(int i, int last, int rest, int rem) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return (rem == 0 ? 0 : -LINF);

    auto key = make_tuple(i, last, rest, rem);
    if (memo.count(key)) return memo[key];

    ll ans = -LINF;
    // 按题意枚举选择, 更新 last/rest/rem
    return memo[key] = ans;
}
```

能过样例后, 再看每个维度范围, 改成数组。

状态完整性检查卡

1. 同样的 `dp` 下标, 未来合法选择是否完全一样?
2. 同样的 `dp` 下标, 最终答案类型是否完全一样?
3. 转移中是否偷偷用了 `path`、`sum`、`last`、`used` 这类没进状态的变量?
4. `memo key` 是否包含所有会影响未来的参数?
5. 多加一维后, 状态数是否还能承受?

常见坑:

- 为了省维度, 把 `last/cnt/rem` 写成全局变量, `memo` 立即失效。
- `tight/leading` 没处理清楚, 数位 DP 容易多算 0 或越过上界。
- `mask` 和 `last` 缺一个: 访问集合知道了, 但不知道当前停在哪里。
- `parent` 作为参数传了, 但 `memo` 只按 `u` 缓存。
- 只因为题面出现一个词就盲目增维, 没有检查这个信息是否影响未来。

暴力/部分替代:

- 状态不确定: 先写不带 `memo` 的 DFS, 用参数显式表达所有历史。
- 参数完整后: 直接 `map<tuple> memo`。
- 状态数太大: 只保留题面真正会影响未来的摘要, 例如 `sum % K`, 不要保留完整 `sum`。

升级方向:

- `dfs(i, last) -> memo[i][last] -> dp[i][last]`。
- `dfs(i, cnt, rem) -> 三维数组或滚动数组`。
- `dfs(pos, tight, leading, last, rem) -> 数位 DP`。
- `dfs(mask, last) -> 状压 DP`。

- `dfs(i,j,lastMove/used)` -> DP-26 网格升维例题。

最小测试样例：

题面摘要：长度为 3 的 0/1 串，相邻不能相同，问方案数。

正确状态：`dfs(pos,last)`

答案：2

合法串：010, 101

如果只写 `dfs(pos)`，说明状态漏了 `last`。

DP-21: P1874 快速求和建模例题

模块编号：DP-21

模块名称：从暴力切分到 DP 建模：P1874 快速求和

标签：DP、建模过程、字符串切分、最小加号数、暴力到表推

一句话用途：用一道完整例题演示“为什么这样定义状态”，帮助初学者从暴力搜索自然推到 DP。

题面触发词：

- 数字字符串中插入加号。
- 让表达式结果等于目标数。
- 求最少加号数量。
- 字符串长度不大但所有切法是指数级。
- 前导零不影响数字大小。

什么时候用：

- 题目是在序列/字符串中切分若干段，并让段值的和、代价或方案数满足目标。
- 暴力枚举每个空隙切或不切，复杂度是 $2^{(len-1)}$ 。
- 后续是否能成功只取决于“处理到的位置”和“当前累计值”，不关心前面具体怎么切。

不要什么时候用：

- 段值可以为负，或后续操作能让和变小，此时 `sum > target` 不能直接剪枝。
- 目标值很大到 `dp[len][target]` 开不下，需要改记忆化搜索、map 状态或其他优化。
- 每一段的贡献不只是段值，还依赖前一段形态，此时需要额外状态，例如 `last`。

复杂度：

- 状态数： $O(len * target)$ 。
- 转移：枚举下一段，整体约 $O(len^2 * target)$ 。
- 本题 `len <= 40`、`target <= 1e5`，约 $1.6e8$ 级别简单整数操作，C++ 可接受。
- 空间： $(len + 1) * (target + 1)$ 个 int，约 16MB。

依赖的标准容器：

- `string`：数字串，0-index。
- `static int dp[MAXL][MAXT]`：DP 表，竞赛速写更稳。
- `long long`：临时解析子串值，防止中间乘 10 溢出；本题会在 `target` 以上立即停止。

输入如何整理：

```
string s;
int target;
cin >> s >> target;
int len = (int)s.size();
```

接口：

`solve_p1874(s, target)` -> 最少加号数量，不可达返回 -1

输出能力：

- 输出最少加号数。
- 不可达时输出 -1。

下游可接：

- DP-03 DFS -> 记忆化 -> 表推升级图。
- DP-03B 状态增维路由。
- DP-20 计数/可行性 DP, 如果目标从“最少加号”改成“方案数/是否存在”。
- DP-25 暴力 DFS 到记忆化搜索: 如果考场上先写出 `dfs(pos, sum)`, 直接加 memo 也能得到同一批状态。

可拼接模块:

- CPP-011 string: 子串和字符处理。
- BRUTE-07: 如果先写 DFS, 可加 memo。
- DP-04: 线性前缀 DP 思路。

题意压缩

给定一个数字字符串 `s` 和整数 `target`。可以在相邻字符之间插入若干个 `+`, 把字符串拆成若干个非负整数段。前导零不影响值, 例如 `030` 的值是 `30`。要求所有段相加等于 `target`, 并输出最少需要插入多少个加号; 如果无论如何都不能等于 `target`, 输出 `-1`。

样例:

99999

45

输出:

4

解释: $9+9+9+9+9=45$, 需要 4 个加号。

第一阶段: 先写暴力, 找决策点

长度为 `len` 的字符串有 `len - 1` 个空隙。每个空隙只有两种选择:

切: 插入一个 `+`

不切: 继续把后面的数字拼到当前段里

所以朴素暴力是:

枚举每个空隙是否切开

计算每种表达式的和

取能等于 `target` 的最少加号数

复杂度是 $O(2^{(len-1)})$ 。本题 `len <= 40`, 最大约 2^{39} , 远远超过考场可承受范围。

结论: 暴力可以帮助理解题目, 但必须找重复状态, 用记忆化或 DP 降复杂度。

第二阶段: 找重叠子问题

从左到右处理字符串。假设已经处理完前 `i` 个字符, 并且前面切出来的段之和是 `sum`。

这时后续还没处理的部分只关心两件事:

1. 现在处理到哪个位置 `i`
2. 当前累计和是多少 `sum`

它不关心前面到底是:

`1 + 23`

`12 + 3`

还是其他切法。只要位置和累计和相同, 后续能不能凑到 `target`、还需要多少加号, 都是同一个子问题。

这就是 DP 的信号:

历史具体路径不重要, 只保留会影响未来的摘要信息。

第三阶段: 定义状态并验可行性

自然状态:

`dp[i][sum]` = 把 `s` 的前 `i` 个字符切成若干段, 段和等于 `sum` 时, 最少需要多少个加号

其中:

- `i` 表示已经用掉前 `i` 个字符。
- `sum` 表示这些段的总和。
- `dp` 值表示优化目标: 最少加号数。

可行性检查：

```
i: 0..40
sum: 0..target, target <= 100000
状态数约 41 * 100001
int 表约 16MB
```

空间可行。因为每段都是非负数，如果当前和已经超过 target，后面再加也不会变小，所以 sum 只需要开到 target。

第四阶段：推导转移

看最后一段怎么来。

假设用数学上的 1-index 位置描述，最后一段是 $s[start..end]$ ，它的值是 val。在加上这一段之前，已经处理完前 $start - 1$ 个字符，累计和是 sum。

于是：

```
前面状态: dp[start - 1][sum]
当前段值: val
新状态: dp[end][sum + val]
```

落到 C++ 代码里，字符串本身仍然是 0-index：如果当前已经处理完前 i 个字符，下一段从 $s[i]$ 开始，到 $s[j]$ 结束，新状态就是 $dp[j + 1][sum + val]$ 。

每切出一个新段，段数加一。加号数 = 段数 - 1。

最直观的写法是：

```
dp[0][0] = 0
其他状态 = INF
```

当从位置 i 新开一段 $s[i..j]$ 时：

```
int add = (i == 0 ? 0 : 1);
dp[j + 1][sum + val] = min(dp[j + 1][sum + val], dp[i][sum] + add);
```

解释：

- 如果 $i == 0$ ，这是第一段，前面没有加号，所以加 0。
- 如果 $i > 0$ ，这段前面必须插入一个加号，所以加 1。

这个版本最贴近题目问法：dp 表里直接存“最少加号数”，不需要最后再减一。

第五阶段：细节与陷阱

1. 大数字剪枝：

子串长度可能很长，不能直接 `stoll`。边枚举边计算 $val = val * 10 + digit$ ，一旦 $sum + val > target$ 就停止扩展这一段。

2. 前导零：

题目说前导零不影响大小。逐位计算天然支持：“003”会得到 3。

3. 不可达：

$dp[len][target]$ 仍是 INF 时输出 -1。

4. 加号数量：

不要把段数当答案。 $a+b+c$ 有 3 段但只有 2 个加号。

5. 目标为正：

本题 $target \geq 1$ 。如果遇到目标可为 0 的变体，也能用同一代码；只是注意全零字符串会有很多 0 段。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;
const int MAXL = 45;
const int MAXT = 100000 + 5;
static int dp[MAXL][MAXT];
```



```

int solve_p1874(const string &s, int target) {
    int len = (int)s.size();

    for (int i = 0; i <= len; i++) {
        for (int sum = 0; sum <= target; sum++) {
            dp[i][sum] = INF;
        }
    }
    dp[0][0] = 0;

    for (int i = 0; i < len; i++) {
        for (int sum = 0; sum <= target; sum++) {
            if (dp[i][sum] == INF) continue;

            long long val = 0;
            for (int j = i; j < len; j++) {
                val = val * 10 + (s[j] - '0');
                if (sum + val > target) break;

                int add = (i == 0 ? 0 : 1);
                int ns = (int)(sum + val);
                dp[j + 1][ns] = min(dp[j + 1][ns], dp[i][sum] + add);
            }
        }
    }

    if (dp[len][target] == INF) return -1;
    return dp[len][target];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    int target;
    cin >> s >> target;

    cout << solve_p1874(s, target) << '\n';
    return 0;
}

```

调用示例:

```

cout << solve_p1874("99999", 45) << '\n'; // 4
cout << solve_p1874("12", 3) << '\n'; // 1: 1+2
cout << solve_p1874("303", 6) << '\n'; // 1: 3+03
cout << solve_p1874("0003", 3) << '\n'; // 0: 整段 0003

```

常见坑:

- 用 `stoll(s.substr(...))`: 长子串可能越界或抛异常, 不适合考场稳写。
- 忘记前导零: `303 -> 3+03` 是合法的。
- 每开一段都无脑 +1, 会把第一段前面也算一个加号; 要写 `i == 0 ? 0 : 1`。
- 只初始化 `dp[i][整段值] = 0`, 却忘记从 `dp[0][0]` 或前缀状态统一转移, 容易漏情况。
- `sum + v` 越界: 循环条件写成 `sum + v <= target`。
- `sum + val > target` 后继续扩展: 既浪费时间, 也可能让 `val` 变大后溢出。

暴力/部分分替代:

本题非常适合从暴力 DFS 升级成记忆化搜索。纯 DFS 只适合小数据; 若给 `dfs(pos, sum)` 加 `memo/vis`, 每个位置和累计和只计算一次, 复杂度会降到 $O(\text{len}^2 * \text{target})$ 。完整可抄代码见 DP-25。

```

#include <bits/stdc++.h>
using namespace std;

```

```

const int INF = 1000000000;

string s;
int target_value;
int len;
int best_answer;

void dfs_bruteforce(int pos, long long sum, int plus_count) {
    if (sum > target_value) return;
    if (plus_count >= best_answer) return;

    if (pos == len) {
        if (sum == target_value) best_answer = min(best_answer, plus_count);
        return;
    }

    long long val = 0;
    for (int end = pos; end < len; end++) {
        val = val * 10 + (s[end] - '0');
        if (sum + val > target_value) break;
        int add = (pos == 0 ? 0 : 1);
        dfs_bruteforce(end + 1, sum + val, plus_count + add);
    }
}

int solve_fast_sum_bruteforce(const string& input, int target) {
    s = input;
    target_value = target;
    len = (int)s.size();
    best_answer = INF;
    dfs_bruteforce(0, 0, 0);
    return best_answer == INF ? -1 : best_answer;
}

```

这个 DFS 可以过很小数据。若改成 `memo[pos][sum]` = 当前位置和当前累计和下最少还要多少加号，就是 DP-25 的记忆化版本。

最小测试样例：

输入：
99999

45
输出：
4

输入：
12
12
输出：
0

输入：
12
3
输出：
1

输入：
303
6
输出：
1

输入：
0003

3

输出:

0

输入:

111

100

输出:

-1

建模口令:

先问: 我处理到哪里?

再问: 未来还需要知道什么?

本题答案: 处理到 `start/end`, 未来只需要知道当前 `sum`。

所以状态是 `dp[i][sum]`, 值是最少加号数。

DP-22: 编辑距离建模例题

模块编号: DP-22

模块名称: 从双序列匹配到 DP 建模: 编辑距离

标签: DP、双序列、编辑距离、Levenshtein、插入、删除、替换、建模过程

一句话用途: 用编辑距离完整演示“双序列 DP”的建模过程: 为什么状态是 `dp[i][j]`, 为什么最后一步只有三类操作。

题面触发词:

- 把字符串 A 变成字符串 B。
- 每次可以删除一个字符、插入一个字符、替换一个字符。
- 求最少操作次数。
- 两个字符串长度都在几千以内。
- 字符串相似度、拼写纠错、最小编辑次数。

什么时候用:

- 操作对象是两个字符串或两个序列。
- 当前选择只影响末尾/当前位置, 不需要记住更久的历史。
- 目标是求最小操作次数。
- $|A| * |B|$ 的二维表能开下。

不要什么时候用:

- 只允许删除, 常常可以转成 LCS。
- 操作代价不同, 可以用编辑距离框架, 但转移里的 +1 要改成对应代价。
- 操作带复杂限制, 例如不能连续替换、某些字符不能删, 需要额外状态。
- 字符串长度到 $1e5$ 级别, 普通 $O(nm)$ 编辑距离不可直接做。

复杂度:

- 时间: $O(nm)$ 。
- 空间: $O(nm)$; 若只要答案, 可滚动为 $O(m)$ 。
- 本题长度小于等于 2000, 状态数约 $4e6$, 二维 `int` 表约 16MB, 可行。

依赖的标准容器:

- `string a, b`。
- `static int dp[MAXN][MAXN]`, 题目长度上限明确时比嵌套 `vector` 更适合速写。

输入如何整理:

```
string a, b;
cin >> a >> b;
int n = (int)a.size();
int m = (int)b.size();
```

接口:

`edit_distance(a, b)` -> 把 a 变成 b 的最少编辑次数

输出能力：

- 最少插入/删除/替换次数。
- 可扩展为恢复一条操作路径。

下游可接：

- DP-09 LCS：只允许插入/删除时常与 LCS 相关。
- DP-02 状态句式库：双序列前缀句式。
- DP-20 计数/可行性 DP：若题目改成“是否能在 k 步内完成”。
- DP-25 暴力 DFS 到记忆化搜索：如果先写出 $\text{dfs}(i, j)$ ，加 memo 后就是同一批二维状态。

可拼接模块：

- CPP-011 string。
- 滚动数组优化。
- 路径恢复。

题意压缩

给定两个只含小写字母的字符串 A 和 B。每次可以执行三种操作之一：

删除 A 中的一个字符

在 A 中插入一个字符

把 A 中的一个字符替换成另一个字符

求把 A 变成 B 的最少操作次数。

样例：

sfdqxbw

gfdgw

输出：

4

第一阶段：从暴力尝试开始，找决策点

如果完全不用 DP，面对 A 和 B 的末尾字符，我们大概会递归尝试：

如果末尾相同：两个末尾都跳过

如果末尾不同：

尝试删除 A 的末尾

尝试在 A 末尾插入 B 的末尾

尝试把 A 的末尾替换成 B 的末尾

这个暴力会不断遇到相同的子问题。例如很多操作序列最后都会变成：

把 A 的前 i 个字符变成 B 的前 j 个字符

只要 i 和 j 一样，后续最优答案就一样，不关心之前到底怎么删、插、替换过。

结论：有重叠子问题，可以记忆化；因为依赖顺序很清楚，也可以表推 DP。

第二阶段：找到无后效性

当我们讨论“把 $A[1..i]$ 变成 $B[1..j]$ ”时，未来只关心两个进度：

A 已经考虑到前 i 个字符

B 已经考虑到前 j 个字符

前面具体操作历史不重要。比如你是先删再替，还是先替再插，只要最后剩下的是同一个前缀转换问题，后续最少代价相同。

这就是双序列 DP 的典型信号：

两个对象各有一个进度，下标就是 i 和 j。

第三阶段：定义状态并验可行性

状态定义：

$\text{dp}[i][j]$ = 把 A 的前 i 个字符变成 B 的前 j 个字符，最少需要多少次操作

注意：

- i 和 j 表示前缀长度，不是字符下标。
- 访问字符时用 $a[i - 1]$ 和 $b[j - 1]$ 。
- dp 值是最少操作次数。

可行性检查：

$n, m \leq 2000$

状态数 = $(n + 1) * (m + 1)$ 约 $4e6$

每个状态 $O(1)$ 转移

时间 $O(nm)$ ，空间 $O(nm)$

这个规模对 C++17 很稳。

第四阶段：倒推最后一步

站在状态 $dp[i][j]$ ，观察最后一个字符：

A 的最后字符： $a[i - 1]$

B 的最后字符： $b[j - 1]$

情况一：最后字符相等

如果 $a[i - 1] == b[j - 1]$ ，最后一个字符已经匹配，不需要操作。问题缩小成：

把 A 的前 $i-1$ 个字符变成 B 的前 $j-1$ 个字符

转移：

$dp[i][j] = dp[i - 1][j - 1];$

情况二：最后字符不相等

必须从三种操作中选一个。

1. 删除 A 的最后字符

先把 $A[1..i-1]$ 变成 $B[1..j]$ ，再删除多出来的 $A[i]$ 。

$dp[i - 1][j] + 1$

2. 插入 B 的最后字符

先把 $A[1..i]$ 变成 $B[1..j-1]$ ，再在末尾插入 $B[j]$ 。

$dp[i][j - 1] + 1$

3. 替换最后字符

先把 $A[1..i-1]$ 变成 $B[1..j-1]$ ，再把 $A[i]$ 替换成 $B[j]$ 。

$dp[i - 1][j - 1] + 1$

取最小：

$dp[i][j] = 1 + \min(\{dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]\});$

第五阶段：初始化

必须处理空串。

$dp[0][0] = 0$

$dp[i][0] = i$ ：把 A 的前 i 个字符变成空串，只能删 i 次

$dp[0][j] = j$ ：把空串变成 B 的前 j 个字符，只能插入 j 次

这一步不是形式主义。因为主转移会访问 $i-1$ 和 $j-1$ ，第 0 行和第 0 列就是所有递推的地基。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 2000 + 5;
static int dp[MAXN][MAXN];

int edit_distance(const string &a, const string &b) {
```

```

int n = (int)a.size();
int m = (int)b.size();

for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a[i - 1] == b[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = 1 + min({
                dp[i - 1][j],      // 删除 a[i-1]
                dp[i][j - 1],      // 插入 b[j-1]
                dp[i - 1][j - 1]   // 替换 a[i-1] 为 b[j-1]
            });
        }
    }
}

return dp[n][m];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    cin >> a >> b;

    cout << edit_distance(a, b) << '\n';
    return 0;
}

```

调用示例:

```

cout << edit_distance("sfdqxbw", "gfdgw") << '\n'; // 4
cout << edit_distance("abc", "abc") << '\n';        // 0
cout << edit_distance("", "abc") << '\n';           // 3

```

从 P1874 到编辑距离: DP 五步法

步骤	P1874 快速求和	编辑距离
识别对象	一个字符串, 切成数字段	两个字符串, 互相匹配
进度维度	用到前 i 个字符	A 前 i 个, B 前 j 个
额外约束	当前和 sum	无额外约束
dp 值	最少加号数	最少操作次数
最后一步	最后一段从哪里开始	最后字符相等/删/插/替
初始化	dp[0][0]=0	空串行列
复杂度	$O(\text{len}^2 * \text{target})$	$O(nm)$

通用心法:

1. 画坐标系: 一个进度、两个进度、区间、树、集合?
2. 写状态句: dp[下标] 表示在什么前提下, 最优值是什么。
3. 倒问最后一步: 最后切在哪里? 最后字符怎么处理? 最后选了谁?
4. 处理空状态: 0 个字符、空区间、空集合、容量 0。
5. 估状态数 * 转移数, 确认能跑。

常见坑:

- dp[i][j] 的 i/j 是前缀长度, 访问字符要用 i-1/j-1。
- 第一行/第一列不初始化, 样例可能过, 大数据会错。
- 字符相等时还加 1, 会把无需操作的匹配算错。

- 插入和删除的解释容易混，但公式要记住：删除看 $dp[i-1][j]$ ，插入看 $dp[i][j-1]$ 。
- $n, m=2000$ 时二维表可开；如果更大，需要滚动数组或其他算法。

暴力/部分分替代：

如果没能马上写出二维表，也可以先顺着“末尾/当前位置怎么处理”写 DFS。纯 DFS 是指数级；一旦给 $dfs(i, j)$ 加 $memo/vis$ ，状态数立刻变成 $n*m$ ，本题规模下通常可以直接满分。完整可抄记忆化代码见 DP-25。

```
#include <bits/stdc++.h>
using namespace std;

string a, b;
int n, m;

int dfs_edit_bruteforce(int i, int j) {
    if (i == n) return m - j;
    if (j == m) return n - i;

    if (a[i] == b[j]) return dfs_edit_bruteforce(i + 1, j + 1);

    int del = dfs_edit_bruteforce(i + 1, j);
    int ins = dfs_edit_bruteforce(i, j + 1);
    int rep = dfs_edit_bruteforce(i + 1, j + 1);
    return 1 + min({del, ins, rep});
}

int solve_edit_distance_bruteforce(const string& x, const string& y) {
    a = x;
    b = y;
    n = (int)a.size();
    m = (int)b.size();
    return dfs_edit_bruteforce(0, 0);
}
```

小数据可以先写这个 DFS。加上 $memo[i][j]$ 后，就变成表推 DP 的同一批状态。

最小测试样例：

输入：

sfdqxbw

gfdgw

输出：

4

输入：

abc

abc

输出：

0

输入：

a

b

输出：

1

输入：

abc

def

输出：

3

DP-25: 两道例题的暴力 DFS 到记忆化搜索

模块编号: DP-25

模块名称: 两道例题的暴力 DFS 到记忆化搜索

标签: DP、记忆化搜索、暴力升级、P1874、编辑距离、自顶向下

一句话用途: 如果考场上没能直接推导出递推 DP, 就先写最自然的暴力 DFS, 再把 DFS 的可变参数变成 memo 下标, 快速拿到部分分甚至满分。

题面触发词:

- “我能递归枚举所有选择, 但复杂度爆炸”
- “同一个位置/同一组参数会被反复算”
- “字符串切分、双序列匹配、选或不选、区间递归”
- “求最大/最小/方案数/是否可行”
- “DP 方程一时想不出来”

什么时候用:

- 暴力 DFS 很容易按题意写出来。
- DFS 的参数个数较少, 且每个参数范围能估算。
- 同样的参数代表同一个子问题, 答案不依赖 “怎么走到这里”。
- 递归深度不大, 或至少不会明显到 $1e5$ 以上。

不要什么时候用:

- 状态有环且没有处理 “正在访问” 的标记, 容易递归死循环。
- DFS 参数漏掉了影响未来的全局变量, 例如 `used[]`、`last`、`path`。
- 状态数量远超内存, 数组 memo 开不下且哈希也会太慢。
- 递归深度极深, 例如链状 $n=5e5$, 可能爆栈。

复杂度:

- 纯暴力: 通常是指数级, 例如 2^n 或 3^n 。
- 记忆化搜索: 约等于 状态数 * 每个状态的转移数。
- P1874 快速求和: 约 $O(\text{len}^2 * \text{target})$ 。
- 编辑距离: $O(nm)$ 。

数据范围信号:

- P1874: $\text{len} \leq 40$, $\text{target} \leq 1e5$, `memo[pos][sum]` 可开。
- 编辑距离: $n, m \leq 2000$, `memo[i][j]` 可开。
- 一般题: 先估 状态总数, 超过 $1e7$ 要谨慎, 超过 $1e8$ 通常危险。

依赖的标准容器:

- `vector<vector<int>>` memo
- `vector<vector<char>>` vis
- `string`
- `tuple/map/unordered_map`: 状态稀疏或参数不好做下标时备用。

输入如何整理:

- 把不变的输入放成全局或传引用: 字符串、数组、目标值。
- DFS 参数只放 “会变化且决定未来” 的量。
- 先写清楚一句话: `dfs(...)` 返回什么。

接口:

暴力 `dfs`(可变参数) -> 加 memo/vis -> 每个状态只算一次

输出能力:

- 最小代价。
- 最大收益。
- 方案数。
- 是否可行。
- 小数据精确解和中数据记忆化部分分。

下游可接:

- DP-03 DFS -> 记忆化 -> 表推升级图
- DP-21 P1874 快速求和

- DP-22 编辑距离
- BRUTE-07/08/09 记忆化搜索实现

可拼接模块：

- vector memo: 参数范围小且整数下标。
- map<tuple<...>, int>: 状态复杂但求稳。
- unordered_map<long long, int>: 状态稀疏且能编码。

1. 考场口令

先写暴力 DFS。

看 DFS 函数参数。

去掉全局不变参数。

剩下的可变参数就是状态。

同样状态答案相同，就加 memo。

要特别问自己一句：

同样的 dfs 参数，未来答案是不是永远一样？

如果答案是“是”，就可以记忆化。

注意：表推 DP 和记忆化 DFS 可能访问同一批“位置 + 约束”状态，但 dp 值的含义不一定完全一样。

题目	表推含义	记忆化含义
P1874 快速求和	dp[i][sum]: 前 i 个字符已经切完且和为 sum 的最少已用加号数	dfs(pos, sum): 已经处理到 pos、当前和为 sum，后面最少还要多少加号
编辑距离	dp[i][j]: A 前 i 个字符变成 B 前 j 个字符的最少操作数	dfs(i, j): 从 A[i..] 变成 B[j..] 的最少后续操作数

考场记法：状态维度相同代表可以互相启发；值的方向要按函数定义重新写清楚。

2. P1874 快速求和：记忆化 DFS 完整代码

定义：

dfs(pos, sum) = 已经处理到 s[pos]，前面数字段总和为 sum 时，从 pos 往后凑到 target 还需要的最少加号数。

注意第一段前面没有加号，所以选择一段 s[pos..end] 后，新增加号数是：

```
int add = (pos == 0 ? 0 : 1);
```

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;

string s;
int target_value;
int len;
vector<vector<int>> memo;
vector<vector<char>> vis;

int dfs_fast_sum(int pos, int sum) {
    if (sum > target_value) return INF;
    if (pos == len) {
        return sum == target_value ? 0 : INF;
    }

    if (vis[pos][sum]) return memo[pos][sum];
    vis[pos][sum] = 1;

    int ans = INF;
    long long val = 0;
```

```

    for (int end = pos; end < len; end++) {
        val = val * 10 + (s[end] - '0');
        if (sum + val > target_value) break;

        int add = (pos == 0 ? 0 : 1);
        int got = dfs_fast_sum(end + 1, (int)(sum + val));
        if (got != INF) {
            ans = min(ans, add + got);
        }
    }

    memo[pos][sum] = ans;
    return ans;
}

int solve_fast_sum_memo(const string& input, int target) {
    s = input;
    target_value = target;
    len = (int)s.size();
    memo.assign(len + 1, vector<int>(target_value + 1, INF));
    vis.assign(len + 1, vector<char>(target_value + 1, 0));

    int ans = dfs_fast_sum(0, 0);
    return ans == INF ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string input;
    int target;
    cin >> input >> target;

    cout << solve_fast_sum_memo(input, target) << '\n';
    return 0;
}

```

为什么复杂度降下来:

纯 DFS 会反复进入相同的 (pos, sum)。

加 memo 后, 每个 (pos, sum) 只算一次。

每个状态枚举后面一段的 end, 最多 len 次。

所以约 $O(len * target * len)$ 。

这道题的递归深度最多 $len \leq 40$, 不用担心爆栈。

3. 编辑距离: 记忆化 DFS 完整代码

定义:

$dfs(i, j)$ = 把 a 从 i 开始的后缀变成 b 从 j 开始的后缀, 最少需要多少次操作。

如果 $a[i] == b[j]$, 当前字符不用动, 直接进入 $dfs(i+1, j+1)$ 。

如果不同, 有三种选择:

删除 $a[i]$: $dfs(i+1, j) + 1$

插入 $b[j]$: $dfs(i, j+1) + 1$

替换 $a[i]$: $dfs(i+1, j+1) + 1$

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

string a, b;
int n, m;

```

```

vector<vector<int>>> memo;
vector<vector<char>>> vis;

int dfs_edit_distance(int i, int j) {
    if (i == n) return m - j;
    if (j == m) return n - i;

    if (vis[i][j]) return memo[i][j];
    vis[i][j] = 1;

    int ans;
    if (a[i] == b[j]) {
        ans = dfs_edit_distance(i + 1, j + 1);
    } else {
        int del_cost = dfs_edit_distance(i + 1, j) + 1;
        int ins_cost = dfs_edit_distance(i, j + 1) + 1;
        int rep_cost = dfs_edit_distance(i + 1, j + 1) + 1;
        ans = min({del_cost, ins_cost, rep_cost});
    }

    memo[i][j] = ans;
    return ans;
}

int solve_edit_distance_memo(const string& x, const string& y) {
    a = x;
    b = y;
    n = (int)a.size();
    m = (int)b.size();
    memo.assign(n + 1, vector<int>(m + 1, 0));
    vis.assign(n + 1, vector<char>(m + 1, 0));
    return dfs_edit_distance(0, 0);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string x, y;
    cin >> x >> y;

    cout << solve_edit_distance_memo(x, y) << '\n';
    return 0;
}

```

为什么复杂度降下来:

纯 DFS 每次最多分出 3 个分支, 复杂度接近指数级。

加 memo 后, 状态只有 (i,j), 总数约 $n*m$ 。

每个状态 $O(1)$ 转移, 所以 $O(nm)$ 。

本题 $n, m \leq 2000$, 递归深度大约 $n+m \leq 4000$, 通常安全。

4. 什么时候记忆化能直接满分

满足这四条, 记忆化搜索经常和表推 DP 一样强:

1. 状态没有环, 或者递归天然走向更小/更后的状态。
2. 状态数能承受。
3. 每个状态转移不太多。
4. 递归深度不爆栈。

P1874 和编辑距离都满足:

题目	状态	递归方向	状态数	结果
P1874	(pos, sum)	pos 变大	len * target	可满分
编辑距离	(i, j)	i/j 变大	n * m	可满分

5. 什么时候只能拿部分分

记忆化不一定总是满分，但仍然很值得写：

- 状态范围大，但实际访问状态少，用 map/unordered_map 可拿中档分。
- 转移很多，状态数乘转移数仍偏大，但比纯暴力好很多。
- 状态漏了一些信息，先补全状态可能会变大，但小数据仍可过。
- 递归深度太深时可能 RE，要准备表推或迭代版。

6. 数组 memo 与 map memo 的选择

优先级：

参数范围小、连续整数 -> vector/数组

状态是 tuple 且范围不清 -> map

状态稀疏且能编码成 long long -> unordered_map

复杂状态求稳模板：

```
#include <bits/stdc++.h>
using namespace std;

map<tuple<int, int, int>, int> memo;

int dfs(int i, int sum, int last) {
    auto key = make_tuple(i, sum, last);
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;

    int ans = 0;
    // 在这里枚举选择，更新 ans。

    memo[key] = ans;
    return ans;
}
```

7. 考场策略总结

第一步：写最直白 DFS，不要一开始硬想 for 循环 DP。

第二步：把函数参数写成一串状态定义。

第三步：删掉不变参数，留下决定未来的参数。

第四步：估算状态数，能开数组就开数组。

第五步：先判非法和终止，确认下标合法后查 memo，在返回前写 memo。

第六步：样例过后交一版，后面再考虑表推、滚动数组或数据结构优化。

关键判断：

如果同一个 dfs 参数再次出现，答案完全一样，就能 memo。

如果答案还依赖 used/path/last/当前方向，这些也必须进状态。

常见坑：

- 用 -1 表示没算过，但合法答案也可能是 -1；稳妥用 vis。
- base case 放在查 memo 后面，导致空状态访问越界；正确顺序通常是先判非法/终止，再查 memo。
- 参数漏信息，例如只写 dfs(pos)，但其实还依赖 sum 或 last。
- sum 可能超过目标，没剪枝导致数组越界。
- 多组数据忘记清空 memo/vis。
- 递归深度很深时爆栈。

暴力/部分分替代：

- 完全不会 DP：先交纯 DFS 小数据版。
- DFS 会重复：加 memo。

- 递归深度危险：把 memo DFS 改成表推。
- 状态太大：删掉无关历史，或只用 map 存实际访问状态。

升级方向：

纯 DFS -> DFS + memo -> 表推 DP -> 滚动数组/数据结构优化

最小测试样例：

P1874：

99999

45

输出：4

编辑距离：

sfdqxbw

gfdgw

输出：4

DP-26：发现有后效性怎么办：升维吸收关键历史

模块编号：DP-26

模块名称：发现有后效性怎么办：升维吸收关键历史

标签：DP、无后效性、有后效性、状态升维、last、used、phase、网格 DP

一句话用途：当你发现 $dp[i]$ 或 $dp[i][j]$ 不够用，因为“前面怎么来的”会影响后面选择时，把那一点关键历史加进状态，让新状态重新满足无后效性。

题面触发词：

- “不能连续两次做某操作”
- “相邻不能相同”
- “上一步方向/颜色/动作影响下一步”
- “最多使用一次机会”
- “是否已经使用过某个技能/优惠/钥匙”
- “当前处于持有/冷却/空闲状态”
- “恰好选 K 个，同时还有相邻限制”

什么时候用：

- 两个局面的位置相同，但因为历史不同，后续合法选择不同。
- 转移时你忍不住想看 path、lastMove、used、previousColor 这类变量。
- 记忆化搜索中，同样的 $dfs(pos)$ 得不到唯一答案，必须改成 $dfs(pos, extra_state)$ 。

不要什么时候用：

- 历史信息能由当前位置唯一推出，不需要重复加维。
- 历史只影响当前得分，不影响未来合法选择，可能直接在转移里处理即可。
- 升维后状态数爆炸，例如 $n * m * 2^k$ 中 k 太大，要考虑换模型或只保留更小摘要。
- 图上可以任意回头且有环时，不一定是 DP，可能要 BFS/Dijkstra/状态搜索。

复杂度：

升维后复杂度 = 原状态数 * 新维度大小 * 每个状态转移数

例如：

- $dp[i][j]$ 加上一维上一步方向 2：状态数变成 $2 * n * m$ 。
- $dp[i]$ 加上是否选择当前元素 2：状态数变成 $2 * n$ 。
- $dp[i][j]$ 加上是否用过一次机会 2：状态数变成 $2 * n * m$ 。
- $dp[i]$ 加上已选数量 K 和当前是否选 2：状态数变成 $2 * n * K$ 。

数据范围信号：

- 新维度只有 2/3/10 种：通常很稳。
- 新维度是容量 W ：看 nW 是否可承受。
- 新维度是集合 mask：要求元素数通常 $n \leq 20$ 。

依赖的标准容器：

- vector
- array<ll, 2>
- array<ll, 3>
- vector<vector<array<ll, 2>>>
- vector<vector<vector<ll>>>

输入如何整理：

- 先写错误状态，例如 $dp[i][j]$ 。
- 找反例：同一个 (i, j) ，不同历史导致下一步能做的选择不同。
- 把这个关键历史压成小整数：last/used/color/phase/taken。

接口：

错误状态 -> 找出影响未来的关键历史 -> 加一维 -> 新状态转移

输出能力：

- 最大收益。
- 最小代价。
- 方案数。
- 是否可行。

下游可接：

- DP-03B 状态增维路由。
- DP-12 网格 DP。
- DP-04/05 线性 DP、选/不选 DP。
- DP-25 暴力 DFS 到记忆化搜索。

可拼接模块：

- GridDP：位置 (i, j) 。
- ChooseOrSkip：选择状态 0/1。
- MemoDFS：先写 $dfs(\dots, last, used)$ 。

1. 核心原则

有后效性 = 同一个旧状态，未来不唯一。

破局方法 = 把影响未来的那一点历史加进状态。

升维不是乱加维度，而是只加“未来还需要知道”的最小历史摘要。

检查句式：

只知道我在 X ，下一步能做什么是否唯一？

如果不唯一，还差哪一点历史？

把那一点历史变成状态标。

2. 例题一：网格最大收益，不能连续向下走两步

题意：

给 $n*m$ 网格，每格有收益。

从 $(1,1)$ 走到 (n,m) ，每步只能向下或向右。

规则：不能连续向下走两步。

求最大收益。

错误状态：

$dp[i][j]$ = 到达 (i, j) 的最大收益

为什么错：

同样到达 (i, j) ，如果上一步是向下，下一步不能继续向下。

如果上一步是向右，下一步可以向下。

只知道 (i, j) 不够，未来不唯一。

正确升维：

$dp[i][j][0]$ = 到达 (i, j) ，且最后一步是向下的最大收益

$dp[i][j][1]$ = 到达 (i, j) ，且最后一步是向右的最大收益

转移：

最后一步向下: 只能从上方来, 且上一个状态最后一步不能也是向下。

```
dp[i][j][0] = dp[i-1][j][1] + a[i][j]
```

最后一步向右: 从左方来, 上一步方向不限。

```
dp[i][j][1] = max(dp[i][j-1][0], dp[i][j-1][1]) + a[i][j]
```

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;
const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static ll a[MAXN][MAXM];
static ll dp[MAXN][MAXM][2];

ll max_grid_no_two_down(int n, int m) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            dp[i][j][0] = dp[i][j][1] = NEG;
        }
    }

    dp[1][1][0] = a[1][1];
    dp[1][1][1] = a[1][1];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (i == 1 && j == 1) continue;

            if (i > 1 && dp[i-1][j][1] != NEG) {
                dp[i][j][0] = max(dp[i][j][0], dp[i-1][j][1] + a[i][j]);
            }
            if (j > 1) {
                ll best_left = max(dp[i][j-1][0], dp[i][j-1][1]);
                if (best_left != NEG) {
                    dp[i][j][1] = max(dp[i][j][1], best_left + a[i][j]);
                }
            }
        }
    }

    ll ans = max(dp[n][m][0], dp[n][m][1]);
    return ans == NEG ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << max_grid_no_two_down(n, m) << '\n';
    return 0;
}
```

最小测试:

```
3 2
1 1
100 1
100 1
```

输出: 103

说明: 不能走下、下、右; 可走下、右、下。

3. 例题二: 爬楼梯, 不能连续跳同样步数

题意:

从第 0 级爬到第 n 级, 每次跳 1 或 2 级。

不能连续两次跳同样步数。

问方案数。

错误状态:

$dp[i]$ = 到达第 i 级的方案数

为什么错:

同样到达第 i 级, 如果上一跳是 1, 下一跳不能跳 1。

如果上一跳是 2, 下一跳不能跳 2。

只知道 i 不够。

正确状态:

$dp[i][last]$ = 到达第 i 级, 上一跳是 last 的方案数

last=0 表示起点还没有上一跳, last=1/2 表示上一跳步数。

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1000000007;

int count_stairs_no_same_jump(int n) {
    if (n == 0) return 1;
    vector<array<int, 3>> dp(n + 1);
    dp[0][0] = 1;

    for (int i = 0; i <= n; i++) {
        for (int last = 0; last <= 2; last++) {
            if (dp[i][last] == 0) continue;
            for (int step = 1; step <= 2; step++) {
                if (step == last) continue;
                if (i + step <= n) {
                    dp[i + step][step] += dp[i][last];
                    if (dp[i + step][step] >= MOD) dp[i + step][step] -= MOD;
                }
            }
        }
    }

    return (dp[n][1] + dp[n][2]) % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
```



```

    cout << count_stairs_no_same_jump(n) << '\n';
    return 0;
}

```

最小测试:

4

输出: 1

说明: 只能 1,2,1。

4. 例题三: 恰好选 K 个数, 且不能选相邻

题意:

给 n 个数, 恰好选 K 个, 不能选相邻位置, 求最大和。

错误状态:

$dp[i][cnt]$ = 处理完前 i 个, 已经选 cnt 个的最大和

为什么还不够:

处理第 i+1 个时, 需要知道第 i 个是否已经选了。

如果第 i 个选了, 第 i+1 个不能选; 如果没选, 就可以选。

正确状态:

$dp[i][cnt][0]$ = 处理完前 i 个, 选了 cnt 个, 且第 i 个没选的最大和

$dp[i][cnt][1]$ = 处理完前 i 个, 选了 cnt 个, 且第 i 个选了的和

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_sum_choose_k_no_adjacent(const vector<ll>& a, int K) {
    int n = (int)a.size() - 1;
    vector<vector<array<ll, 2>>> dp(n + 1, vector<array<ll, 2>>(K + 1, {NEG, NEG}));

    dp[0][0][0] = 0;
    for (int i = 1; i <= n; i++) {
        for (int cnt = 0; cnt <= K; cnt++) {
            dp[i][cnt][0] = max(dp[i - 1][cnt][0], dp[i - 1][cnt][1]);
            if (cnt > 0 && dp[i - 1][cnt - 1][0] != NEG) {
                dp[i][cnt][1] = dp[i - 1][cnt - 1][0] + a[i];
            }
        }
    }

    ll ans = max(dp[n][K][0], dp[n][K][1]);
    return ans == NEG ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, K;
    cin >> n >> K;
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    cout << max_sum_choose_k_no_adjacent(a, K) << '\n';
    return 0;
}

```

最小测试:

```
5 2
2 7 9 3 1
```

输出: 11

说明: 选 2 和 9。

5. 例题四: 网格最大收益, 最多一次收益翻倍

题意:

从 (1,1) 到 (n,m), 只能向右或向下。

可以选择最多一个格子, 把该格收益翻倍。

求最大收益。

有后效性的地方:

同样走到 (i,j), 如果已经用过翻倍机会, 后面就不能再用。

如果还没用过, 后面仍然可以用。

正确状态:

dp[i][j][0] = 到达 (i,j), 还没用过翻倍机会的最大收益

dp[i][j][1] = 到达 (i,j), 已经用过翻倍机会的最大收益

完整 C++17:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_grid_one_double(int n, int m, vector<vector<ll>>& a) {
    vector<vector<array<ll, 2>>> dp(n + 1, vector<array<ll, 2>>(m + 1, {NEG, NEG}));

    dp[1][1][0] = a[1][1];
    dp[1][1][1] = a[1][1] * 2;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (i == 1 && j == 1) continue;

            array<ll, 2> best = {NEG, NEG};
            if (i > 1) {
                best[0] = max(best[0], dp[i - 1][j][0]);
                best[1] = max(best[1], dp[i - 1][j][1]);
            }
            if (j > 1) {
                best[0] = max(best[0], dp[i][j - 1][0]);
                best[1] = max(best[1], dp[i][j - 1][1]);
            }

            if (best[0] != NEG) {
                dp[i][j][0] = max(dp[i][j][0], best[0] + a[i][j]);
                dp[i][j][1] = max(dp[i][j][1], best[0] + 2 * a[i][j]);
            }
            if (best[1] != NEG) {
                dp[i][j][1] = max(dp[i][j][1], best[1] + a[i][j]);
            }
        }
    }

    ll ans = max(dp[n][m][0], dp[n][m][1]);
    return ans == NEG ? -1 : ans;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<ll>> a(n + 1, vector<ll>(m + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }

    cout << max_grid_one_double(n, m, a) << '\n';
    return 0;
}

```

最小测试:

```

2 2
1 10
2 3

```

输出: 24

说明: 路径 1->10->3, 并把 10 翻倍。

6. 例题五: 股票买卖, 冷冻期一天

题意:

给每天股价。每天可以不动、买入、卖出。
卖出后的下一天不能买入, 求最大收益。

错误状态:

$dp[i]$ = 第 i 天结束后的最大收益

为什么错:

同样第 i 天结束, 有可能手里持股、空闲、刚卖出处于冷冻。
这三种状态下一天能做的操作不同。

正确状态:

hold = 手里持股

free = 手里没股且不在冷冻, 可买

cool = 今天刚卖出, 下一天冷冻

完整 C++17:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG = numeric_limits<ll>::min() / 4;

ll max_profit_with_cooldown(const vector<ll>& price) {
    int n = (int)price.size() - 1;
    vector<array<ll, 3>> dp(n + 1, {NEG, NEG, NEG});

    dp[0][0] = NEG; // hold
    dp[0][1] = 0;   // free
    dp[0][2] = NEG; // cool

    for (int i = 1; i <= n; i++) {
        dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] - price[i]);
        dp[i][1] = max(dp[i - 1][1], dp[i - 1][2]);
        dp[i][2] = dp[i - 1][0] + price[i];
    }
}

```

```

    return max(dp[n][1], dp[n][2]);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<ll> price(n + 1);
    for (int i = 1; i <= n; i++) cin >> price[i];

    cout << max_profit_with_cooldown(price) << '\n';
    return 0;
}

```

最小测试:

```

5
1 2 3 0 2
输出: 3

```

7. 升维清单

发现的问题	加什么维度	典型状态
上一步动作影响下一步	lastMove	dp[i][j][lastMove]
相邻不能相同	lastColor/lastValue	dp[i][last]
当前元素选没选影响下一个	take	dp[i][0/1]
机会是否用过	used	dp[i][j][used]
还剩几次机会	rest	dp[i][rest]
处于持有/冷却/空闲	phase	dp[i][phase]
已访问哪些点	mask	dp[mask][last]

8. 从 DFS 角度理解升维

如果你先写 DFS, 升维会更自然:

不能连续向下:

```
dfs(i, j, lastMove)
```

不能连续同样跳法:

```
dfs(pos, lastStep)
```

恰好选 K 且不能相邻:

```
dfs(i, cnt, prevTaken)
```

最多一次翻倍:

```
dfs(i, j, used)
```

股票冷冻期:

```
dfs(day, phase)
```

这时 DFS 的可变参数就是 memo 的 key, 也是表推 DP 的下标。

常见坑:

- 忘记把影响未来的变量放进 memo key, 导致不同历史被错误合并。
- 加了太多无关历史, 状态数爆炸。
- last 的初始值没有设计好; 可以用特殊值, 或像网格例题那样给起点两个虚拟方向。
- 最大值题初始不可达状态要用 NEG, 不要默认 0。
- 允许负收益时, 默认 0 会错误地制造一条不存在的路径。

暴力/部分分替代:

- 先写 dfs(..., last/used/phase)。
- 小数据不加 memo 也能跑。

- 中数据加 memo。
- 状态小且无环后，再改成表推 DP。

升级方向：

错误 `dp[i][j]`

-> 找到导致未来不同的历史 `last/used/phase`

-> `dp[i][j][last]`

-> memo DFS 或表推

-> 如果维度太大，压缩/换模型

最小测试样例：

网格不能连续向下：

```
3 2
1 1
100 1
100 1
输出：103
```

爬楼梯不能连续同样跳法：

```
4
输出：1
```

恰好选 K 且不能相邻：

```
5 2
2 7 9 3 1
输出：11
```

DS-00 数据结构路由与接口总表

模块编号：DS-00

模块名称：数据结构路由与接口总表

标签：[数据结构][路由][拼接]

一句话用途：根据操作类型快速选择 PrefixSum、Difference、树状数组、SegmentTree、SparseTable、MonotonicStack、MonotonicQueue、DSU 或 Compressor。

题面触发词：

- 区间查询。
- 区间修改。
- 单点修改。
- 动态维护。
- 排名、逆序对。
- 合并集合、连通块。
- 滑动窗口最大/最小。
- 连续区间满足和/计数条件。
- 排序数组配对、两数之和、三数之和。
- 最近更大/更小。

什么时候用：

- 题目核心不是复杂推导，而是需要维护某种数据。
- 暴力每次扫描会超时。
- 操作可以归类为查询、修改、合并、统计。

不要什么时候用：

- 数据范围很小，直接暴力更快更稳。
- 题目关键在 DP/图论建模，数据结构只是辅助，此时先完成主模型。
- 动态需求不清楚时，不要一上来写最长的线段树。

复杂度：

- 按模块不同，从 $O(1)$ 、 $O(\log n)$ 到 $O(n \log n)$ 预处理不等。

数据范围参考：

操作规模	暴力风险	优先模块
$n, q \leq 2000$	暴力可能可过	先暴力或前缀和
$n, q \leq 2e5$	每次扫描会 TLE	树状数组 / SegmentTree
值域大但点数少	数组开不下	Compressor + 树状数组/SegmentTree

依赖的标准容器：

- 1-index 数组：上限明确时优先全局静态数组 `a[MXN]`；上限不清楚时才用 `vector<ll> a(n + 1)`。
- 闭区间：`[l, r]`。
- 坐标压缩后编号也从 1 开始。

输入如何整理：

```
int n, q;
cin >> n >> q;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口：

```
build(a) // 本卷主模板默认接 1-index vector<ll>
init(n)
add(pos, val)
setv(pos, val)
range_add(l, r, val)
prefix(pos)
query(l, r)
at(pos)
```

考场常数优先口径：

本卷为了拼接统一，很多接口写成 `build(vector<ll>& a)`。

如果题目上限明确、你想直接用全局静态数组，可以把 `build(a)` 改成 `build(n, a)`，内部仍按 `1..n` 循环。

核心不变：数组和查询全部保持 1-index、闭区间 `[l, r]`。

输出能力：

- 静态区间和、动态区间和、区间最值、滑动窗口最值、连通性、排名统计等。

下游可接：

- DP 优化。
- 扫描线。
- 逆序对。
- 动态排名。
- Kruskal。
- 二分答案。

可拼接模块：

题面操作	模块组合
静态区间和	Array + PrefixSum
静态区间最值	Array + SparseTable
单点加 + 区间和	树状数组
区间加 + 最终还原	Difference
区间加 + 单点查	差分树状数组
区间加 + 区间和	双树状数组 或 LazySegmentTree
动态区间 min/max	SegmentTree
值域很大	Compressor + 树状数组/SegmentTree
逆序对	Compressor + 树状数组
连通性/合并	DSU
最小生成树	Graph.edges + DSU + Kruskal
滑动窗口最值	MonotonicQueue
连续区间和/计数满足条件	TwoPointers / SlidingWindow
排序数组配对	TwoPointers

题面操作	模块组合
最近更大/更小	MonotonicStack

模板代码:

```
// 数据结构选择口诀:
// 不修改区间和 -> PrefixSum
// 单点改区间和 -> 树状数组 (代码类名 BIT)
// 区间改最终一次输出 -> Difference
// 区间改区间查 -> LazySegmentTree / 双树状数组 (代码类名 RangeBIT)
// 静态最值 -> SparseTable
// 动态最值 -> SegmentTree
// 值域大 -> Compressor first
// 合并集合 -> DSU
```

调用示例:

```
// 静态区间和
PrefixSum ps;
ps.build(a);
cout << ps.query(l, r) << "\n";
```

```
// 单点加, 区间和
BIT fw;
fw.build(a);
fw.add(pos, delta);
cout << fw.query(l, r) << "\n";
```

常见坑:

- 树状数组下标不能为 0。
- 坐标范围查询不要直接用 $id(L)$ 和 $id(R)$, 应使用 $lower_id/upper_id$ 。
- 前缀和不支持修改。
- SparseTable 不支持修改, 且只适合 $min/max/gcd$ 这类可重叠查询。
- 区间 $[l, r]$ 必须统一为闭区间。

暴力/部分替代:

- 静态查询可每次循环求和, $O(nq)$, 小数据可过。
- 动态修改可直接维护数组并每次扫描, 先拿小数据。
- 合并集合可 BFS/DFS 查连通, 小数据可过。

升级方向:

- 暴力扫描 -> PrefixSum/树状数组。
- 离散大值域 -> Compressor。
- 树状数组不够表达复杂区间最值 -> SegmentTree。
- 普通 SegmentTree -> LazySegmentTree。

最小测试样例:

```
n=1
单点区间 l=r
整段区间 l=1,r=n
负数数组
多次修改同一点
```

DS-06: 双指针与滑动窗口

模块编号: DS-06

模块名称: 双指针、相向指针、快慢指针与滑动窗口

标签: 双指针、滑动窗口、相向指针、快慢指针、连续区间、排序数组

一句话用途: 把双重循环枚举区间/配对降到线性或 $O(n \log n)$, 常用于连续子数组、排序数组配对、去重和窗口计数。

题面触发词：

- 连续子数组、连续区间、最长/最小子段。
- 正整数数组，区间和满足条件。
- 排序数组，两数之和、三数之和。
- 不重复字符最长子串。
- 去重、原地压缩数组。
- 快慢指针、链表判环。

什么时候用：

- 左右端点都只会单调移动，不会反复回退。
- 数组全为非负数，区间和随右端扩张不减，随左端右移不减。
- 数组已排序或可以先排序，配对关系有单调性。
- 要维护一个连续窗口里的计数、和、种类数。

不要什么时候用：

- 数组有负数时，区间和不再单调，普通滑窗求“和不超过 S”可能错。
- 查询不是连续区间，而是任意子集。
- 每次移动端点后需要复杂区间最值，可能要接单调队列、树状数组或线段树。
- 排序会破坏原下标且题目需要原顺序时，不能直接排序相向指针。

复杂度：

- 同向双指针/滑动窗口：通常 $O(n)$ 。
- 相向双指针：排序后 $O(n \log n)$ ，双指针扫描 $O(n)$ 。
- 三数之和：排序后固定一个数，内层相向指针， $O(n^2)$ 。

依赖的标准容器：

- 普通数组默认 1-index，上限明确时优先静态数组。
- string。
- array<int, 256> 或 vector<int> 维护字符计数。
- 排序配对常接 sort。

输入如何整理：

```
const int MAXN = 200000 + 5;
int n;
cin >> n;
static long long a[MAXN];
for (int i = 1; i <= n; i++) cin >> a[i];
```

字符串窗口：

```
string s;
cin >> s; // 0-index
```

接口：

同向窗口: for r in 1..n expand, while bad shrink l

相向指针: sort(a+1,a+n+1), l=1, r=n, compare sum

快慢指针: slow 记录答案尾部, fast 扫描所有元素

输出能力：

- 最长/最短满足条件的连续区间长度。
- 满足条件的配对数量或一组配对。
- 去重后的长度。
- 字符串最长无重复子串。

下游可接：

- PrefixSum: 有负数时改前缀和 + 哈希/二分。
- MonotonicQueue: 窗口内最大/最小。
- Greedy: 排序后相向配对。
- DP: 把 $O(n^2)$ 枚举前驱优化成窗口。

可拼接模块：

- CPP-003 / CPP-012 排序和二分。
- DS-01 PrefixSum。

- DS-04 MonotonicQueue.
- STR-01 / CPP-011 string.

一眼路由

题面信号	模板	前提
正整数数组，最长和不超过 S	同向滑动窗口	元素非负
正整数数组，最短和至少 S	同向滑动窗口	元素非负
排序数组两数之和	相向指针	有序
三数之和/三元组	固定一个 + 相向指针	先排序
删除有序数组重复项	快慢指针	有序或相邻重复
最长无重复字符串	窗口 + 计数	字符集可计数
固定长度窗口最值	单调队列	查 max/min

模板 1：最长连续子数组，和不超过 S

前提: $a[i] \geq 0$ 。

```
int longest_sum_at_most(const vector<long long> &a, long long S) {
    int n = (int)a.size() - 1;
    int ans = 0;
    int l = 1;
    long long sum = 0;

    for (int r = 1; r <= n; r++) {
        sum += a[r];
        while (l <= r && sum > S) {
            sum -= a[l];
            l++;
        }
        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

调用示例:

```
vector<long long> a = {0, 2, 1, 3, 2};
cout << longest_sum_at_most(a, 4) << '\n'; // 2: 1+3 或 3? 2+1
```

模板 2：最短连续子数组，和至少 S

前提: $a[i] \geq 0$ 。

```
int shortest_sum_at_least(const vector<long long> &a, long long S) {
    int n = (int)a.size() - 1;
    const int INF = 1000000000;
    int ans = INF;
    int l = 1;
    long long sum = 0;

    for (int r = 1; r <= n; r++) {
        sum += a[r];
        while (l <= r && sum >= S) {
            ans = min(ans, r - l + 1);
            sum -= a[l];
            l++;
        }
    }

    return ans == INF ? -1 : ans;
}
```

模板 3：排序数组两数之和是否存在

```
bool two_sum_exists(long long a[], int n, long long target) {
    sort(a + 1, a + n + 1);
    int l = 1;
    int r = n;

    while (l < r) {
        long long sum = a[l] + a[r];
        if (sum == target) return true;
        if (sum < target) l++;
        else r--;
    }
    return false;
}
```

如果要保留原下标：

```
vector<pair<long long, int>> v;
for (int i = 1; i <= n; i++) v.push_back({a[i], i});
sort(v.begin(), v.end());
```

模板 4：有序数组原地去重

输入为 1-index 有序数组 $a[1..n]$ 。

```
int unique_sorted(int a[], int n) {
    if (n == 0) return 0;
    int slow = 1;
    for (int fast = 2; fast <= n; fast++) {
        if (a[fast] != a[slow]) {
            slow++;
            a[slow] = a[fast];
        }
    }
    return slow; // 去重后有效区间为  $a[1..slow]$ 
}
```

STL 版本：

```
sort(a + 1, a + n + 1);
n = unique(a + 1, a + n + 1) - (a + 1); // 新长度
```

模板 5：最长无重复字符串

字符串自然 0-index。

```
int longest_unique_substring(const string &s) {
    vector<int> cnt(256, 0);
    int ans = 0;
    int l = 0;

    for (int r = 0; r < (int)s.size(); r++) {
        unsigned char cr = (unsigned char)s[r];
        cnt[cr]++;

        while (cnt[cr] > 1) {
            unsigned char cl = (unsigned char)s[l];
            cnt[cl]--;
            l++;
        }

        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

模板 6：三数之和为 0，去重计数/列举

```
vector<array<int, 3>> three_sum_zero(int a[], int n) {
    sort(a + 1, a + n + 1);
    vector<array<int, 3>> ans;

    for (int i = 1; i <= n; i++) {
        if (i > 1 && a[i] == a[i - 1]) continue;

        int l = i + 1;
        int r = n;
        while (l < r) {
            long long sum = (long long)a[i] + a[l] + a[r];
            if (sum == 0) {
                ans.push_back({a[i], a[l], a[r]});
                l++;
                r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (sum < 0) {
                l++;
            } else {
                r--;
            }
        }
    }
    return ans;
}
```

常见坑：

- 有负数数组不能直接用“sum 太大就左端右移”的滑动逻辑。
- while 收缩条件写错，会漏掉刚好满足的窗口。
- 相向指针通常要求数组有序；没排序时移动方向没有意义。
- 排序后原下标丢失，需要提前存 {value, id}。
- 字符串窗口用 char 当数组下标时，稳妥写 unsigned char。
- 三数之和去重需要跳过重复的 i/l/r。

暴力/部分分替代：

- 区间问题小数据：双重循环枚举 [l, r]，每次累计和。
- 两数之和小数据：双重循环枚举 i, j。
- 字符串小数据：枚举每个左端，向右用 set 检查重复。

升级方向：

- 双重循环区间和 -> 正数数组滑动窗口。
- 双重循环配对 -> 排序 + 相向指针。
- 固定窗口最值 -> 单调队列。
- 有负数的区间和 -> PrefixSum + 哈希/二分/数据结构。

最小测试样例：

最长和不超过 S：

a = [2, 1, 3, 2], S=4 -> 2

最短和至少 S：

a = [2, 1, 3, 2], S=5 -> 2

两数之和：

[1, 2, 4, 7], target=6 -> true

最长无重复子串：

abca -> 3

GRAPH-00 标准 Graph 与建图场景

模块编号: GRAPH-00

模块名称: 标准 Graph、建图场景、有向/无向图

标签: [图论][标准容器][建图][1-index]

一句话用途: 把题面里的边统一整理成 Graph, 后续 BFS、DFS、Dijkstra、Topo、Kruskal、Floyd、Bellman-Ford、SCC、LCA 都尽量共用它。

题面触发词:

- 有 n 个点、 m 条边。
- 道路、航线、关系、依赖、连接、能到达。
- 树、森林、无向图、有向图、带权图。
- 点编号通常是 $1..n$ 。

什么时候用:

- 普通图论题先用本模块建图。
- 需要从点出发遍历时使用 $G.g[u]$ 。
- 需要处理全部边、排序边、初始化矩阵时使用 $G.edges$ 。
- 权值、距离、答案统一优先用 ll 。

不要什么时候用:

- 最大流、最小费用流不要直接用本 Graph, 它们需要残量边, 见 GRAPH-10。
- 极限内存卡得很死时, 可能要链式前向星。
- 网格 BFS 通常直接用二维数组和方向数组, 除非题目明显要转图。

复杂度:

- 建图: $O(n + m)$ 空间, $O(m)$ 时间。
- 无向边在 $G.g$ 中存两次, 在 $G.edges$ 中只存一次逻辑边。

数据范围参考:

- $n, m \leq 2e5$ 常规可用。
- m 特别大时注意 $\text{vector}<\text{vector}<\text{AdjEdge}>>$ 的常数和内存。

依赖的标准容器:

- Graph。
- 1-index 点编号。
- ll 边权。

输入如何整理:

```
int n, m;
cin >> n >> m;
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 无向带权
}
```

接口:

```
Graph G(n)
G.init(n)
G.add_undirected(u, v, w)
G.add_directed(u, v, w)
G.g[u]      从 u 出发的邻接边
G.edges     每条输入逻辑边
```

输出能力:

- 统一邻接表。
- 统一全边表。
- 记录有向/无向信息, 方便 Floyd、Bellman-Ford 等模块。

下游可接:

- DFS/BFS、无权最短路、Dijkstra、Topo、DAG DP、SCC、LCA。
- Floyd、Bellman-Ford、Kruskal。

可拼接模块:

- GRAPH-01 连通性。
- GRAPH-02/03/04 最短路。
- GRAPH-05 Topo/DAG DP。
- GRAPH-06 DSU/Kruskal。
- GRAPH-08 SCC。
- GRAPH-09 树和 LCA。

模板代码:

```
struct AdjEdge {
    int to;
    ll w;
    int edge_index; // 内部边下标, 只给模板内部跳父边/查原边用
    bool directed;
    int input_id;    // 对外边号, 默认 1-index
};

struct FullEdge {
    int from, to;
    ll w;
    bool directed;
    int input_id;    // 对外边号, 默认 1-index
};

struct Graph {
    int n;
    vector<vector<AdjEdge>> g;
    vector<FullEdge> edges;

    Graph(int n = 0) {
        init(n);
    }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        edges.clear();
    }

    void add_edge_raw(int u, int v, ll w, bool directed) {
        int edge_index = (int)edges.size(); // 内部 0-based, 不输出
        int input_id = edge_index + 1;      // 题面边号统一 1-based
        edges.push_back({u, v, w, directed, input_id});
        g[u].push_back({v, w, edge_index, directed, input_id});
        if (!directed) {
            g[v].push_back({u, w, edge_index, directed, input_id});
        }
    }

    void add_undirected(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, false);
    }

    void add_directed(int u, int v, ll w = 1) {
        add_edge_raw(u, v, w, true);
    }
};
```

调用示例:

```

Graph G(n);
G.add_undirected(1, 2);    // 无向无权
G.add_undirected(2, 3, 5); // 无向带权
G.add_directed(3, 4, 7);   // 有向带权 3 -> 4
G.add_directed(4, 5);      // 有向无权 4 -> 5, 更不容易写错

for (auto e : G.g[2]) {
    int v = e.to;
    ll w = e.w;
    int input_id = e.input_id; // 1-index, 可直接输出题面边号
}

for (auto e : G.edges) {
    int u = e.from;
    int v = e.to;
}

```

常见坑:

- 无向图只写 `add_undirected`, 有向图只写 `add_directed`; 考场不要直接调用 `add_edge_raw`。
- 不要写 `G.add_edge(u, v, true)` 这种四参/布尔接口; 本资料主模板故意不暴露它, 避免把 `true` 当权值。
- Kruskal 只能遍历 `G.edges`, 不要遍历 `G.g`, 否则无向边会重复。
- Topo 和 SCC 通常要求有向边, 建图时统一写 `G.add_directed(u, v, w)`; 不要临时改回布尔参数接口。
- Floyd 初始化时要看 `e.directed`, 无向边要对称赋值。
- 多组数据必须重新 `G.init(n)`。
- 如果题目点编号是 `0..n-1`, 读入时立刻 `u++, v++` 转成内部 `1..n`, 后续不要切换到 `0-index`。
- 点编号默认 `1-index`; 对外边号用 `input_id`, 也是 `1-index`。内部 `edge_index` 不输出, 只给 Lowlink 跳父边等模板内部使用。

图类型与模块路由协议

图类型	建图方式	优先模块	误用风险
无向无权图	<code>add_undirected(u,v)</code>	DFS/BFS、连通块、二分图	Topo/SCC 不适用
无向带权图	<code>add_undirected(u,v,w)</code>	Dijkstra、Kruskal、树 LCA	Kruskal 遍历 <code>edges</code> , 不要遍历 <code>g</code>
有向无权图	<code>add_directed(u,v)</code>	BFS、Topo、SCC	不要临时改回布尔参数接口
有向带权图	<code>add_directed(u,v,w)</code>	Dijkstra、Bellman-Ford、DAG DP	有负边不能 Dijkstra
DAG	全部有向边且无环	Topo、DAG DP	<code>topo.size()!=n</code> 说明有环
树	无向、连通、 <code>m=n-1</code>	TreeDFS、LCA、树形 DP	一般图不能当树
流网络	FlowGraph	Dinic	不用普通 Graph

暴力/部分替代:

- 点很少时可以用邻接矩阵 `dist[n+1][n+1]`。
- 只判断一两次连通时, 可临时用二维 `bool adj` + DFS。

升级方向:

- 普通无权图: 接 BFS。
- 非负权: 接 Dijkstra。
- 小图全源: 接 Floyd。
- 最小生成树: 接 DSU/Kruskal。
- 树路径: 接 DFS distance + LCA。

最小测试样例:

```

n=3
add 1 2 undirected
add 2 3 directed
G.g[1] 有 2
G.g[2] 有 1 和 3
G.g[3] 没有 2
G.edges.size() = 2

```

GRAPH-03 Dijkstra、路径恢复、多源最短路

模块编号: GRAPH-03

模块名称: Dijkstra、路径恢复、多源最短路

标签: [图论][Dijkstra][非负权][路径恢复][多源最短路]

一句话用途: 边权非负时求最短路, 并可记录前驱恢复一条最短路径; 多个起点时把所有源点一起入堆。

题面触发词:

- 边权非负、道路长度、花费、时间。
- 从一个点到其他点的最短距离。
- 从多个仓库/医院/起点出发到最近目标。
- 输出最短路径经过的点。

什么时候用:

- 边权 $w \geq 0$ 。
- n, m 大, 不能 Floyd。
- 单源或多源最短路。
- 需要恢复一条最短路径。

不要什么时候用:

- 有负权边, 不能用 Dijkstra。
- $n \leq 500$ 且要求任意两点最短路, 可用 Floyd。
- 边权全为 1 时 BFS 更简单。
- 要第 k 短路、动态最短路时, 本模板不够。

复杂度:

- $O((n + m) \log n)$ 时间。
- $O(n + m)$ 空间。

数据范围参考:

- $n, m \leq 2e5$ 常规可用。
- 距离可能很大, 用 `ll`。

依赖的标准容器:

- `Graph`。
- 从 `G.g[u]` 遍历出边。
- `priority_queue`。
- 依赖主骨架: `using ll = long long; const ll LINF = 4'000'000'000'000'000LL;`

输入如何整理:

```
Graph G(n);
for (int i = 1; i <= m; i++) {
    int u, v;
    ll w;
    cin >> u >> v >> w;
    G.add_undirected(u, v, w); // 有向边改用 G.add_directed(u, v, w)
}
```

接口:

```
vector<ll> dijkstra(const Graph& G, int s)
ShortestPathResult dijkstra_with_parent(const Graph& G, int s)
ShortestPathResult dijkstra_multi_source(const Graph& G, vector<int> sources)
vector<int> restore_path(pre, s, t)
```

输出能力:

- `dist[v]`: 最短距离, 不可达为 `LINF`。
- `pre[v]`: 恢复路径用的前驱点。
- 多源版本中 `pre[source]=0`。

下游可接:

- 路径恢复。
- 最短路 DAG。

- 二分答案检查。
- 树上路径特判。

可拼接模块：

- Graph + Dijkstra + restore_path。
- Graph + MultiSourceDijkstra。
- Dijkstra + DP。

模板代码：

```
struct ShortestPathResult {
    vector<ll> dist;
    vector<int> pre;
};

ShortestPathResult dijkstra_with_parent(const Graph &G, int s) {
    ShortestPathResult res;
    res.dist.assign(G.n + 1, LINF);
    res.pre.assign(G.n + 1, 0);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    res.dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != res.dist[u]) continue;
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            if (res.dist[u] + w < res.dist[v]) {
                res.dist[v] = res.dist[u] + w;
                res.pre[v] = u;
                pq.push({res.dist[v], v});
            }
        }
    }
    return res;
}

vector<ll> dijkstra(const Graph &G, int s) {
    return dijkstra_with_parent(G, s).dist;
}

ShortestPathResult dijkstra_multi_source(const Graph &G, const vector<int> &sources) {
    ShortestPathResult res;
    res.dist.assign(G.n + 1, LINF);
    res.pre.assign(G.n + 1, 0);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    for (int s : sources) {
        if (s < 1 || s > G.n) continue;
        if (res.dist[s] == 0) continue;
        res.dist[s] = 0;
        pq.push({0, s});
    }
    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();
        if (du != res.dist[u]) continue;
        for (auto e : G.g[u]) {
            int v = e.to;
            ll w = e.w;
            if (res.dist[u] + w < res.dist[v]) {
                res.dist[v] = res.dist[u] + w;
                res.pre[v] = u;
            }
        }
    }
}
```



```

        pq.push({res.dist[v], v});
    }
}
}
return res;
}

vector<int> restore_path(const vector<int> &pre, int s, int t) {
    vector<int> path;
    for (int cur = t; cur != 0; cur = pre[cur]) {
        path.push_back(cur);
        if (cur == s) break;
    }
    if (path.empty() || path.back() != s) return {};
    reverse(path.begin(), path.end());
    return path;
}

```

调用示例:

```

auto res = dijkstra_with_parent(G, s);
if (res.dist[t] == LINF) {
    cout << "-1\n";
} else {
    cout << res.dist[t] << "\n";
    auto path = restore_path(res.pre, s, t);
}

```

```

vector<int> sources = {1, 5, 9};
auto multi = dijkstra_multi_source(G, sources);

```

常见坑:

- 有负权边时会错。
- $LINF + w$ 可能溢出, 只有从堆中弹出的可达点才松弛。
- 堆里旧状态要用 $du \neq dist[u]$ 跳过。
- 无向图用 `G.add_undirected(...)`; 有向图用 `G.add_directed(...)`。
- 多源最短路不是跑多次 Dijkstra, 而是所有源点距离设 0 后一起入堆。
- `restore_path` 只能恢复到起点可达的目标。

暴力/部分替代:

- $n \leq 500$ 可用 Floyd。
- m 很小、点很少可用 Bellman-Ford。
- 边权全为 1 时用 BFS。

升级方向:

- 需要多次任意两点查询: 小图 Floyd; 大图通常要换模型。
- 需要方案数: 在 Dijkstra 松弛时维护 `cnt[v]`。
- 需要路径限制: 可能变成状态最短路, 把状态拆成点。

最小测试样例:

```

4 4
1 2 5
1 3 1
3 2 1
2 4 2
s=1
dist[4]=4
path: 1 3 2 4

```

MATH-01 gcd / lcm

模块编号: MATH-01

模块名称：最大公约数与最小公倍数

标签：[数学][数论][gcd][lcm]

一句话用途：快速处理整除、约分、周期同步、比例化简和两个数的公共因子问题。

题面触发词：

- 最大公约数、最小公倍数。
- 互质、约分、最简分数。
- 周期同时发生、两个循环何时重合。
- 能否整除、公共因子、最大公共长度。
- 把若干数按比例化简。

什么时候用：

- 题目明显问 $\gcd(a,b)$ 、 $\text{lcm}(a,b)$ 。
- 需要判断 $\gcd(a,b) == 1$ 。
- 需要把分数 x/y 约成最简。
- 周期题中要求两个周期第一次同时出现，常用 lcm 。

不要什么时候用：

- 需要所有因子列表时，只求 $\gcd$ 不够，要枚举因子或质因数分解。
- lcm 可能超过 long long 时不能直接乘。
- 浮点数比例不要直接 $\gcd$ ，先转成整数或避免浮点。

复杂度：

- $\gcd(a,b)$: $O(\log \min(a,b))$ 。
- 多个数的 $\gcd/\text{lcm}$ ：每加入一个数做一次 $\gcd$ 。

数据范围参考：

- $a, b \leq 1e18$: long long 可存， lcm 乘法要防溢出。
- $n \leq 2e5$: 顺序合并 $\gcd/\text{lcm}$ 可用。

依赖的标准容器：

- 不依赖特殊容器。
- 多个数时使用 1-index 数组 `vector<ll> a(n + 1)`。

输入如何整理：

```
int n;
cin >> n;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口：

`gcd_ll(a,b)` -> 最大公约数

`lcm_ll(a,b)` -> 最小公倍数，普通版

`lcm_limit(a,b,limit)` -> 超过 `limit` 时返回 `limit + 1`；若 `limit` 已经是 `LLONG_MAX`，则返回 `limit`

输出能力：

- 两数或多数组组合的最大公约数。
- 两数或多数组组合的最小公倍数。
- 互质判断。
- 分数约分。

下游可接：

- 逆元和模运算里的互质判断。
- 中国剩余类问题的前置检查。
- 周期 DP、模拟题、字符串周期题。

可拼接模块：

- MATH-02 模运算。
- MATH-03 逆元。
- MATH-04 质因数分解。
- STR-02 KMP/Z 函数求字符串周期后接 $\gcd/\text{lcm}$ 。

模数是否为质数的分支：

本模块本身不依赖模数。

如果后续要在 mod 下做除法：

mod 是质数 -> 可接费马逆元。

mod 不是质数 -> 必须检查 $\gcd(x, \text{mod}) == 1$ ，再用扩展 gcd 求逆元。

模板代码：

```
using ll = long long;

ll gcd_ll(ll a, ll b) {
    if (a == LLONG_MIN || b == LLONG_MIN) {
        // 极端数据兜底；普通竞赛题很少卡 LLONG_MIN。
        unsigned long long x = a < 0 ? 0ULL - (unsigned long long)a : (unsigned long long)a;
        unsigned long long y = b < 0 ? 0ULL - (unsigned long long)b : (unsigned long long)b;
        while (y) {
            unsigned long long t = x % y;
            x = y;
            y = t;
        }
        return (ll)x;
    }
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a;
}

ll lcm_ll(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    ll g = gcd_ll(a, b);
    __int128 x = (__int128)a / g * b;
    if (x < 0) x = -x;
    assert(x <= LLONG_MAX); // 若可能超过 Long Long, 改用 Lcm_limit。
    return (ll)x;
}

ll lcm_limit(ll a, ll b, ll limit) {
    if (a == 0 || b == 0) return 0;
    __int128 aa = a, bb = b;
    if (aa < 0) aa = -aa;
    if (bb < 0) bb = -bb;
    ll g = gcd_ll(a, b);
    aa /= g;
    __int128 lim = limit;
    ll over = (limit == LLONG_MAX ? limit : limit + 1);
    if (bb != 0 && aa > lim / bb) return over;
    __int128 res = aa * bb;
    if (res > lim) return over;
    return (ll)res;
}

调用示例：

ll g = 0;
for (int i = 1; i <= n; i++) g = gcd_ll(g, a[i]);

ll l = 1;
for (int i = 1; i <= n; i++) {
    l = lcm_limit(l, a[i], 4'000'000'000'000'000'000LL);
}
```

```

}

if (gcd_ll(x, y) == 1) {
    // x 和 y 互质
}

```

常见坑：

- $\text{lcm} = a * b / \text{gcd}(a, b)$ 容易先乘溢出，写成 $a / \text{gcd} * b$ 。
- $\text{gcd}(0, x) = \text{abs}(x)$ ，多数组合时初始值可设为 0。
- 负数取 gcd 时先转正；如果题目可能出现 `LLONG_MIN`，用本页安全版，不要直接 $a = -a$ 。
- C++17 有 `std::gcd`，但纸质模板自己写更可控。
- `lcm_ll` 只在结果保证不超过 `long long` 时使用；可能爆时用 `lcm_limit`。

暴力/部分替代：

- 小数据可以从 $\min(a, b)$ 往下枚举第一个同时整除的数求 gcd。
- 小数据可以从 $\max(a, b)$ 往上枚举第一个同时被整除的数求 lcm。
- 周期题不会推公式时，可先模拟到 `lcm_limit` 的上限拿部分分。

升级方向：

- 多次区间 gcd 查询 -> Sparse Table 或 Segment Tree。
- 需要因子个数/因子和 -> 质因数分解。
- 模意义下除法 -> 逆元。

最小测试样例：

```

gcd_ll(12, 18) = 6
lcm_ll(12, 18) = 36
gcd_ll(0, 5) = 5
lcm_limit(1000000000000, 1000000000000, 1000000000000) = 1000000000000

```

SIM-01 模拟、字符串扫描与高精度

模块编号：SIM-01

模块名称：模拟、字符串扫描与高精度

标签：模拟、字符串、高精度、大整数、非负整数、考场模板

一句话用途：题面让你“按规则一步一步做”或整数超过 `long long` 时，用字符串和稳定循环先拿分。

题面触发词：

- 模拟过程、按顺序执行操作、每一轮变化。
- 字符串表示的数字、位数很大、结果很大。
- 高精度加法、高精度减法、高精度乘小整数。
- 不能使用内置大整数。

什么时候用：

- 规则直接，按题面顺序执行即可。
- 数字长度可能超过 18 位，`long long` 会溢出。
- 只涉及非负大整数的加、减、比较、乘小整数。
- 需要把字符逐个扫描，统计、替换、进位或借位。

不要什么时候用：

- 大整数需要乘大整数、除法、取模很多次，本模块只提供基础版本。
- 题目本质是 DP、图论、贪心，模拟只是读入或输出辅助。
- 数字长度很小且保证在 `long long` 内，直接整数更快更短。

复杂度：

- 字符串扫描： $O(n)$ 。
- 大整数比较： $O(\text{len})$ 。
- 大整数加法： $O(\text{len})$ 。
- 大整数减法： $O(\text{len})$ ，要求 $a \geq b$ 。
- 大整数乘小整数： $O(\text{len} * \text{位运算常数})$ ，通常记 $O(\text{len})$ 。

数据范围参考：

- 位数 $\leq 1e5$ ：字符串高精度可用。
- 操作次数很多时，总复杂度按“总位数扫描次数”估算。
- 乘数 k 用 `long long` 存，题面保证是小整数时使用。

依赖的标准容器：

- `string`：存非负大整数。
- `vector<int>`：存操作、方向或状态，数组默认 1-index。

输入如何整理：

```
string a, b;
cin >> a >> b;
a = strip0(a);
b = strip0(b);

int n;
cin >> n;
vector<int> op(n + 1);
for (int i = 1; i <= n; i++) cin >> op[i];
```

模拟整理顺序：

1. 把题面状态写成变量或数组。
2. 把每一步操作写成一个循环。
3. 把边界判断写成 `inside/check` 函数。
4. 字符串数字先 `strip0`，再比较或计算。

接口：

`strip0(s)` -> 去掉前导零，空结果返回 `"0"`。
`cmp_big(a,b)` -> 比较非负大整数，返回 `-1/0/1`。
`add_big(a,b)` -> 非负大整数加法。
`sub_big(a,b)` -> 非负大整数减法，要求 $a \geq b$ 。
`mul_small(a,k)` -> 非负大整数乘 `long long` 小整数 k ， k 可为负。
`inside(x,y,n,m)` -> 网格模拟边界判断。

输出能力：

- 输出模拟后的状态。
- 输出大整数计算结果。
- 输出比较结果。

下游可接：

- STR-01 基础字符串操作。
- BASIC-00 控制结构。
- BRUTE 部分模拟。
- MATH 快速幂或取模模块。

可拼接模块：

- 大整数输入接 `strip0`。
- 高精度加减接模拟计数。
- 字符串扫描接 KMP/Hash 前的预处理。
- 小数据模拟接暴力部分。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

string strip0(string s) {
    int i = 0;
    while (i + 1 < (int)s.size() && s[i] == '0') {
        i++;
    }
}
```

```

    s = s.substr(i);
    if (s.empty()) return "0";
    return s;
}

int cmp_big(string a, string b) {
    a = strip0(a);
    b = strip0(b);
    if (a.size() != b.size()) {
        return a.size() < b.size() ? -1 : 1;
    }
    if (a == b) return 0;
    return a < b ? -1 : 1;
}

string add_big(string a, string b) {
    a = strip0(a);
    b = strip0(b);
    int i = (int)a.size() - 1;
    int j = (int)b.size() - 1;
    int carry = 0;
    string res;

    while (i >= 0 || j >= 0 || carry) {
        int x = carry;
        if (i >= 0) x += a[i--] - '0';
        if (j >= 0) x += b[j--] - '0';
        res.push_back(char('0' + x % 10));
        carry = x / 10;
    }

    reverse(res.begin(), res.end());
    return strip0(res);
}

string sub_big(string a, string b) {
    a = strip0(a);
    b = strip0(b);
    int i = (int)a.size() - 1;
    int j = (int)b.size() - 1;
    int borrow = 0;
    string res;

    while (i >= 0) {
        int x = (a[i] - '0') - borrow;
        int y = (j >= 0 ? b[j] - '0' : 0);
        if (x < y) {
            x += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res.push_back(char('0' + (x - y)));
        i--;
        j--;
    }

    reverse(res.begin(), res.end());
    return strip0(res);
}

string mul_small(string a, ll k) {
    a = strip0(a);

```

```

if (a == "0" || k == 0) return "0";

__int128 mag = k;
bool neg = false;
if (mag < 0) {
    neg = true;
    mag = -mag;
}

__int128 carry = 0;
string res;
for (int i = (int)a.size() - 1; i >= 0; i--) {
    __int128 cur = (__int128)(a[i] - '0') * mag + carry;
    res.push_back(char('0' + cur % 10));
    carry = cur / 10;
}
while (carry > 0) {
    res.push_back(char('0' + carry % 10));
    carry /= 10;
}

reverse(res.begin(), res.end());
string ans = strip0(res);
return neg ? "-" + ans : ans;
}

bool inside(int x, int y, int n, int m) {
    return 1 <= x && x <= n && 1 <= y && y <= m;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string op;
    cin >> op;

    if (op == "add") {
        string a, b;
        cin >> a >> b;
        cout << add_big(a, b) << '\n';
    } else if (op == "sub") {
        string a, b;
        cin >> a >> b;
        if (cmp_big(a, b) < 0) {
            cout << '-' << sub_big(b, a) << '\n';
        } else {
            cout << sub_big(a, b) << '\n';
        }
    } else if (op == "mul") {
        string a;
        ll k;
        cin >> a >> k;
        cout << mul_small(a, k) << '\n';
    } else if (op == "cmp") {
        string a, b;
        cin >> a >> b;
        cout << cmp_big(a, b) << '\n';
    }

    return 0;
}

```

调用示例:

```
string a = "000999";
string b = "1";

cout << strip0(a) << '\n';           // 999
cout << add_big(a, b) << '\n';       // 1000
cout << sub_big("1000", "1") << '\n'; // 999
cout << mul_small("123", 45) << '\n'; // 5535
cout << cmp_big("0010", "9") << '\n'; // 1
```

// 按字符模拟进位或统计。

```
string s;
cin >> s;
int cnt = 0;
for (int i = 0; i < (int)s.size(); i++) {
    if (isdigit((unsigned char)s[i])) cnt++;
}
```

常见坑:

- 高精度数字必须当字符串读, 不能先读进 long long。
- 减法模板要求 $a \geq b$; 如果题目可能为负, 要先比较并输出负号。
- `strip0("0000")` 必须返回 "0"。
- 字符转数字用 `c - '0'`, 数字转字符用 `char('0' + x)`。
- 乘小整数时 k 的绝对值和进位用 `__int128`, 避免 `LLONG_MIN` 取负和中间乘法溢出。
- 字符串下标是 0-index, 普通数组按本书约定优先 1-index。
- `isdigit` 传入非 ASCII 字符时建议转 `unsigned char`。
- 多次模拟操作时, 每组数据要重置状态和答案。

暴力/部分替代:

- 位数 ≤ 18 的子任务可先用 long long。
- 乘小整数很小, 可用重复加法拿小数据。
- 复杂模拟不会优化时, 先按题面逐步执行, 拿小范围分。
- 大整数只需要比较大小时, 不要写完整加减, 先写 `strip0 + cmp_big`。

升级方向:

long long 溢出 -> string 高精度
重复加法乘小数 -> `mul_small`
大量取模 -> 边读边取模
大整数乘大整数 -> 另写 $O(nm)$ 乘法或 FFT, 低优先级

最小测试样例:

输入
add
999
1

输出
1000

补充自测:

`cmp 0010 9 -> 1`
`sub 1000 1 -> 999`
`mul 123 45 -> 5535`
`add 0000 000 -> 0`

STR-01 string 常用操作

模块编号: STR-01

模块名称: C++ string 常用操作与索引转换

标签: [字符串][string][基础操作][0-index]

一句话用途: 统一字符串读入、截取、查找、拼接、排序和 0-index/1-index 转换, 减少低级错误。

索引约定:

本模块内部使用 C++ string 自然 0-index: 位置 $0..n-1$ 。

题面若给第 k 个字符, 通常是 1-index: 读入后用 $k--$ 。

子串 `substr(pos, len)` 的 `pos` 是 0-index, `len` 是长度, 不是右端点。

若题面给闭区间 $[l, r]$ 且为 1-index, 则 C++ 写 `s.substr(l-1, r-l+1)`。

题面触发词:

- 字符串处理、字符替换、统计字符。
- 子串、前缀、后缀。
- 字典序排序。
- 翻转、拼接、删除、插入。
- 大小写转换。

什么时候用:

- 题目只需要简单字符串操作, 不需要 KMP/Hash。
- 字符串长度不大, 可以直接用 `substr/find`。
- 需要把题面 1-index 位置转成 C++ 0-index。

不要什么时候用:

- n, m 很大且要重复匹配子串, `find/substr` 可能 TLE。
- 需要大量判断任意子串是否相等, 优先 Rolling Hash。
- 多模式串匹配, 优先 Trie/AC 自动机。

复杂度:

- `s.size()`: $O(1)$ 。
- `s.substr(pos, len)`: $O(len)$ 。
- `s.find(t)`: 通常可用但最坏不作为算法保证。
- 排序字符串数组: $O(\text{总比较成本} * \log n)$ 。

数据范围参考:

- $|s| \leq 1e5$: 一次线性扫描没问题。
- 重复 `substr` 复制总长度可能到 $O(n^2)$, 要小心。

依赖的标准容器:

- `string`。
- `vector<int>` / `vector<string>`。
- 字符计数常用 `array<int, 26>` 或 `vector<int>(256)`。

输入如何整理:

```
string s;  
cin >> s; // 无空格字符串
```

```
string line;  
getline(cin, line); // 含空格整行, 注意先处理上一行换行
```

接口:

```
to0(pos1) -> 题面 1-index 转 0-index  
substr_lidx(s, l, r) -> 题面 1-index 闭区间子串  
count_lower(s) -> 统计小写字母  
is_prefix(s, t) -> t 是否为 s 的前缀  
is_suffix(s, t) -> t 是否为 s 的后缀
```

输出能力:

- 字符频次。
- 子串、前缀、后缀。
- 字典序比较。
- 简单模拟修改后的字符串。

下游可接:

- KMP/Z 函数。
- Trie。
- Rolling Hash。
- STR-05 Manacher。

可拼接模块：

- STR-02 KMP/Z。
- STR-03 Trie/Rolling Hash。
- STR-05 Manacher。
- DP LCS/编辑距离。

模板代码：

```
int to0(int pos1) {
    return pos1 - 1;
}

string substr_lidx(const string &s, int l, int r) {
    if (l < 1 || r < 1 || r > (int)s.size()) return "";
    return s.substr(l - 1, r - l + 1);
}

array<int, 26> count_lower(const string &s) {
    array<int, 26> cnt{};
    for (char c : s) {
        if ('a' <= c && c <= 'z') cnt[c - 'a']++;
    }
    return cnt;
}

bool is_prefix(const string &s, const string &t) {
    if (t.size() > s.size()) return false;
    for (int i = 0; i < (int)t.size(); i++) {
        if (s[i] != t[i]) return false;
    }
    return true;
}

bool is_suffix(const string &s, const string &t) {
    int n = (int)s.size(), m = (int)t.size();
    if (m > n) return false;
    for (int i = 0; i < m; i++) {
        if (s[n - m + i] != t[i]) return false;
    }
    return true;
}
```

调用示例：

```
string s = "abcdef";
int l = 2, r = 4; // 题面 1-index
cout << substr_lidx(s, l, r) << "\n"; // bcd

auto cnt = count_lower(s);
cout << cnt['a' - 'a'] << "\n";
```

常见坑：

- `s[i]` 是 0-index。
- `substr(pos, len)` 第二个参数是长度，不是右端点。
- `getline` 前如果刚用过 `cin >> x`，要吃掉换行。
- `char` 可能是 signed，做 ASCII 桶建议转 `unsigned char` 或用 `vector<int>(256)`。
- 循环写 `i < s.size() - 1` 时，空串会让无符号减法出事；先转 `int n=s.size()`。

暴力/部分替代：

- 子串匹配小数据：从每个位置开始逐字符比较。

- 子串相等小数据：直接 substr 比较。
- 多次修改小数据：直接改 string。

升级方向：

- 单模式匹配大数据 -> KMP/Z。
- 多模式前缀统计 -> Trie。
- 任意子串相等 -> Rolling Hash。
- 回文子串 -> STR-05 Manacher 或中心扩展。

最小测试样例：

```
s = abcdef
题目 [2,4] -> substr_idx = bcd
is_prefix(abcdef, abc) = true
is_suffix(abcdef, def) = true
```

STR-05 Manacher 回文算法

模块编号：STR-05

模块名称：Manacher 回文半径、最长回文子串、区间回文判断

标签：[字符串][回文][Manacher][最长回文子串][1-index]

一句话用途：在线性时间 $O(n)$ 求每个中心能扩展出的最长回文半径，适合长字符串的最长回文子串、统计回文子串数量、大量区间回文判断。

索引约定：

本模板把原字符串 raw 转成 1-index 字符串 $s = " " + raw$ 。
 $s[1..n]$ 是真实字符。

$d1[i]$ ：以 i 为中心的奇数回文半径。

覆盖区间： $[i - d1[i] + 1, i + d1[i] - 1]$ 。

对应最长奇数回文长度： $2 * d1[i] - 1$ 。

$d2[i]$ ：以 $i-1$ 和 i 中间为中心的偶数回文半径。

覆盖区间： $[i - d2[i], i + d2[i] - 1]$ 。

对应最长偶数回文长度： $2 * d2[i]$ 。

题目如果给 1-index 区间 $[l, r]$ ，直接调用 $is_pal(l, r)$ 。

题目触发词：

- 最长回文子串。
- 回文半径。
- 回文子串数量。
- 很多次询问 $s[1..r]$ 是否为回文。
- 字符串长度很大，中心扩展 $O(n^2)$ 可能超时。

什么时候用：

- n 到 $1e5$ 、 $1e6$ 级别，需要处理所有回文中心。
- 回文查询次数很多，不能每次双指针检查。
- 需要把“某段是不是回文”作为 DP 或枚举的快速判断条件。

不要什么时候用：

- 只判断一个字符串整体是否回文：直接双指针或 reverse。
- 只做几次短区间回文判断：直接检查更快写。
- 需要动态修改字符串后再查回文：Manacher 是静态预处理，修改后要重建。
- 题目是“最长回文子序列”：那是 DP，不是 Manacher。

复杂度：

- 预处理： $O(n)$ 。
- 最长回文子串：预处理后 $O(n)$ 扫一遍。
- 单次区间回文判断： $O(1)$ 。
- 统计回文子串数量： $O(n)$ ，答案可能需要 long long。

数据范围参考:

数据范围	建议
$n \leq 2000$	中心扩展或区间 DP 都能尝试
$n \leq 1e5$	Manacher 稳
$n \leq 1e6$	Manacher + 静态全局数组, 避免反复分配

输入如何整理:

```
string raw;
cin >> raw;
build_manacher(raw);
```

接口:

build_manacher(raw): 预处理 d1/d2。
is_pal(l,r): 判断 l-index 闭区间 [l,r] 是否为回文。
longest_pal_len(): 返回最长回文子串长度。
count_pal_substrings(): 返回回文子串总数。

模板代码:

```
const int MAXN = 1000000 + 5;

int n;
string s;           // s[1..n]
int d1[MAXN];       // odd radius
int d2[MAXN];       // even radius, center between i-1 and i

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) {
            k++;
        }
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) {
            k++;
        }
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

bool is_pal(int l, int r) {
    if (l > r) return true;
    if (l < 1 || r > n) return false;
```

```

int len = r - l + 1;
if (len & 1) {
    int mid = (l + r) / 2;
    return d1[mid] >= len / 2 + 1;
} else {
    int mid = (l + r + 1) / 2;
    return d2[mid] >= len / 2;
}
}

```

```

int longest_pal_len() {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, 2 * d1[i] - 1);
        ans = max(ans, 2 * d2[i]);
    }
    return ans;
}

```

```

long long count_pal_substrings() {
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        ans += d1[i];
        ans += d2[i];
    }
    return ans;
}

```

完整可运行代码 1: 最长回文子串长度

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000 + 5;

int n;
string s;
int d1[MAXN], d2[MAXN];

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) k++;
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

```

```

    }
}

int longest_pal_len() {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = max(ans, 2 * d1[i] - 1);
        ans = max(ans, 2 * d2[i]);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string raw;
    cin >> raw;
    build_manacher(raw);
    cout << longest_pal_len() << "\n";
    return 0;
}

```

完整可运行代码 2: 多次判断区间是否回文

```

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000 + 5;

int n;
string s;
int d1[MAXN], d2[MAXN];

void build_manacher(const string &raw) {
    n = (int)raw.size();
    s = " " + raw;

    int l = 1, r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 1 && i + k <= n && s[i - k] == s[i + k]) k++;
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    l = 1;
    r = 0;
    for (int i = 1; i <= n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 1 && i + k <= n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
}

bool is_pal(int l, int r) {
    if (l > r) return true;
}

```

```

    if (l < 1 || r > n) return false;
    int len = r - l + 1;
    if (len & 1) {
        int mid = (l + r) / 2;
        return d1[mid] >= len / 2 + 1;
    }
    int mid = (l + r + 1) / 2;
    return d2[mid] >= len / 2;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string raw;
    cin >> raw;
    build_manacher(raw);

    int q;
    cin >> q;
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << (is_pal(l, r) ? "YES" : "NO") << "\n";
    }
    return 0;
}

```

调用示例:

```

string raw = "abacaba";
build_manacher(raw);
cout << longest_pal_len() << "\n";           // 7
cout << is_pal(1, 7) << "\n";               // true
cout << count_pal_substrings() << "\n";     // 所有回文子串个数

```

常见坑:

- d1[i] 和 d2[i] 是半径, 不是长度。
- 奇数回文中心是一个字符 i; 偶数回文中心是缝隙, 在 i-1 和 i 之间。
- 题面如果给的是 1-index 区间, 本模板可以直接用; 不要再 l--, r--。
- 空格、中文等复杂字符按字节处理; 算法竞赛通常是小写字母或 ASCII 字符。
- count_pal_substrings() 可能到 $n*(n+1)/2$, 必须用 long long。
- Manacher 只能处理静态字符串, 字符串修改后必须重建。

暴力/部分分替代:

```

bool slow_pal(const string &raw, int l, int r) {
    // raw 是普通 0-index string, 题面 [l,r] 是 1-index.
    l--;
    r--;
    while (l < r) {
        if (raw[l] != raw[r]) return false;
        l++;
        r--;
    }
    return true;
}

```

- $n \leq 2000$: 可以枚举中心向两边扩展, 求最长回文。
- q 很小: 每次双指针判断区间, 先拿部分分。
- 题目是 “最少切成若干回文串”: 先用 Manacher 或 slow_pal 得到 is_pal(l,r), 再接 DP。

最小测试样例 1:

输入:
babad

输出:

3

最小测试样例 2:

输入:

abacaba

4

1 7

2 4

2 6

3 5

输出:

YES

NO

YES

YES

TRAIN-00 第 7 卷: 通用 WA / RE / TLE 自检

模块编号: TRAIN-00

模块名称: 通用 WA / RE / TLE 自检

标签: 调试、提交前检查、WA、RE、TLE、部分分

一句话用途: 提交前用 3 分钟扫掉最常见错误, 失败后能快速判断是模型错、边界错还是复杂度错。

题面触发词: 样例过了但不放心、隐藏点 WA、运行错误、超时、多组数据、边界、无解、极大数据。

什么时候用: 每次提交前; 从暴力升级到优化版后; 出现 WA/RE/TLE 后不知道从哪查时。

不要什么时候用: 不要用本清单代替重新读题; 如果题面模型已经选错, 改小 bug 救不了正解。

复杂度: 检查清单本身无复杂度。

数据范围参考: 所有题目。

依赖的标准容器: Array、Graph、Query、State、Compressor, 以及各卷模块的统一接口。

输入如何整理: 先圈 n/m/q/T/值域/是否多测/是否有修改/边权符号/是否取模/无解输出。

接口: 提交前检查顺序。

输出能力: 定位 WA、RE、TLE 高风险点, 并给出回退到部分分版本的路线。

下游可接: TRAIN-01 反例库、TRAIN-02 路由训练、05_review 质检。

可拼接模块: ROUTE-00、ROUTE-01、CPP-009、BRUTE-15、DP 常见坑、DS/GRAPH/MATH/STR 模块。

模板代码: 无。本模块是检查卡。

调用示例: 样例 AC 后, 按 “WA -> RE -> TLE -> 最小反例 -> 提交版本” 顺序扫一遍。

常见坑: 只测样例; 只测随机不测边界; 升级版改坏输入层; 忘记多测清空; 答案类型用 int。

暴力/部分分替代: 如果优化版不稳, 保留暴力或记忆化版本, 先提交能过小数据的版本。

升级方向: 把最小反例加入本地自测; 用暴力和优化版对拍小数据; 把可疑高级结构退回稳妥结构。

最小测试样例: n=1、空/无解、最大值、全相等、重复边/重复值、多组数据两组连续。

1. 90 秒错误定位

现象	第一判断	先查什么	常见处理
样例 WA	读题或输出格式错	下标、是否多测、答案位置	手算样例, 打印核心数组
小极端 WA	边界错	n=1、0、空集、无解、负数	加特判或改初始化
随机小数据 WA	模型/转移错	与暴力对拍	回退到暴力, 逐步升级
大数据 WA	溢出或复杂边界	long long、INF、取模、压缩	换 11, 检查边界

现象	第一判断	先查什么	常见处理
RE	越界/空容器/爆栈	数组大小、front/top/back、递归深度	加边界，改迭代或显式栈
TLE	复杂度不够	$n*q$ 、 n^2 、重复 DFS	换路由，先 memo / 树状数组 / BFS

2. 通用 WA 检查

检查项	典型错误	最小验错
题目目标	求最小却写最大；求方案数却输出最优值	手算 2-3 个元素
数据范围	$n=2e5$ 还写双重循环	最大范围估复杂度
下标体系	数组 1-index，字符串 0-index 混用	$l=1$ 、 $r=n$ 、子串首尾
初始化	最大值题 $ans=0$ ，全负数错	全负数组
答案位置	背包输出 $dp[w]$ 还是 $\max dp[j]$	容量不必装满
无解输出	INF 直接打印	不连通、不可达、容量不足
多组数据	图、memo、队列、数组未清空	两组数据，第二组很小
取模	加法/乘法中途溢出或忘 %MOD	答案超过 MOD
排序比较	cmp 用 $<=$	相等元素
图方向	有向边当无向，或反过来	2 点 1 条单向边
边权	有负边还用 Dijkstra	3 点负边反例
重复值/重边	压缩、最短路、MST 未处理	两条不同权重重边
DP 循环顺序	0/1 背包正序导致重复选	一个物品被选两次

3. 通用 RE 检查

检查项	典型错误	快速修法
数组大小	开 n 却访问 n 或 $n+1$	1-index 开 $n+2$
差分	$diff[r+1]$ 越界	开 $n+2$ ，或判断 $r < n$
树状数组	$add(0, x)$ 死循环	压缩 id 从 1 开始
空容器	$q.front()$ / $st.top()$ / $v.back()$	访问前判 $empty()$
递归深度	链状树 DFS 爆栈	改迭代或确认平台栈
位运算	$1 \ll n$ 当 $n \geq 31$	用 $1LL \ll n$ ，且 n 不要太大
除法取模	除以 0 或逆元不存在	先判分母
vector 尺寸	负数或过大转成 unsigned	检查输入和内存
Floyd/DP 内存	$5000*5000$ 11 爆内存	换算法或滚动数组

4. 通用 TLE 检查

检查项	危险写法	升级方向
区间查询	每问循环 $[1, r]$ ， nq	PrefixSum / 树状数组 / SegmentTree
区间修改	每次循环加	Difference / LazySegmentTree
DFS	重复状态未缓存	memo / DP / BFS
最短路	n 次 Dijkstra 或 Floyd 用在大图	按询问数量重选模型
Dijkstra	堆里旧状态不跳过	<code>if (du != dist[u]) continue</code>
字符串匹配	每个位置重新比较模式串	KMP / Z / Hash
map 太慢	状态很多还用 <code>map<tuple></code>	数组 memo 或安全编码
输出太慢	每次 endl 刷新	用 <code>'\n'</code>

5. 提交前 7 问

1. 我这版能过哪些数据范围？如果不能满分，部分分边界在哪里？
2. 是否看清多组数据，所有全局容器是否清空？
3. 所有答案、距离、方案数、乘法中间量是否用 long long？
4. 是否处理 $n=1$ 、空/无解、全相等、负数、重复值？
5. 图题是否确认方向、边权符号、是否连通、是否有重边自环？
6. DP 是否确认初始化、循环顺序、答案位置、不可达值？
7. 优化版不稳时，是否保留了能拿小数据的暴力/记忆化版本？

6. 失败后的回退路线

WA:

小数据先写暴力对拍；没有暴力就手造极端样例。

RE:

先关掉优化和递归深处，查越界、空容器、多测清空。

TLE:

先确认复杂度是否根本不够；不够就按 ROUTE-00 重路由。

考场口令:

先保分，再修正解。

如果高级版 10 分钟查不出错，先交低版本。